



# CHƯƠNG 5

## TẦNG LIÊN KẾT VÀ MẠNG CỤC BỘ

---

NHẬP MÔN MẠNG MÁY TÍNH



## A note on the use of these PowerPoint slides:

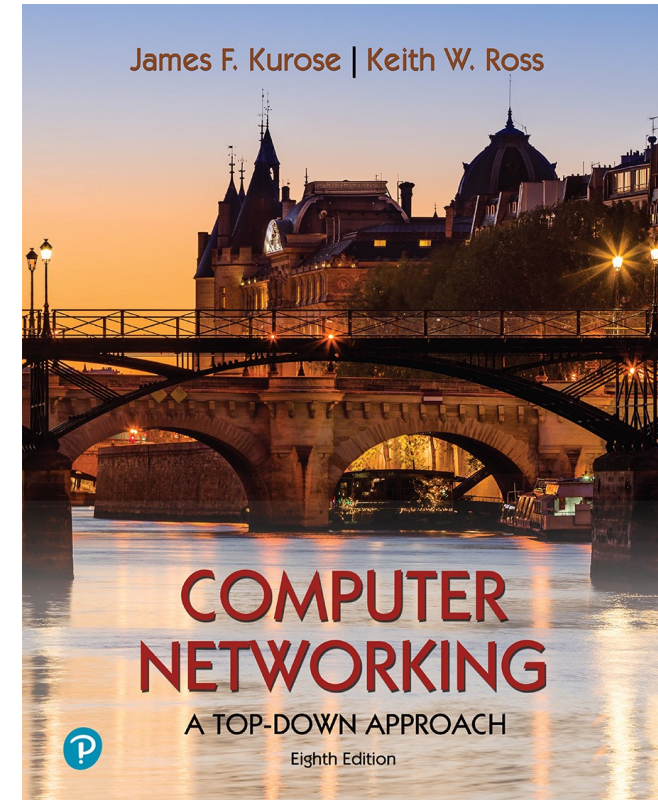
We're making these slides freely available to all (faculty, students, readers). They're in PowerPoint form so you see the animations; and can add, modify, and delete slides (including this one) and slide content to suit your needs. They obviously represent a *lot* of work on our part. In return for use, we only ask the following:

- If you use these slides (e.g., in a class) that you mention their source (after all, we'd like people to use our book!)
- If you post any slides on a www site, that you note that they are adapted from (or perhaps identical to) our slides, and note our copyright of this material.

For a revision history, see the slide note for this page.

Thanks and enjoy! JFK/KWR

All material copyright 1996-2020  
J.F Kurose and K.W. Ross, All Rights Reserved



## *Computer Networking: A Top-Down Approach*

8<sup>th</sup> edition

Jim Kurose, Keith Ross  
Pearson, 2020

# Tầng liên kết và LAN: mục tiêu



- Hiểu các nguyên lý bên trong các dịch vụ của tầng liên kết, đó là:
  - Phát hiện lỗi, sửa lỗi
  - Chia sẻ một kênh broadcast: Đa truy cập
  - Địa chỉ tầng liên kết
  - Mạng cục bộ: Ethernet, VLAN



# Tầng liên kết, LAN: nội dung chính

- Giới thiệu
- Phát hiện lỗi, sửa lỗi
- Các giao thức đa truy cập
- Mạng LAN
  - Địa chỉ MAC, ARP
  - Ethernet
  - Switch
  - VLAN
- Ngữ cảnh của một yêu cầu dịch vụ web



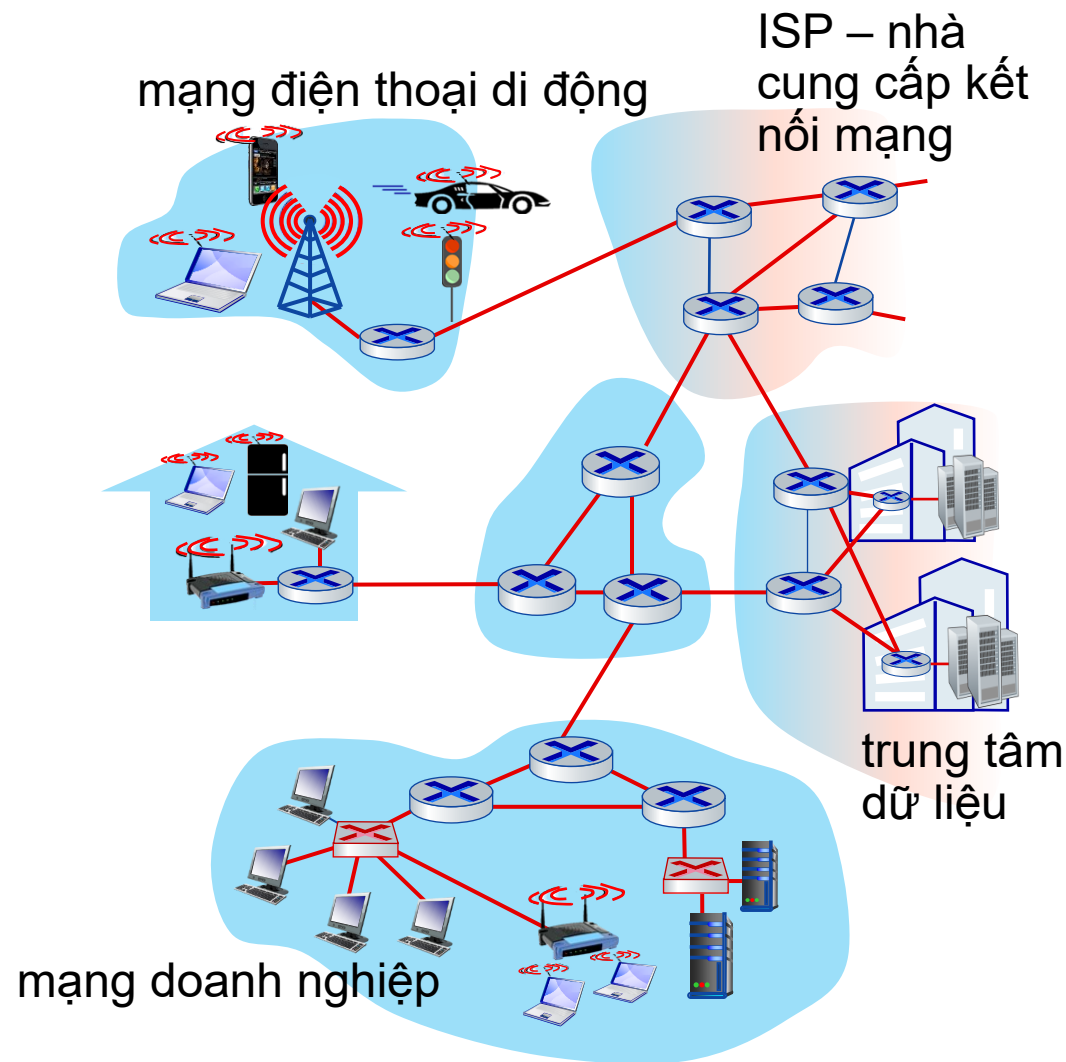


# Tầng liên kết: giới thiệu



Các thuật ngữ:

- Tầng liên kết: Data link layer
- Host và router: các nút (node)
- Liên kết (links): Các kênh liên lạc kết nối các nút lân cận với nhau.
  - Kết nối có dây
  - Kết nối không dây
  - LANs
- Gói tin thuộc tầng 2: *frame* hoặc “encapsulate datagram”



*Tầng liên kết* có nhiệm vụ chuyển datagram từ một nút đến một nút *liền kề vật lý* qua một đường liên kết



# Tầng liên kết: ngữ cảnh hoạt động

- Datagram được truyền bởi các giao thức liên kết khác nhau qua các liên kết khác nhau:
  - ví dụ: Đoạn đầu liên kết qua WiFi, đoạn tiếp theo liên kết qua Ethernet
- Mỗi giao thức liên kết có thể cung cấp nhiều dịch vụ khác nhau
  - ví dụ: một giao thức liên kết có thể cung cấp dịch vụ truyền dữ liệu tin cậy hoặc dịch vụ truyền không tin cậy

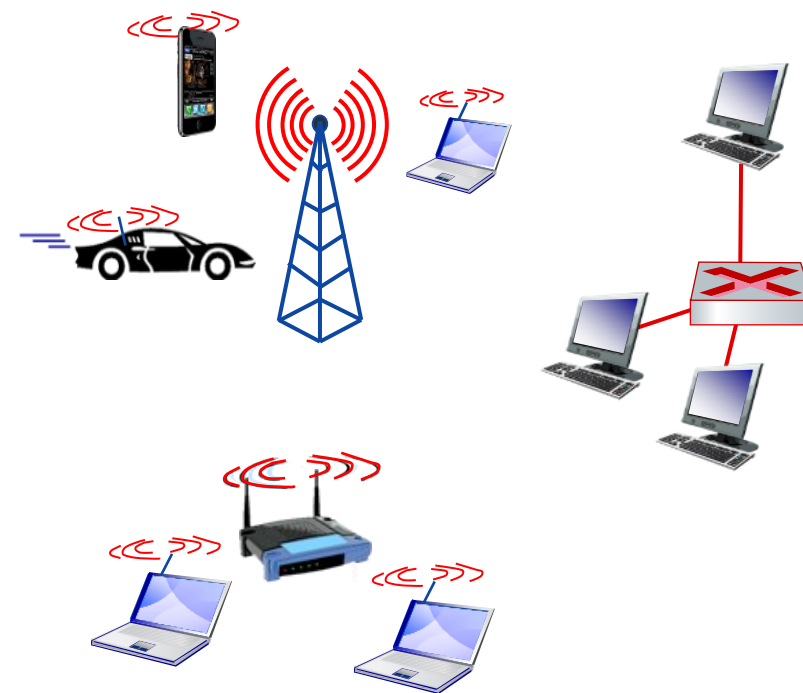
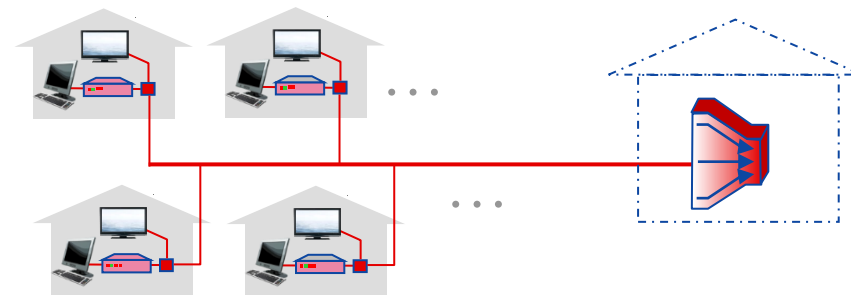
Tương tự như hệ thống giao thông vận tải:

- Chuyển đi từ Princeton đến Lausanne có thể sử dụng các phương tiện:
  - xe limo: Princeton đến JFK
  - máy bay: JFK đến Geneva
  - xe lửa: Geneva đến Lausanne
- Khách du lịch = **datagram**
- Các đoạn vận chuyển = **liên kết truyền thông**
- Cách thức vận chuyển = **giao thức tầng liên kết**
- Đại lý du lịch = **các thuật toán định tuyến**

# Tầng liên kết: Các dịch vụ



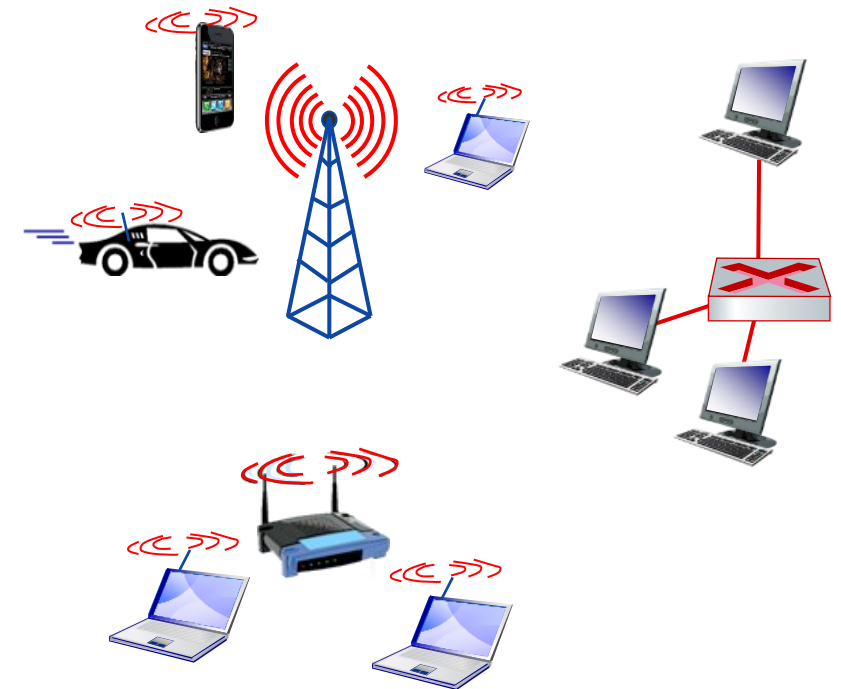
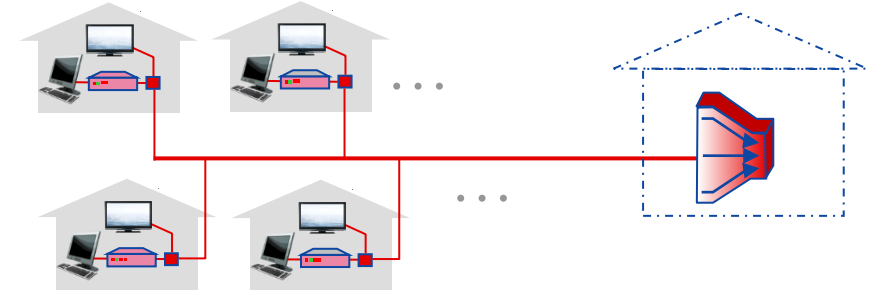
- **Framing, Truy cập kênh:** đóng gói datagram vào frame, thêm header và trailer
- **Các liên kết truy cập:**
  - Truy cập kênh truyền (chọn kênh) nếu là môi trường dùng chung.
  - Địa chỉ “MAC” trong header của frame được dùng để xác định nguồn gửi, đích tới (khác với địa chỉ IP!)
- **Truyền tin cậy giữa các nút liên kết**
  - Chúng ta đã biết làm thế nào để làm điều này!
  - Hiếm khi được sử dụng trên các liên kết có lỗi bit thấp
  - Liên kết không dây: tỷ lệ lỗi cao
    - Hỏi: tại sao cần cả độ tin cậy ở cấp độ liên kết và ở cấp độ end-to-end?



# Tầng liên kết: Các dịch vụ (tiếp theo)



- **Kiểm soát lưu lượng:**
  - Tạo sự đồng bộ giữa các nút gửi và nút nhận liên kề
- **Phát hiện lỗi:**
  - Lỗi do suy giảm tín hiệu, nhiễu.
  - Bên nhận phát hiện lỗi; quá trình truyền lại tín hiệu hoặc bỏ dữ liệu
- **Sửa lỗi:**
  - Bên nhận xác định lỗi *và sửa* (các) bit lỗi mà không cần truyền lại
- **Truyền bán song công (Half duplex) và song công (Full-duplex):**
  - Với truyền bán song công, các nút ở cả hai đầu của liên kết đều có thể truyền nhưng không đồng thời

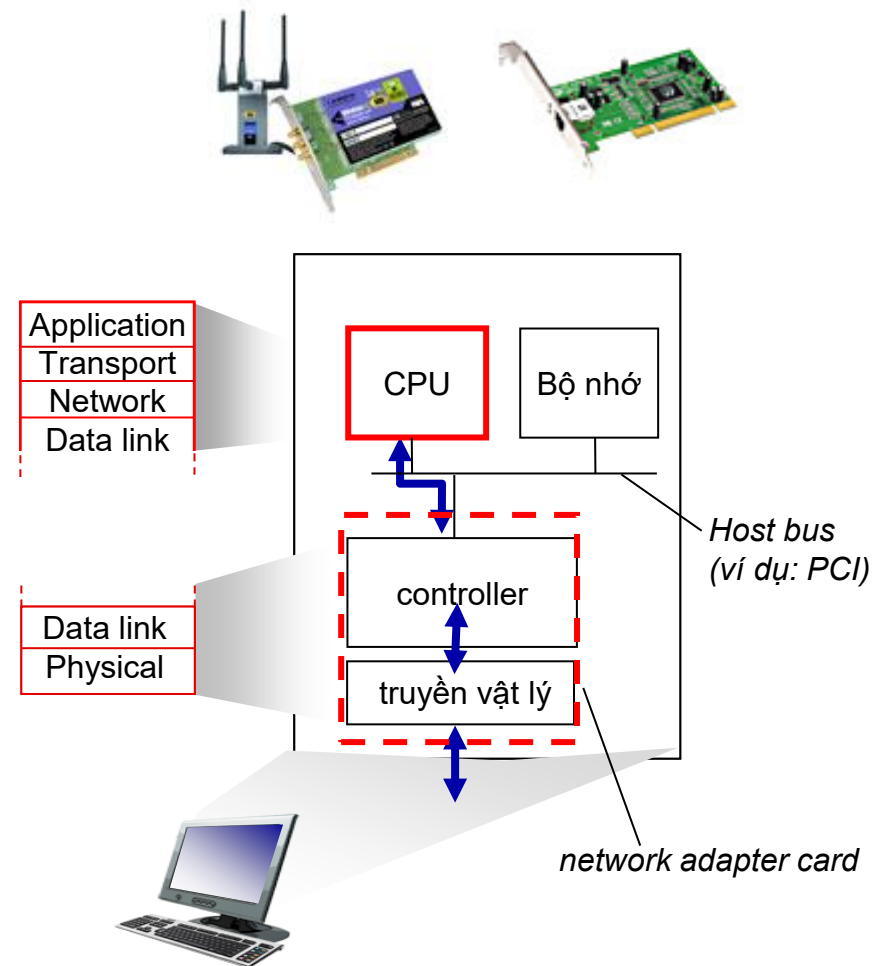




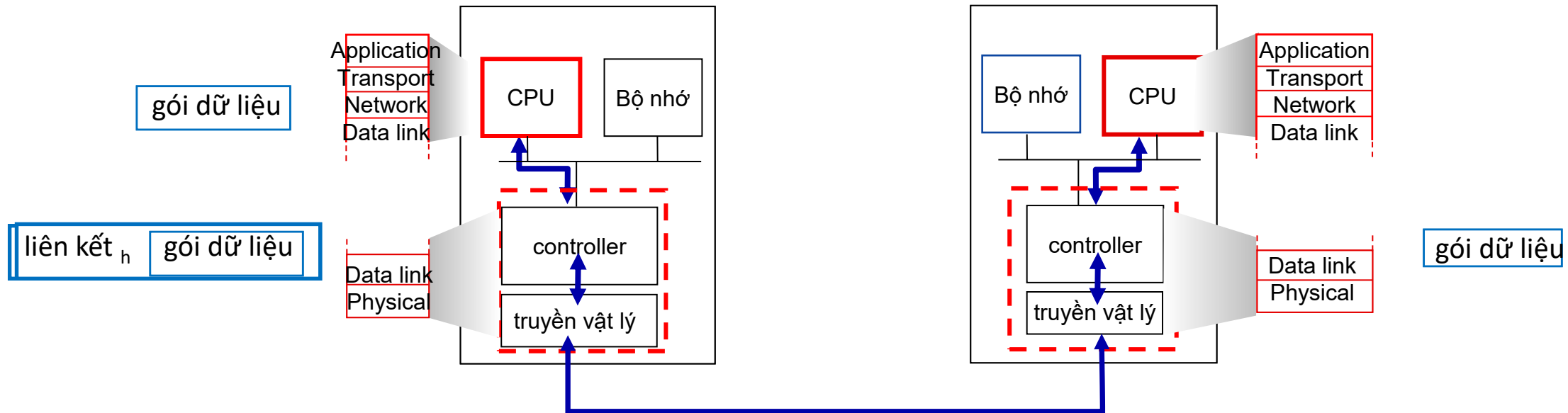
# Tầng liên kết được triển khai ở đâu?



- Trong mỗi host
- Tầng liên kết được triển khai trong *card mạng* (network interface card - NIC) hoặc trên các chip
  - Ethernet, card WiFi hoặc chip
  - Triển khai cả Tầng liên kết và Tầng vật lý
- Gắn vào các bus hệ thống của host
- Hoặc tổ hợp của hardware, software, firmware



# Giao diện giao tiếp



Bên gửi:

- Đóng gói datagram vào frame
- Thêm các bit kiểm tra lỗi, truyền dữ liệu đáng tin cậy, kiểm soát luồng, v.v.

Bên nhận:

- Tìm lỗi, truyền dữ liệu đáng tin cậy, kiểm soát luồng, v.v.
- Trích xuất datagram và chuyển lên tầng trên (tầng network)

# Tầng liên kết, LAN



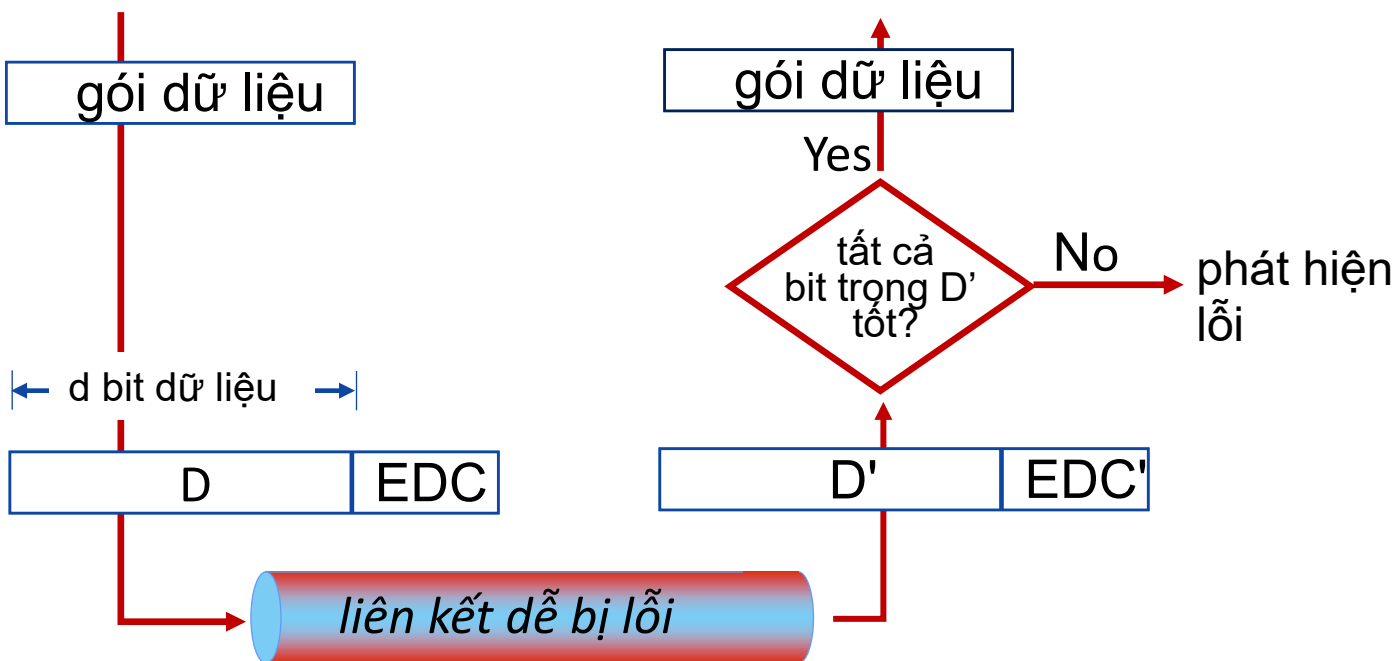
- Giới thiệu
- Phát hiện lỗi, sửa lỗi
- Nhiều giao thức truy cập
- Mạng LAN
  - Định địa chỉ, ARP
  - Ethernet
  - Switch
  - VLAN
- Ngữ cảnh của một yêu cầu dịch vụ web



# Phát hiện lỗi



**EDC** (error detection and correction): các bit phát hiện và sửa lỗi  
**D** (data): dữ liệu được bảo vệ bằng cách kiểm tra lỗi, có thể bao gồm các trường header



Phát hiện lỗi không đáng tin cậy 100%!

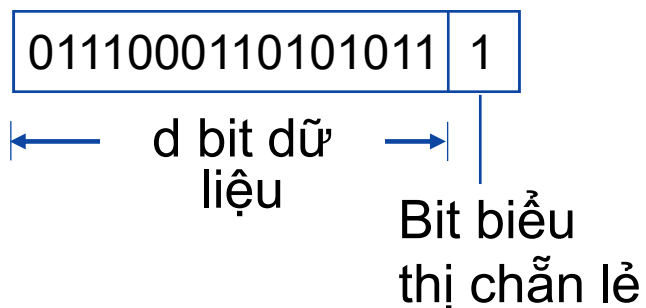
- Giao thức có thể bỏ lỡ một số lỗi, nhưng hiếm
- Trường EDC lớn hơn giúp phát hiện và hiệu chỉnh lỗi tốt hơn

# Kiểm tra chẵn lẻ (Parity checking)



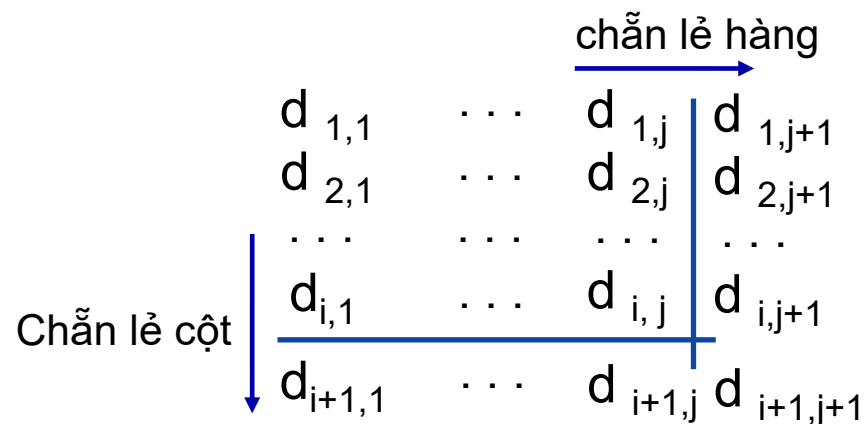
## Chẵn lẻ bit đơn :

- Chỉ phát hiện lỗi bit đơn



## Chẵn lẻ bit hai chiều:

- phát hiện *và sửa* lỗi bit đơn



## Kiểm tra (Parity) chẵn :

- 1: Tổng số bit 1 trong d là lẻ
- 0: Tổng số bit 1 trong d là chẵn

VD: 1 0 1 0 1 1 | 1  
1 1 1 1 0 0  
0 1 1 1 0 1  
1 0 1 0 1 0

phát hiện và sửa chữa được bit đơn lỗi:

1 0 1 0 1 1 | 1  
1 0 1 1 0 0 | 0 → Hàng có bit lỗi  
0 1 1 1 0 1  
1 0 1 0 1 0 | 0  
↓  
Cột có bit lỗi



# Internet checksum



**Mục tiêu :** phát hiện lỗi (là các bit bị đảo lộn 0 thành 1 và ngược lại) trong phân đoạn được truyền.

## Bên gửi:

- Xử lý nội dung của segment UDP (bao gồm các trường header UDP và IP) dưới dạng chuỗi các số nguyên 16 bit
- **Checksum:** Cộng (tổng bù 1) của các đoạn nội dung của segment.
- Giá trị checksum được ghi vào trường checksum của UDP

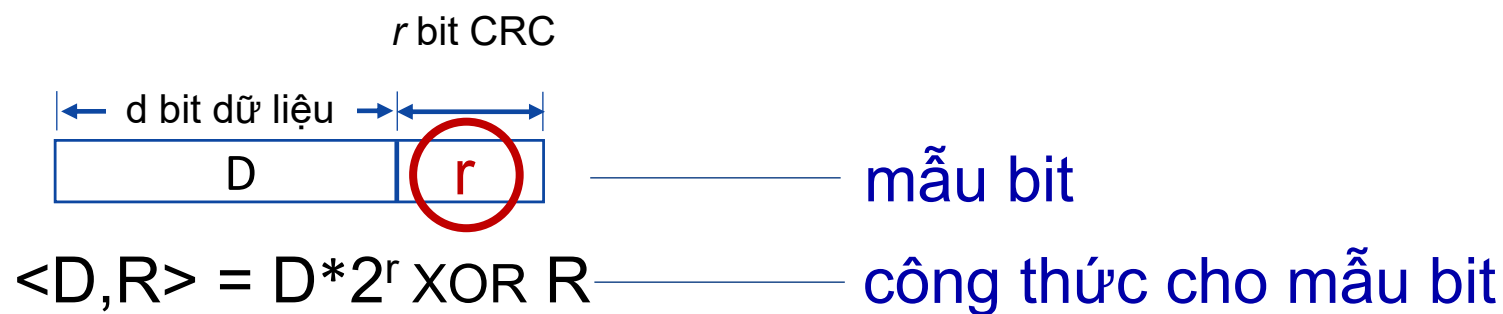
## Bên nhận:

- Tính toán checksum của segment nhận được
- Kiểm tra xem checksum tính toán được có bằng giá trị trong trường checksum không:
  - Nếu không bằng - đã phát hiện lỗi
  - Nếu bằng - không phát hiện lỗi.  
*Nhưng có thể có lỗi khác...*

# Cyclic Redundancy Check (CRC)



- Phương pháp phát hiện lỗi tốt hơn
- **D**: bit dữ liệu (coi đây là dữ liệu nhị phân)
- **G**: mẫu bit (bộ tạo), có độ dài  $r+1$  bit (độ dài  $r$  là số bit lỗi có thể được phát hiện)



Mục tiêu: chọn  $r$  bit CRC, **R**, sao cho  $\langle D, R \rangle$  chia hết cho  $G$  (mod 2: chia nhị phân)

- Bên nhận biết  $G$ , chia  $\langle D, R \rangle$  cho  $G$ . Nếu phần dư khác 0: phát hiện lỗi!
- Có thể phát hiện số lượng các lỗi bit nhỏ hơn  $r+1$  bit
- Được sử dụng rộng rãi trong thực tế (Ethernet, WiFi 802.11 )

# Cyclic Redundancy Check (CRC): ví dụ



Chúng ta muốn:

$$D * 2^r \text{ XOR } R = nG$$

tương đương với:

$$D * 2^r = nG \text{ XOR } R$$

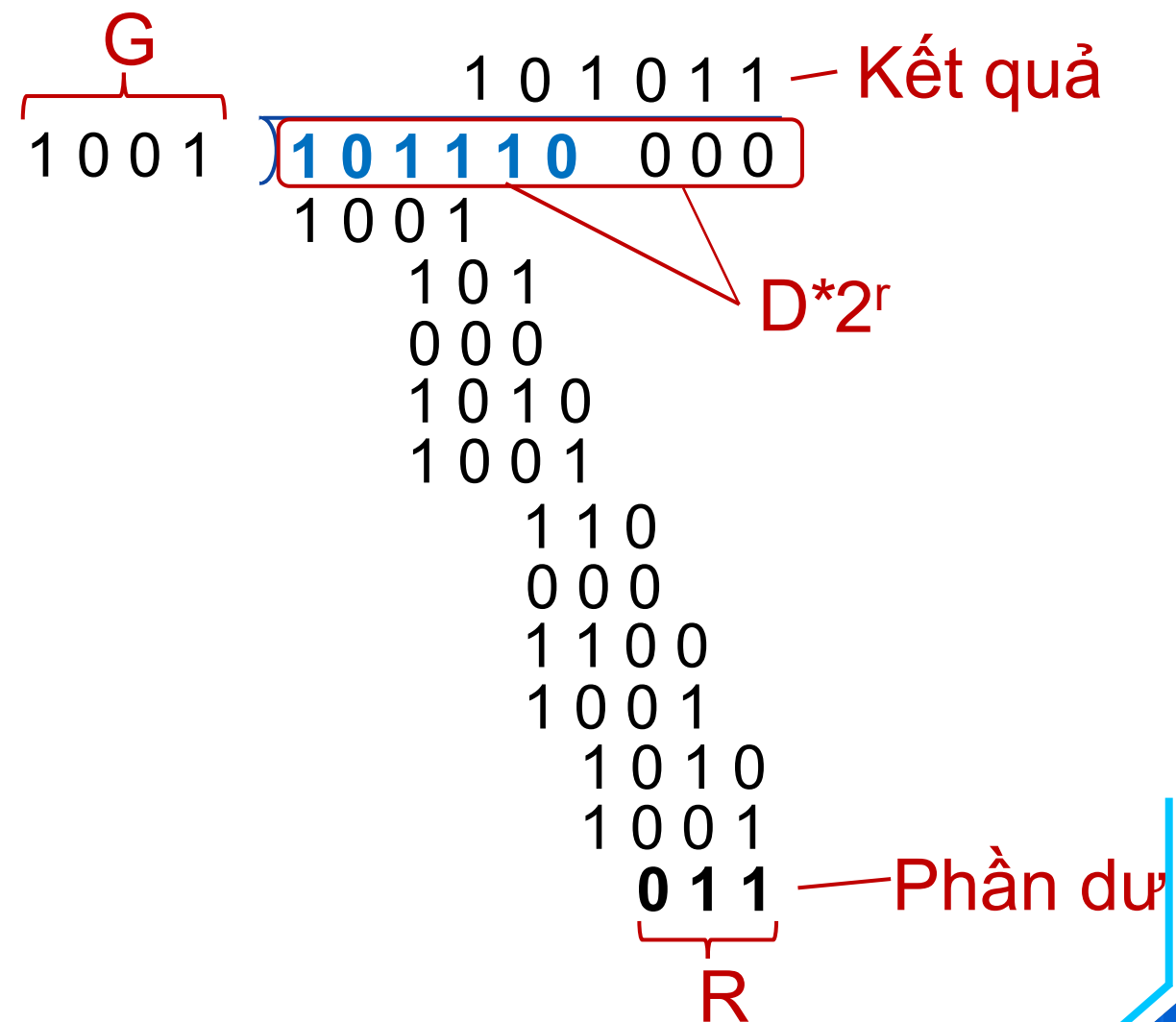
hoặc tương đương với việc:

chúng ta chia  $D * 2^r$  cho  $G$ , được phần dư  $R$  thỏa mãn:

$$R = \text{chia lấy dư} \left[ \frac{D * 2^r}{G} \right]$$

Chú ý:

$D * 2^r$  = là phép dịch chuyển nhị phân dữ liệu  $D$  qua trái  $r$  bit



# Tầng liên kết, LAN

- Giới thiệu
- Phát hiện lỗi, sửa lỗi
- Các giao thức đa truy cập
- Mạng LAN
  - Định địa chỉ, ARP
  - Ethernet
  - Switch
  - VLAN
- Ngữ cảnh của một yêu cầu dịch vụ web



# Các Liên kết và giao thức đa truy cập



Có hai loại “liên kết”:

- Điểm-điểm (End-end)

- Liên kết điểm-điểm giữa bộ chuyển mạch Ethernet, thiết bị đầu cuối
- PPP cho truy cập quay số

- **Quảng bá (qua mạng có dây hoặc môi trường dùng chung)**

- Các công nghệ Ethernet đời cũ
- Hybrid fiber-coaxial trong mạng truy cập dựa trên cáp đồng trục
- Mạng LAN không dây 802.11, 4G/5G, vệ tinh



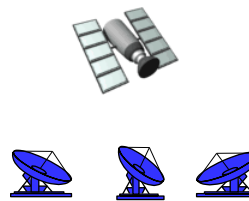
dây dùng chung (ví dụ:  
cáp Ethernet)



đài chia sẻ: 4G/5G



đài chia sẻ: WiFi



radio dùng chung: vệ tinh



con người tại một bữa tiệc  
cocktail  
(không khí chung, âm học)

# Giao thức đa truy cập



- Một kênh dùng chung (broadcast) được chia sẻ cho nhiều thiết bị
- Hai hoặc nhiều phiên truyền đồng thời bởi các nút: xảy ra xung đột.
  - *Sự xung đột* là khi một nút nhận được hai hoặc nhiều tín hiệu cùng một lúc

## Giao thức đa truy cập

- Là thuật toán phân phối nhằm xác định cách thức để các nút chia sẻ kênh truyền, hay gọi cách khác là xác định khi nào nút có thể truyền
- Giao tiếp giữa các nút để chia sẻ kênh phải sử dụng chính kênh đó!
  - không sử dụng thêm kênh truyền khác để phối hợp hoạt động



# Một giao thức đa truy cập lý tưởng



*Giả sử ta có:* kênh đa truy cập (MAC) tốc độ  $R$  bps

*Ta mong muốn:*

1. Khi một nút muốn truyền, nó có thể gửi ở tốc độ  $R$ .
2. Khi  $M$  nút muốn truyền, mỗi nút có thể gửi ở tốc độ trung bình  $R/M$
3. Phi tập trung hoàn toàn:
  - Không có nút đặc biệt để phối hợp truyền
  - Không cần đồng bộ hóa đồng hồ, khe thời gian.
4. Đơn giản



# Các giao thức đa truy cập (MAC): phân loại

Được chia ra ba loại:

## ■ Phân hoạch kênh

- Là chia kênh thành các “kênh nhỏ” (vd: khe thời gian, tần số, mã)
- Phân bổ các kênh nhỏ đã chia cho các nút để sử dụng độc quyền

## ■ Truy cập ngẫu nhiên

- Kênh không bị chia nhỏ, cho phép có đụng độ khi truyền
- Truyền lại nếu có đụng độ xảy ra

## ■ Truyền luân phiên

- Các nút truyền luân phiên nhau, tuy nhiên các nút muốn gửi nhiều thì cần phải có nhiều lượt truyền hơn

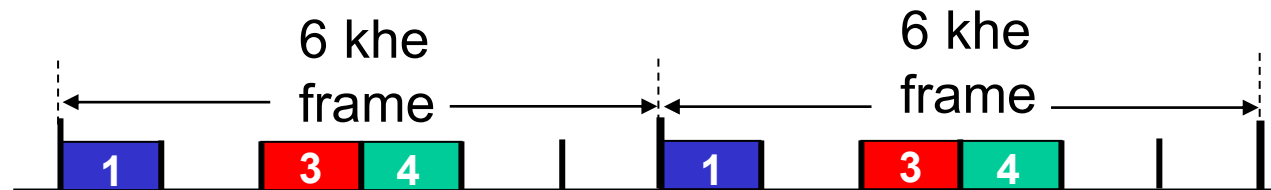
# Giao thức MAC phân hoạch kênh: TDMA



**TDMA (time division multiple access):** đa truy cập phân chia theo thời gian

- Truy cập vào kênh lần lượt theo vòng
- Mỗi trạm sở hữu một slot độ dài cố định (độ dài = thời gian truyền gói) trong mỗi vòng
- Các trạm không truyền trong slot của mình thì các trạm khác cũng không được sử dụng slot đó.

Ví dụ: LAN 6 trạm, 1,3,4 có gói để gửi, các khe 2,5,6 nhàn rỗi

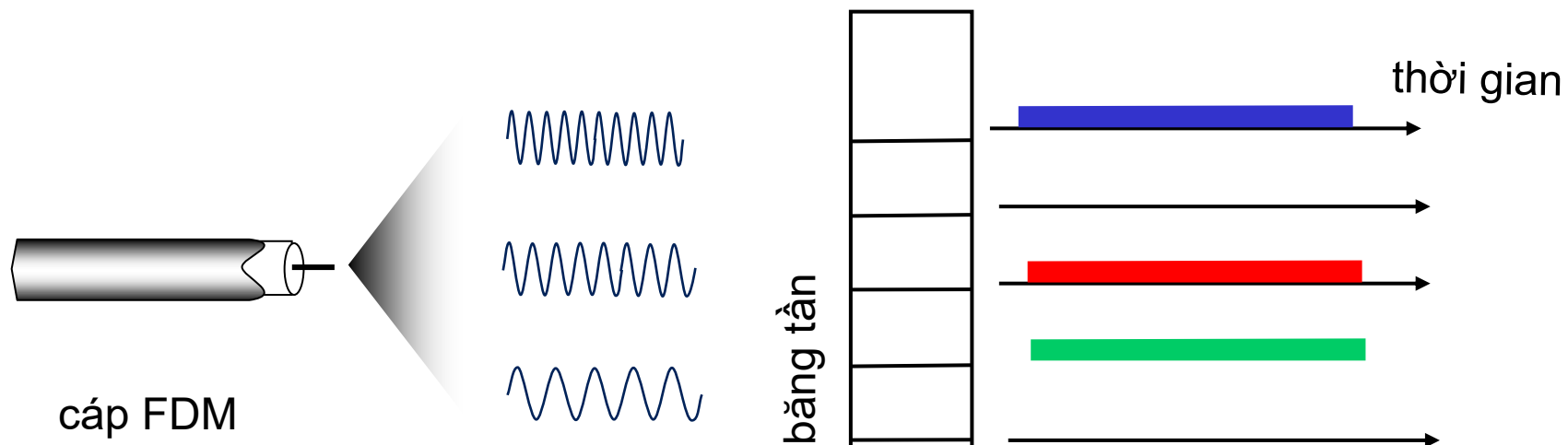


# Các giao thức MAC phân hoạch kênh: FDMA



FDMA (frequency division multiple access): đa truy cập phân chia theo tần số

- Toàn bộ kênh được chia thành các dải tần số riêng biệt
- Mỗi trạm được gán một dải tần cố định
- Các trạm không truyền trong dải tần số của mình thì các trạm khác cũng không được sử dụng dải tần số đó
- Ví dụ: LAN 6 trạm, 1,3,4 có gói để gửi, băng tần 2,5,6 nhàn rỗi



# Giao thức truy cập ngẫu nhiên



- Khi nút có gói để gửi
  - Truyền ở tốc độ dữ liệu toàn kênh R.
  - Không có sự phối hợp *trước* giữa các nút
- Nếu hai hoặc nhiều nút truyền đồng thời: “xung đột”
- **Giao thức MAC truy cập ngẫu nhiên** chỉ định:
  - Cách phát hiện xung đột
  - Cách khôi phục sau xung đột (ví dụ: trì hoãn việc truyền lại)
- Ví dụ về các giao thức MAC truy cập ngẫu nhiên:
  - ALOHA, slotted ALOHA
  - CSMA, CSMA/CD, CSMA/CA

# Slotted ALOHA



## Giả định:

- Tất cả các frame có cùng kích thước
- Thời gian chia thành các slot có kích thước bằng nhau (thời gian truyền 1 frame)
- Các nút bắt đầu truyền ở đầu mỗi slot
- Các nút được đồng bộ hóa
- Nếu 2 hoặc nhiều nút truyền trong slot, tất cả các nút phát hiện xung đột

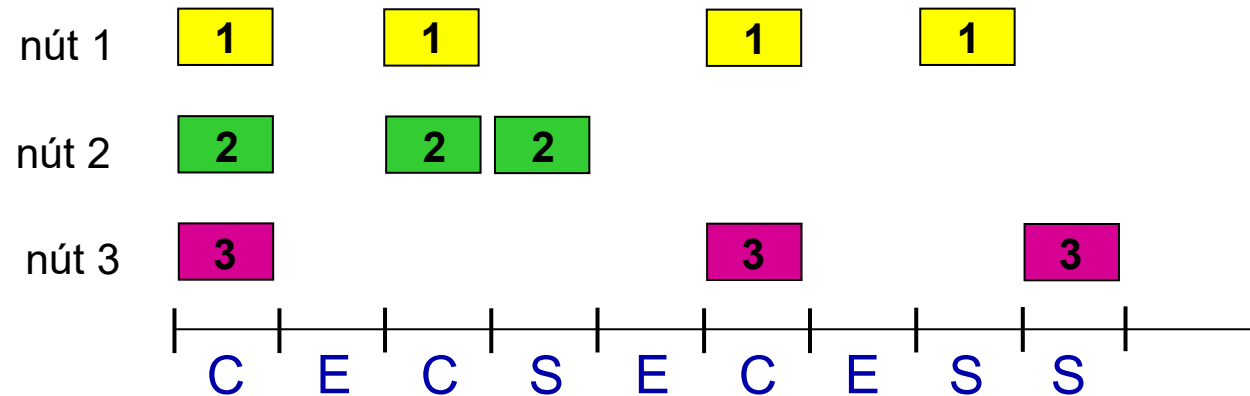
## Hoạt động:

- Khi nút nhận được frame mới, truyền trong slot tiếp theo
  - *Nếu không có xung đột*: nút có thể gửi frame mới trong slot tiếp theo
  - *Nếu xung đột*: nút truyền lại frame trong mỗi slot tiếp theo với xác suất  $p$  cho đến khi thành công

ngẫu nhiên hóa - tại sao ?



# Slotted ALOHA



## Ưu điểm:

- Nếu chỉ 1 nút hoạt động, nó có thể truyền liên tục ở tốc độ cực đại của kênh
- Phi tập trung cao: chỉ các khe (slot) ở các nút cần được đồng bộ hóa
- Đơn giản

## Nhược điểm:

- Có xung đột: lãng phí slot
- Nhiều slot nhàn rỗi
- Các nút có thể phát hiện xung đột trong thời gian ngắn hơn thời gian truyền toàn bộ gói tin
- Phải đồng bộ hóa đồng hồ

# Slotted ALOHA: độ hiệu quả



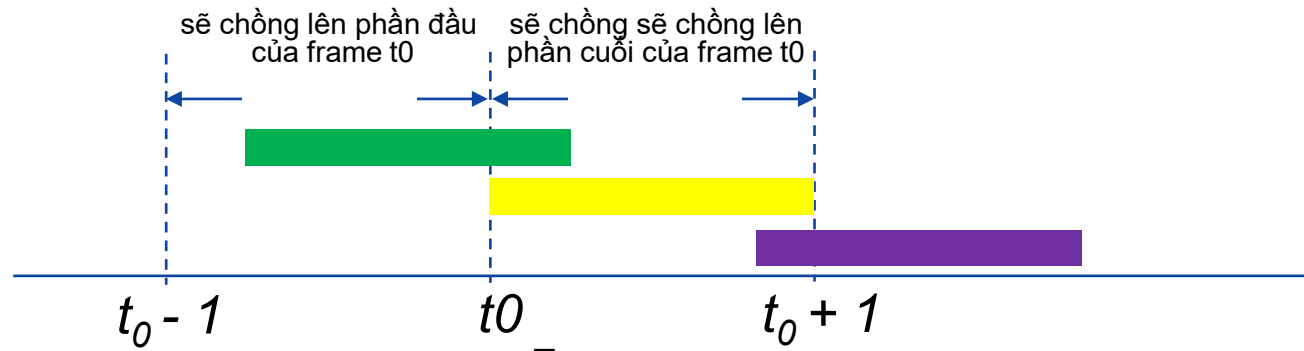
**Hiệu quả:** tỷ lệ dài hạn của các slot thành công (nhiều nút, tất cả đều có nhiều frame để gửi)

- *Giả định:* có  $N$  nút có nhiều frame để gửi, mỗi frame truyền trong slot với xác suất  $p$ 
    - Xác suất nút thành công trong một slot =  $p(1-p)^{N-1}$
    - Xác suất *bất kỳ* nút nào cũng thành công =  $Np(1-p)^{N-1}$
    - Hiệu quả tối đa: tìm  $p^*$  để tối đa hóa  $Np(1-p)^{N-1}$
    - Đối với nhiều nút, lấy giới hạn  $Np^*(1-p^*)^{N-1}$  khi  $N$  tiến tới vô cùng
- hiệu quả tối đa =  $1/e = 0.37$*
- *Trường hợp tốt nhất:* kênh được sử dụng để truyền hữu ích 37% thời gian!

# Pure ALOHA



- Unslotted Aloha: đơn giản hơn, không đồng bộ hóa
  - Khi frame đầu tiên đến: truyền ngay lập tức
- Xác suất xung đột tăng khi không đồng bộ hóa:
  - Khi frame được gửi tại  $t_0$  xung đột với các frame khác được gửi trong khoảng  $[t_0 - 1, t_0 + 1]$



- Hiệu suất Pure Aloha: 18%!

# CSMA: đa truy cập cảm biến sóng mang



Đơn giản: lắng nghe trước khi truyền:

- Nếu kênh truyền nhàn rỗi: truyền toàn bộ frame
- Nếu kênh truyền bận: trì hoãn truyền
- Tương tự con người: không ngắt lời người khác!

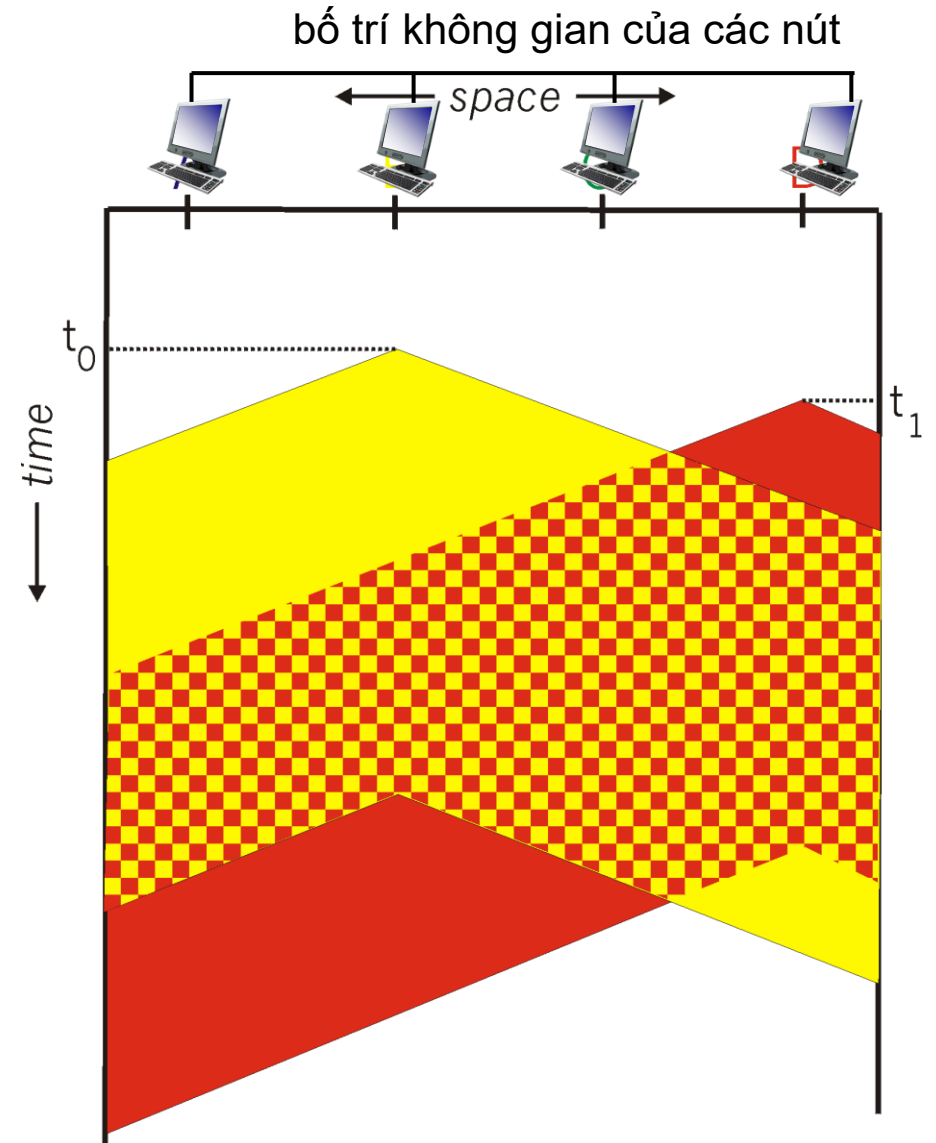
**CSMA/CD (collision detection):** CSMA với kỹ thuật *phát hiện xung đột*

- Xung đột được *phát hiện* trong thời gian ngắn
- Nếu đang truyền gặp xung đột thì ngừng truyền, giảm lãng phí kênh
- Xung đột dễ dàng được phát hiện với đường truyền có dây, khó phát hiện với đường truyền không dây
- Tương tự con người: người đối thoại lịch sự.

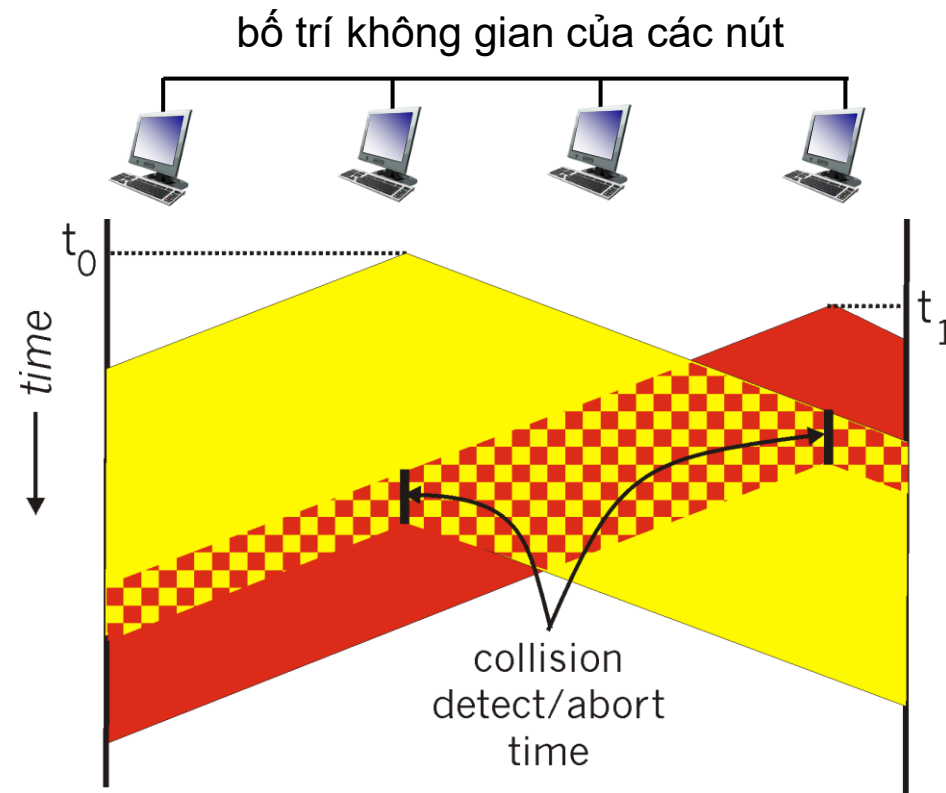
# CSMA: xung đột



- Xung đột vẫn *có thể* xảy ra với cảm biến sóng mang:
  - Do có độ trễ trong lan truyền, đó là 2 node bắt đầu truyền nhưng không phát hiện ra tín hiệu của nhau
- **Xung đột xảy ra:** toàn bộ thời gian truyền gói bị lãng phí
  - khoảng cách và độ trễ lan truyền đóng vai trò chính trong việc xác định xác suất xung đột



- CSMA/CD giảm lượng thời gian lãng phí khi xung đột
  - Việc truyền bị hủy bỏ khi phát hiện xung đột



# Thuật toán CSMA/CD Ethernet



1. NIC (network interface card) nhận datagram từ tầng mạng, tạo frame
2. Nếu NIC cảm nhận được kênh:  
    nếu **nhàn rỗi**: bắt đầu truyền frame.  
    nếu **bận**: đợi cho đến khi kênh nhàn rỗi, sau đó truyền
3. Nếu NIC truyền toàn bộ frame mà không có xung đột, thì NIC đã hoàn thành việc truyền frame!
4. Nếu NIC phát hiện một đường truyền khác trong khi gửi: hủy bỏ truyền, gửi tín hiệu tắc nghẽn
5. Sau khi hủy bỏ truyền, NIC nhập **backoff nhị phân (cấp số nhân)** :
  - sau lần xung đột thứ  $m$  , NIC chọn ngẫu nhiên  $K$  từ  $\{0, 1, 2, \dots, 2^m - 1\}$  .  
NIC đợi một khoảng thời gian  $K * 512$ \*(thời gian truyền 1 bit), và quay lại Bước 2
  - nhiều xung đột hơn: khoảng thời gian đợi sẽ dài hơn



# Độ hiệu quả của CSMA/CD



- $t_{prop}$  = độ trễ lan truyền tối đa giữa 2 nút trong mạng LAN
- $t_{trans}$  = thời gian để truyền max-size của frame

$$efficiency = \frac{1}{1 + 5t_{prop}/t_{trans}}$$

- Hiệu quả tăng lên 1
  - Khi  $t_{prop}$  tiến về 0
  - Khi  $t_{trans}$  tiến đến vô cùng
- Hiệu suất CSMA/CD tốt hơn ALOHA, hơn nữa nó đơn giản, rẻ, phi tập trung !

# Các giao thức MAC “luân phiên” (Taking turns)



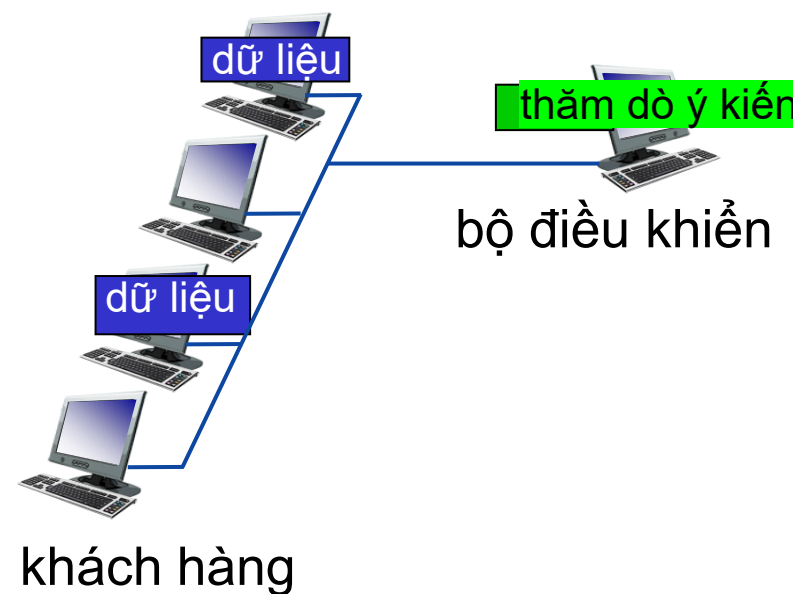
- Giao thức MAC phân hoạch kênh
  - Chia sẻ kênh *hiệu quả* và *công bằng* khi tải cao
  - Không hiệu quả khi tải thấp: độ trễ trong truy cập kênh, băng thông  $1/N$  được phân bổ ngay cả khi chỉ có 1 nút hoạt động!
- Giao thức MAC truy cập ngẫu nhiên
  - Hiệu quả ở mức tải thấp: nút đơn có thể sử dụng đầy đủ kênh
  - Tải cao: xung đột tăng cao
- Các giao thức “luân phiên”
  - Kết hợp những điểm tốt nhất của hai giao thức trên!

# Các giao thức MAC “luân phiên”



## Kiểm soát vòng:

- Nút điều khiển “mời” các nút khác (máy khách) lần lượt truyền
- Thường được sử dụng với các thiết bị "câm"
- Mối quan tâm:
  - Chi phí kiểm soát vòng
  - Độ trễ
  - Điểm lỗi duy nhất (bộ kiểm soát)

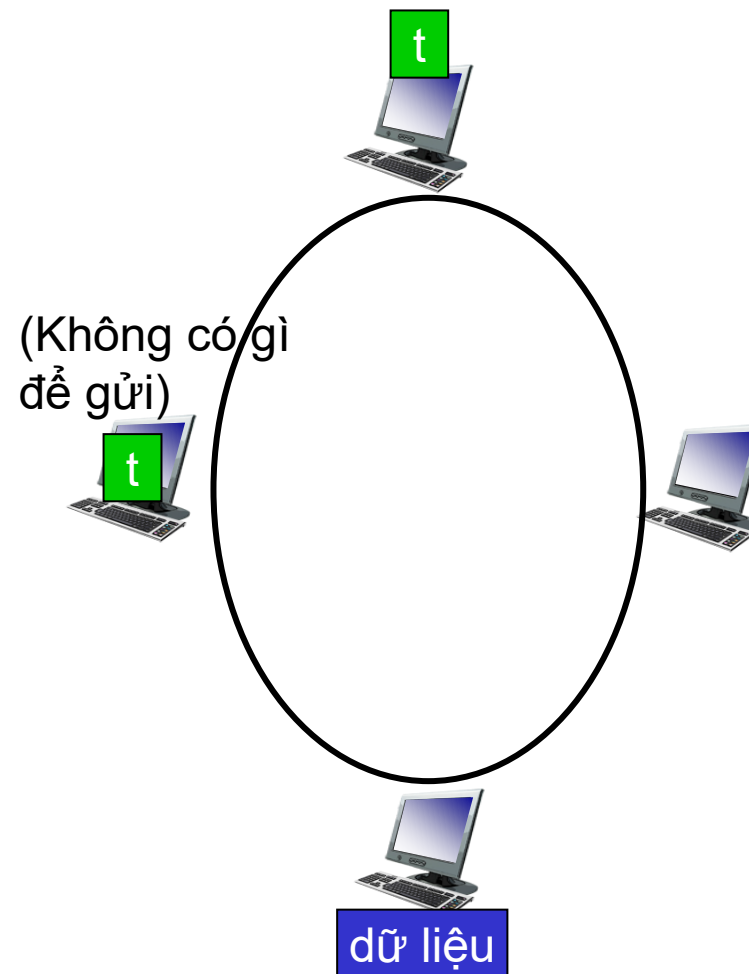


# Các giao thức MAC “luân phiên”

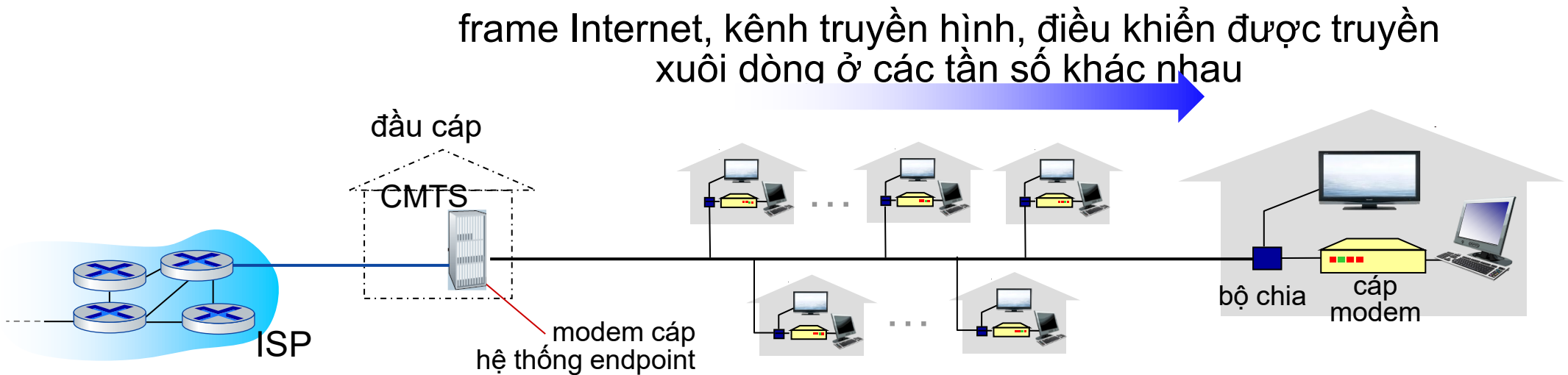


## Quá trình chuyển Token:

- Điều khiển quá trình chuyển *Token* tuần tự từ nút này sang nút khác.
- Bản tin token
- Mỗi quan tâm:
  - Chi phí token
  - Độ trễ
  - Tồn tại một điểm lỗi (chính là token)

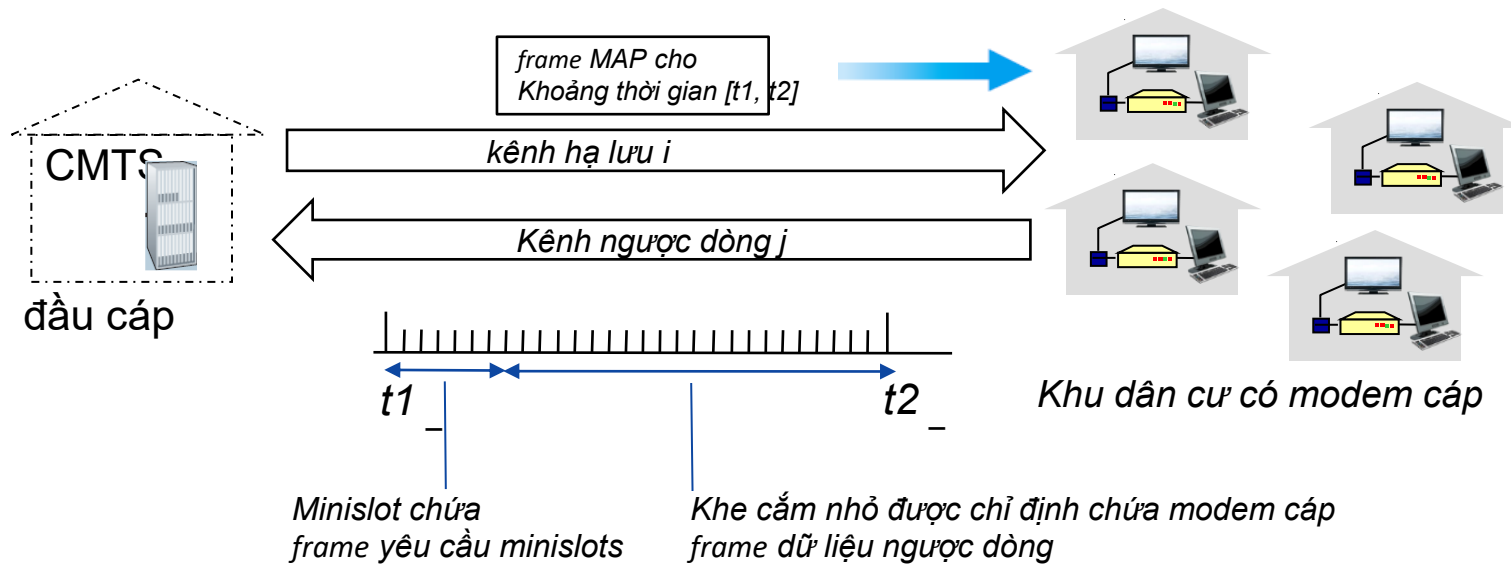


# Mạng truy cập cáp: FDM, TDM và truy cập ngẫu nhiên!



- **Nhiều kênh FDM downstream** (phát sóng): tối đa 1,6 Gbps/kênh
  - CMTS đơn lẻ truyền vào các kênh
- **Nhiều kênh upstream** (tối đa 1 Gbps/kênh)
  - **đa truy cập**: tất cả người dùng tranh chấp (truy cập ngẫu nhiên) đối với các time slot của kênh upstream; hay được gán theo TDM.

# Mạng truy cập cáp:



## DOCSIS (data over cable service interface specification)

- FDM qua các kênh tần số upstream, downstream
- TDM upstream: một số vị trí được chỉ định, một số có tranh chấp
  - Downstream MAP frame : chỉ định các vị trí upstream
  - Yêu cầu đối với các slot upstream (và dữ liệu) được truyền truy cập ngẫu nhiên (thuật toán backoff nhị phân) trong các slot được chọn

# Tóm tắt các giao thức



- **Phân hoạch kênh**, theo thời gian, tần số hoặc mã
  - Phân chia thời gian, Phân chia tần số
- **Truy cập ngẫu nhiên** (động)
  - ALOHA, S-ALOHA, CSMA, CSMA/CD
  - Cảm biến sóng mang: dễ dàng trong một số công nghệ (có dây), khó trong các công nghệ khác (không dây)
  - CSMA/CD được sử dụng trong Ethernet
  - CSMA/CA được sử dụng trong 802.11
- **Luân phiên**
  - Phân chia phiên tập trung, Token passing
  - Bluetooth, FDDI, Token ring

# Tầng liên kết, LAN



- Giới thiệu
- Phát hiện lỗi, sửa lỗi
- Nhiều giao thức truy cập
- **Mạng LAN**
  - Định địa chỉ, ARP
  - Ethernet
  - Switch
  - VLAN
- Ngữ cảnh của một yêu cầu dịch vụ web





# Địa chỉ MAC (Media Access Control)



- Địa chỉ IP 32 bit:
  - Địa chỉ tầng mạng cho các cổng
  - Được sử dụng để chuyển tiếp tầng 3 (tầng mạng)
  - Ví dụ: 128.119.40.136
- Địa chỉ MAC (hoặc địa chỉ LAN, địa chỉ vật lý hoặc địa chỉ Ethernet):
  - Chức năng: được sử dụng “cục bộ” để chuyển frame từ một cổng vật lý này sang một cổng khác có kết nối vật lý với nó (cùng mạng IP)
  - Địa chỉ MAC 48 bit (hầu hết các mạng LAN) được ghi trong ROM của NIC, đôi khi cũng có thể cài đặt bằng phần mềm
  - Ví dụ: 1A-2F-BB-76-09-AD

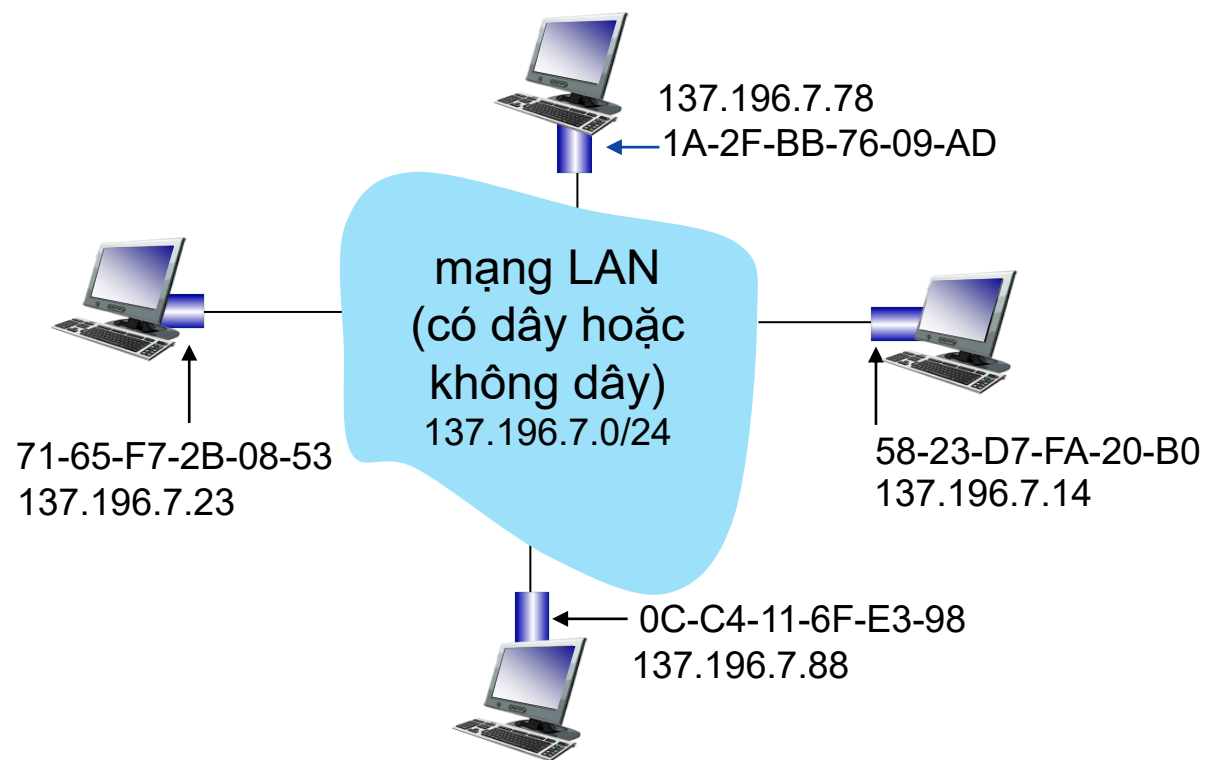
*ký hiệu thập lục phân (hệ 16)  
(mỗi “ký tự” đại diện cho 4 bit)*

# Địa chỉ MAC (Media Access Control)



Mỗi cổng trên mạng LAN có:

- Địa chỉ **MAC** 48 bit duy nhất
- Có địa chỉ IP cục bộ 32 bit duy nhất (như ví dụ dưới đây)



# Địa chỉ MAC

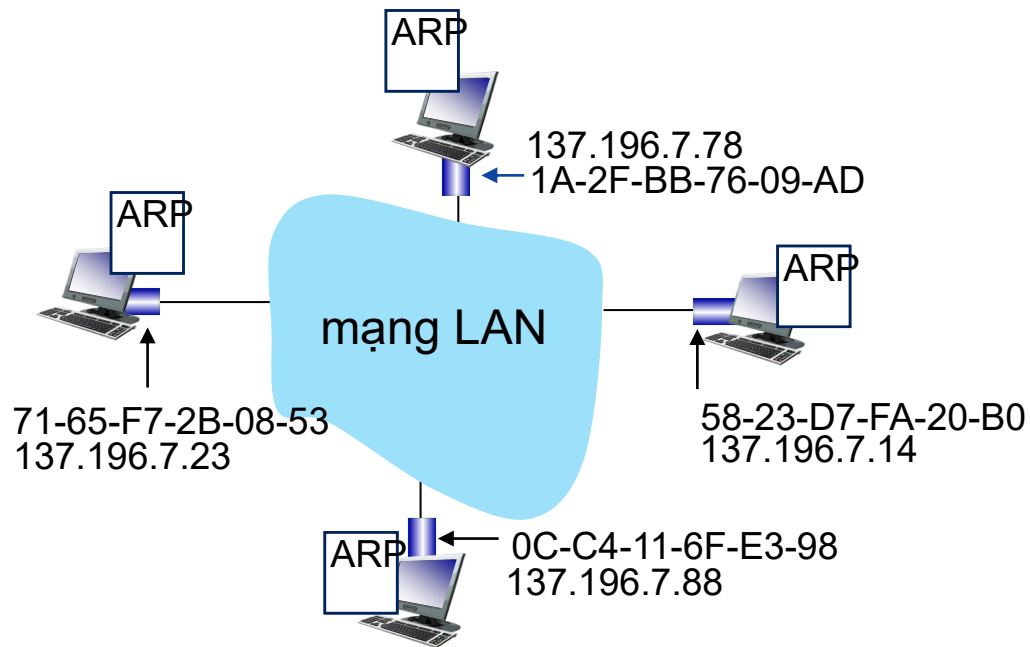


- Phân bổ địa chỉ MAC do IEEE quản lý
- Mỗi nhà sản xuất mua một phần không gian địa chỉ MAC (để đảm bảo tính duy nhất trên toàn cầu)
- So sánh với đời thật:
  - Địa chỉ MAC: như Số CMND
  - Địa chỉ IP: như địa chỉ nhà
- Địa chỉ MAC cứng: tính di động
  - Có thể di chuyển cổng từ mạng LAN này sang mạng LAN khác
  - Tới vị trí mới thì gọi lại địa chỉ IP là *không thể*: do phụ thuộc vào mạng con IP mà nút được thêm vào.

# ARP (address resolution protocol)



*Câu hỏi:* cách nào để xác định địa chỉ MAC của một cổng, khi biết địa chỉ IP của nó?



**Bảng ARP:** mỗi nút IP (máy chủ, bộ định tuyến) trên mạng LAN có một bảng để:

- Ánh xạ địa chỉ IP/MAC cho một số nút mạng LAN:

**< địa chỉ IP; Địa chỉ MAC; TTL >**

- TTL (Time To Live): thời gian sau đó dòng ánh xạ địa chỉ sẽ bị xóa (thường là 20 phút)

# Cách thức hoạt động của giao thức ARP



Ví dụ: A muốn gửi datagram cho B

- MAC của B không có trong bảng ARP của A nên A sử dụng ARP để tìm địa chỉ MAC của B

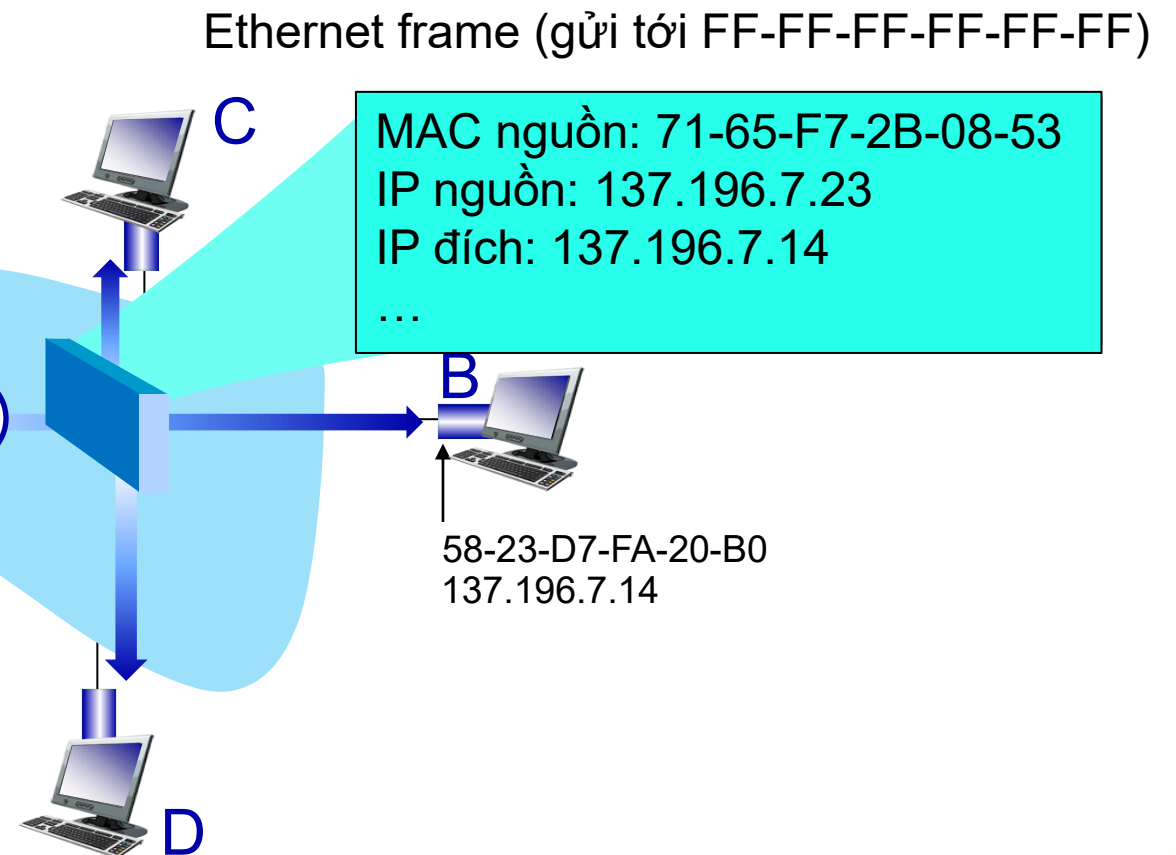
A gửi ARP quảng bá, chứa địa chỉ IP của B

- 1
  - địa chỉ MAC đích = FF-FF-FF-FF-FF-FF
  - tất cả các nút trên mạng LAN nhận truy vấn ARP

Bảng ARP trong A

| địa chỉ IP | địa chỉ MAC | TTL |
|------------|-------------|-----|
|            |             |     |

71-65-F7-2B-08-53  
137.196.7.23

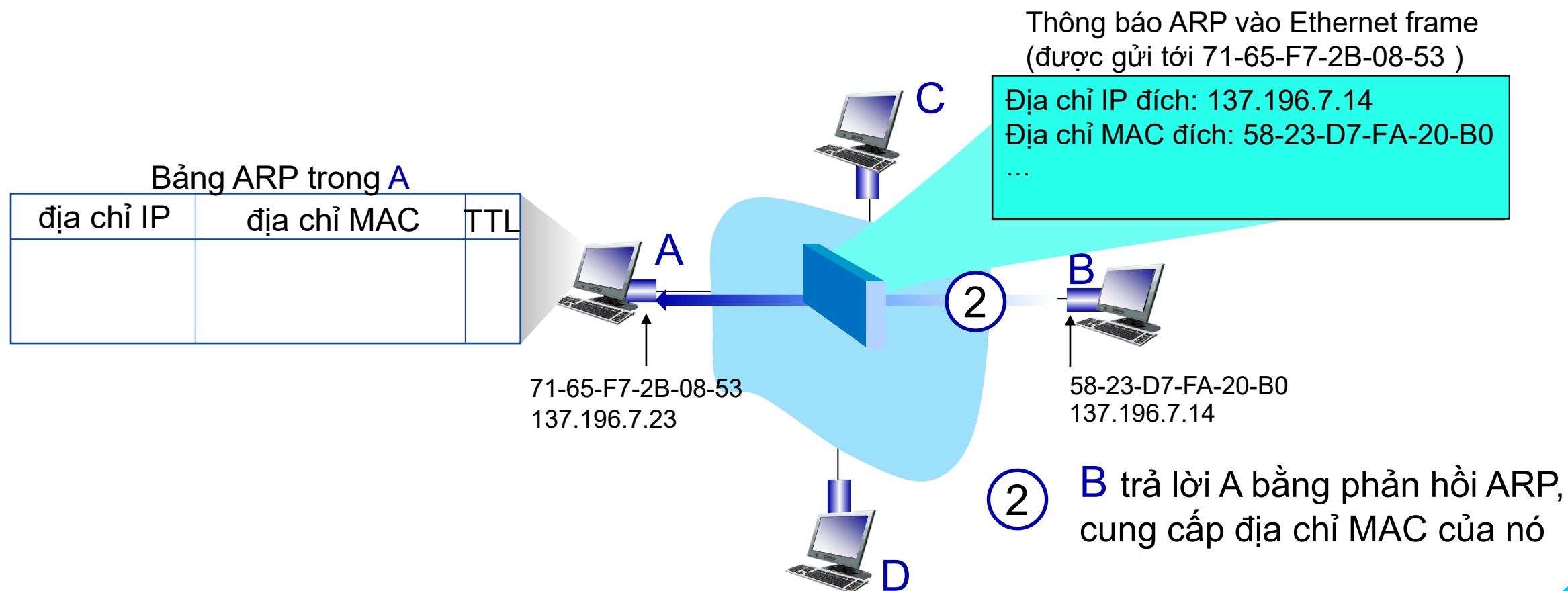


# Cách thức hoạt động của giao thức ARP



Ví dụ: A muốn gửi datagram cho B

- MAC của B không có trong bảng ARP của A nên A sử dụng ARP để tìm địa chỉ MAC của B

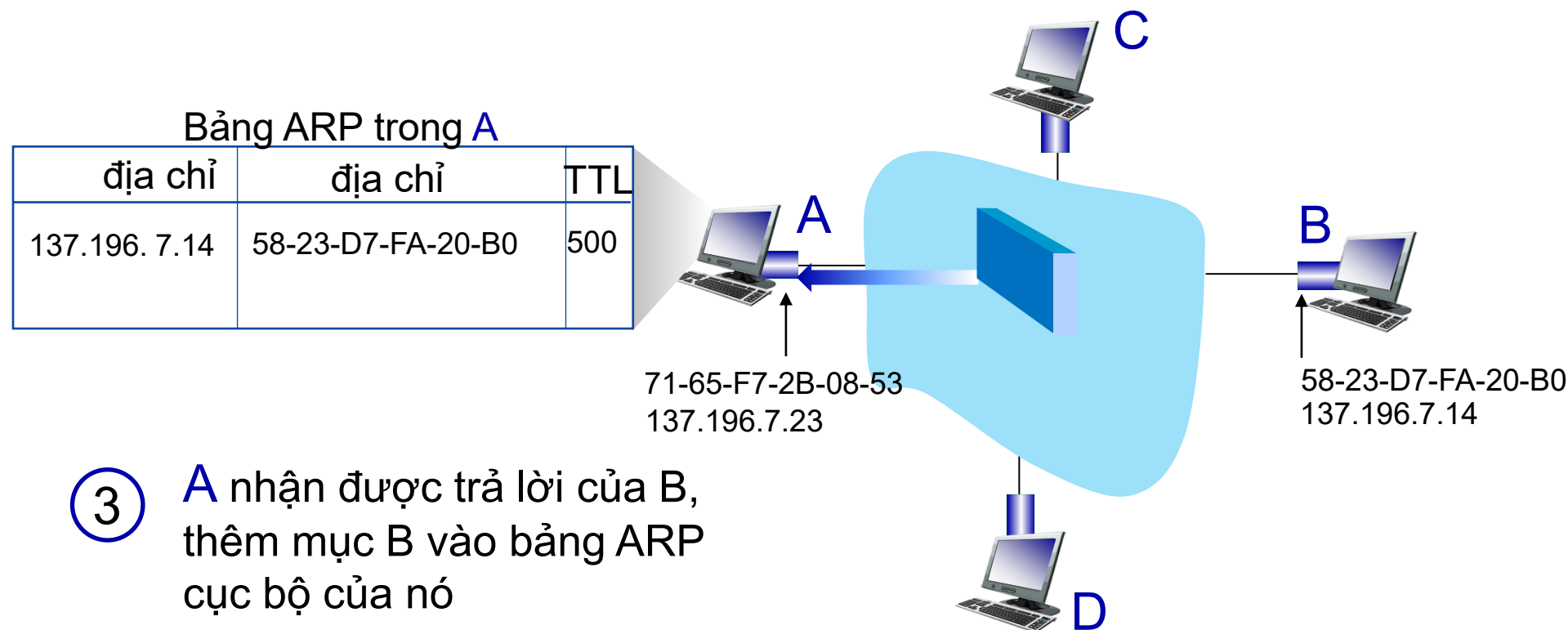


# Cách thức hoạt động của giao thức ARP



Ví dụ: A muốn gửi datagram cho B

- MAC của B không có trong bảng ARP của A nên A sử dụng ARP để tìm địa chỉ MAC của B

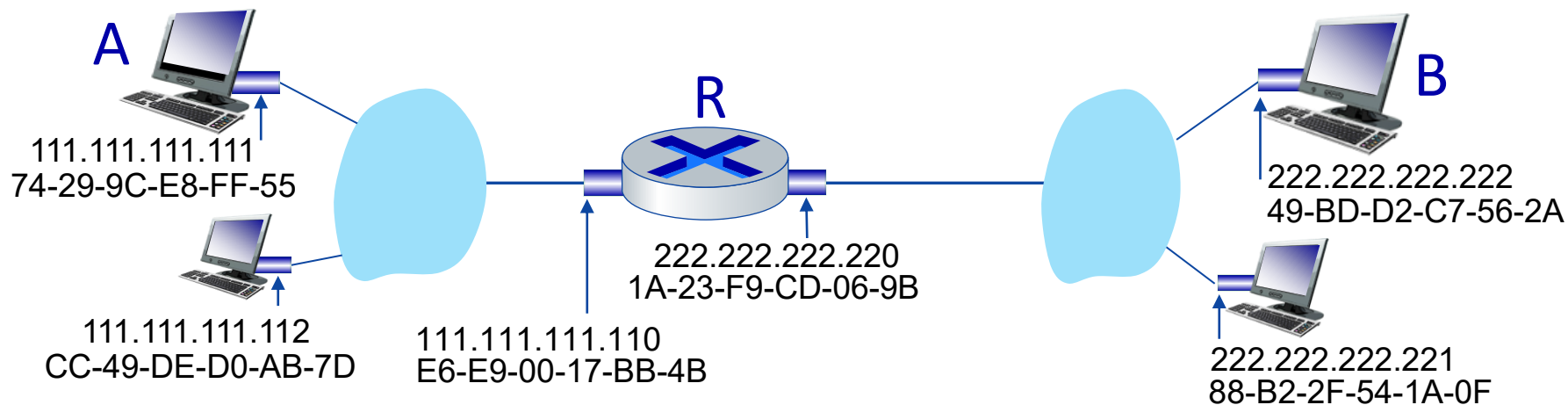


# Định tuyến đến nút trong mạng con khác: gán địa chỉ



Ngữ cảnh: gửi một datagram từ A đến B qua bộ định tuyến R

- Tập trung vào địa chỉ ở cấp độ IP (datagram) và lớp MAC (frame)
- Giả sử:
  - A biết địa chỉ IP của B
  - A biết địa chỉ IP của cổng bộ định tuyến R kết nối với mạng của A (làm thế nào?)
  - A biết địa chỉ MAC của cổng bộ định tuyến R kết nối với mạng của A (bằng cách nào?)

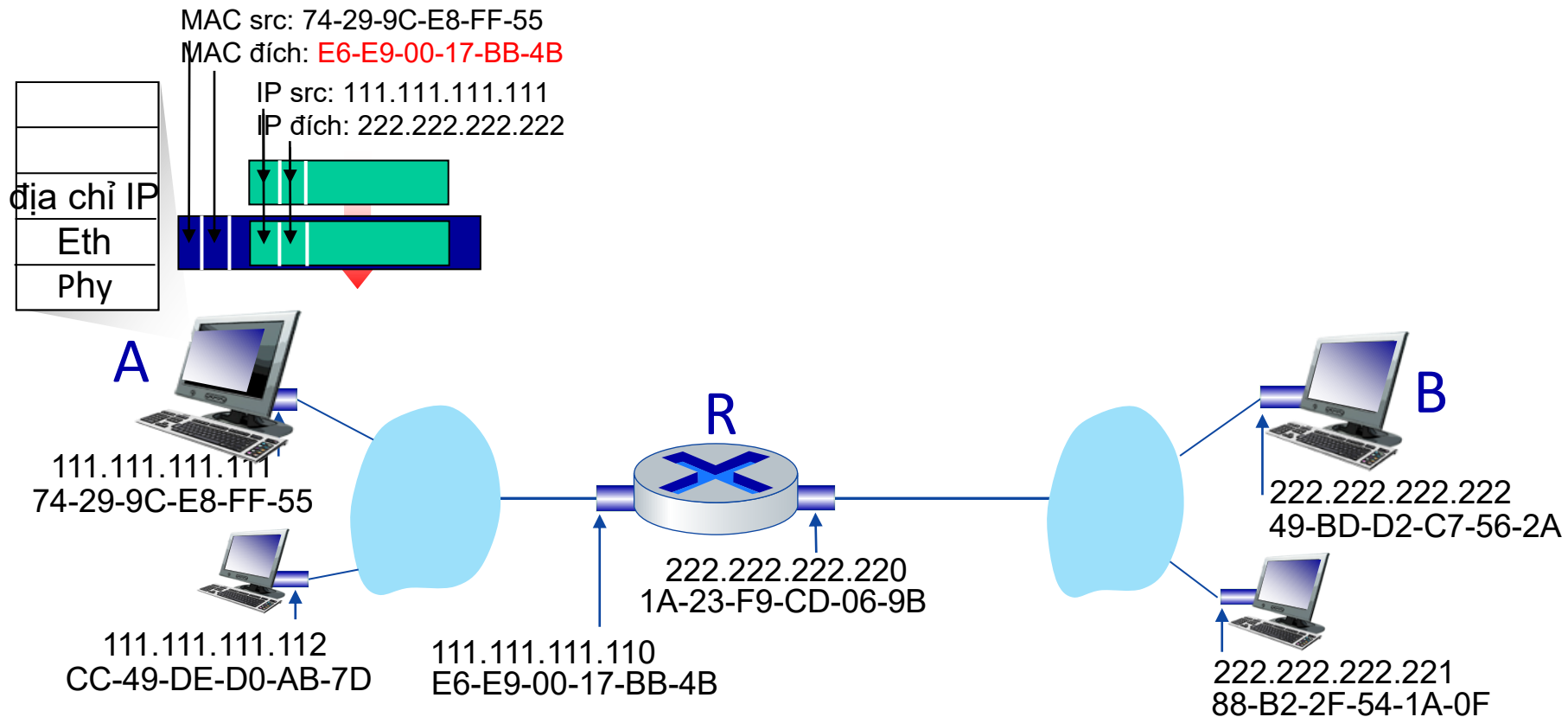




# Định tuyến đến nút trong mạng con khác: gán địa chỉ



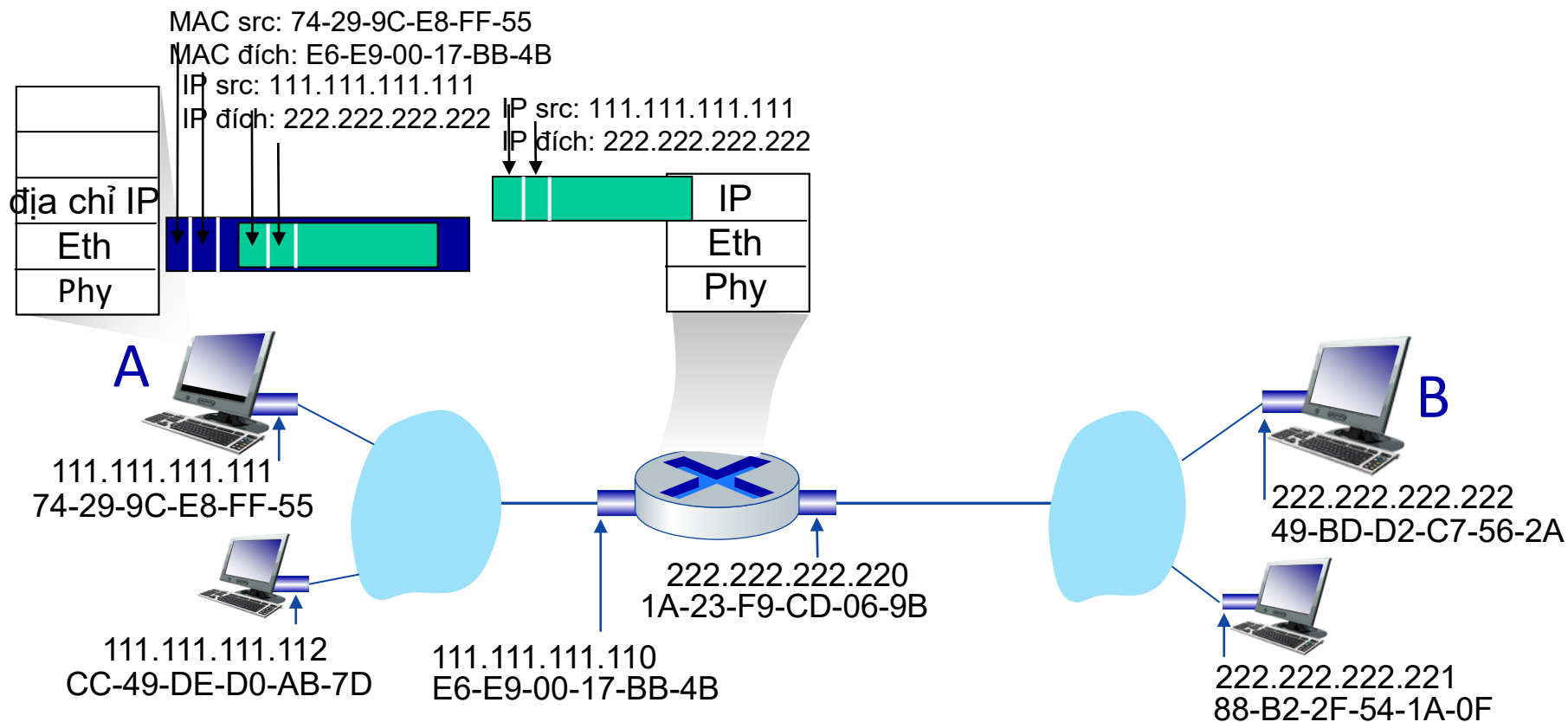
- A tạo IP datagram với IP nguồn A, đích B
- A tạo frame lớp liên kết chứa gói dữ liệu IP A-đến-B
  - Địa chỉ MAC của R là đích đến của frame



# Định tuyến đến nút trong mạng con khác: gán địa chỉ



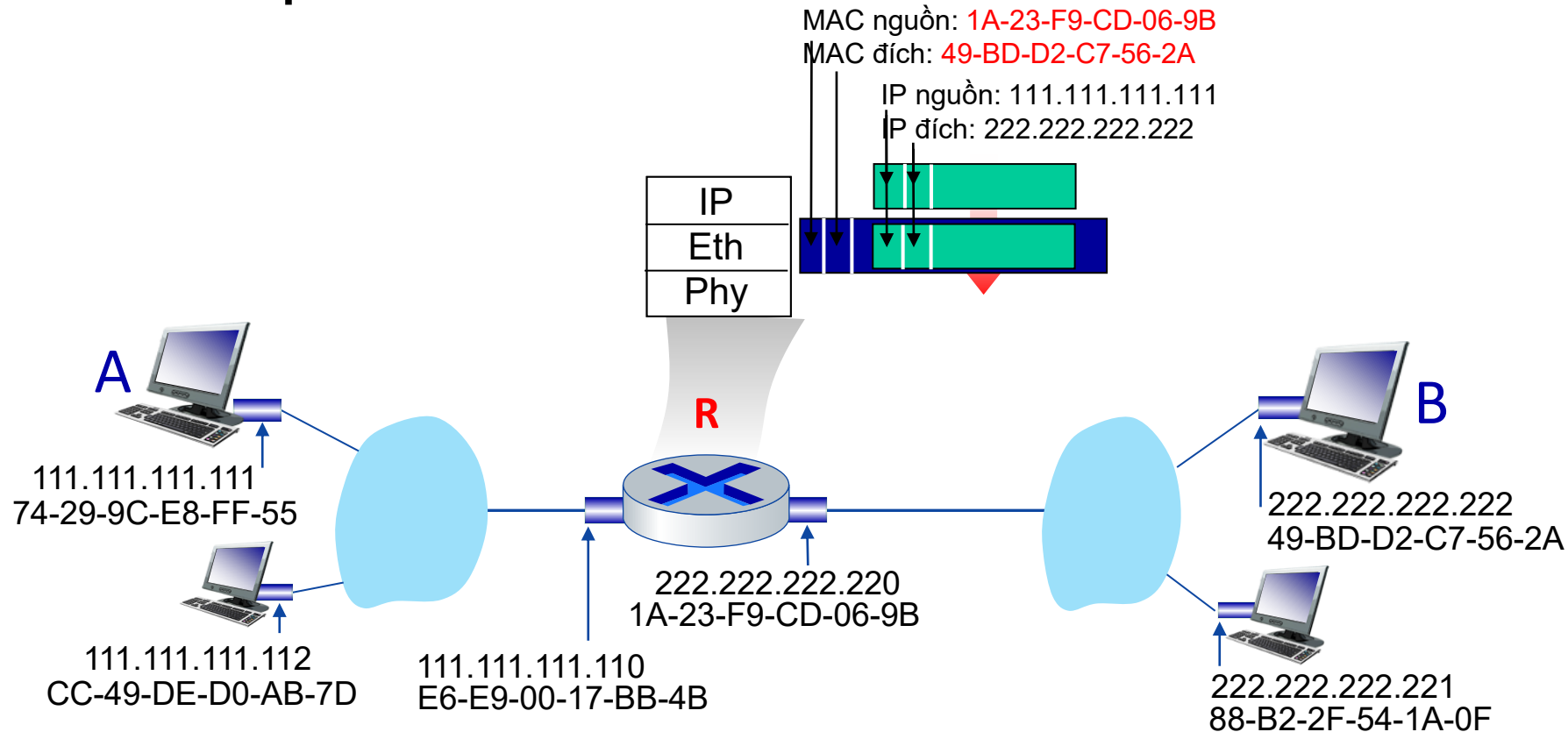
- Frame được gửi từ A đến R
- Frame nhận được tại R, datagram bị loại bỏ, được chuyển tới tầng IP tại R



# Định tuyến đến nút trong mạng con khác: gán địa chỉ



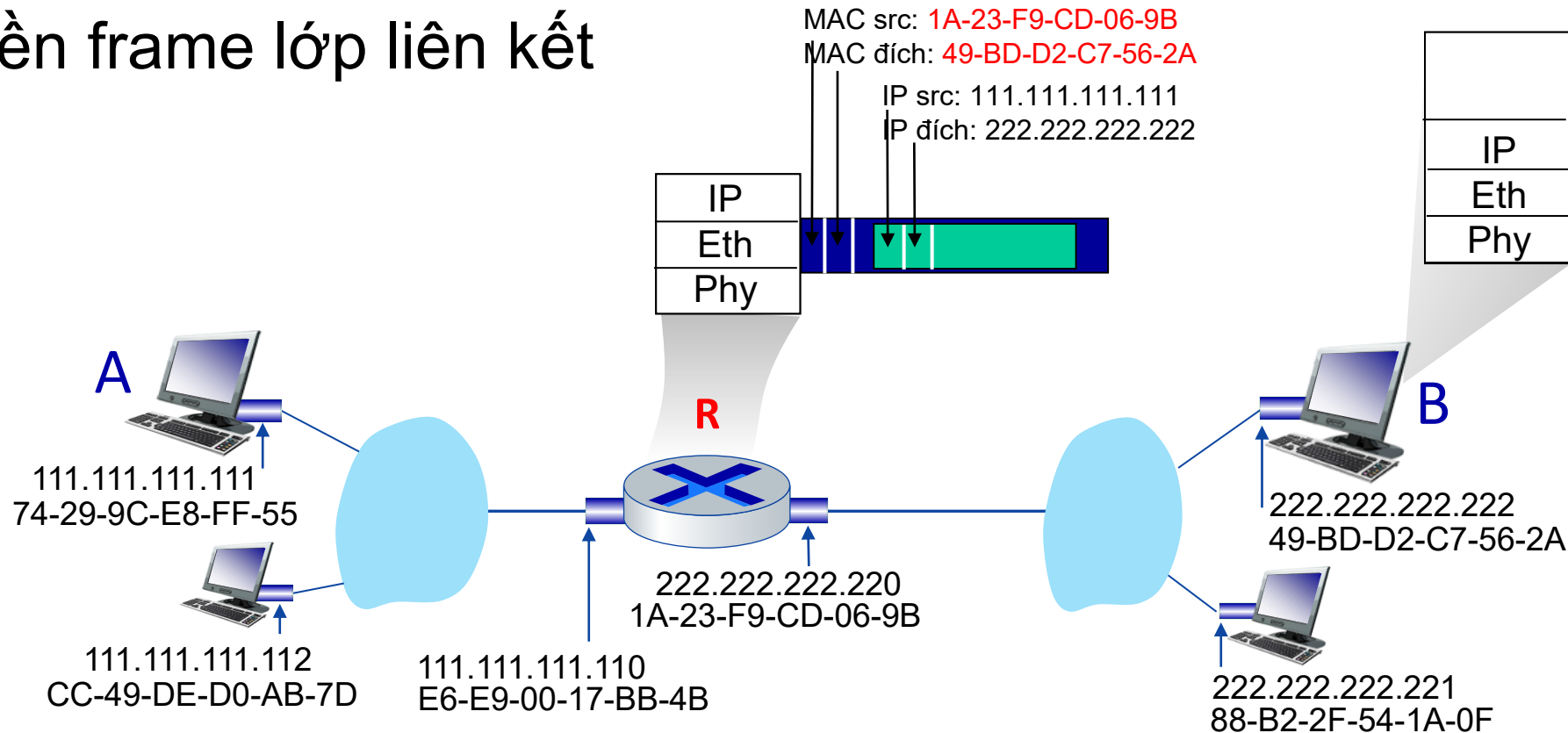
- R xác định cổng gửi đi, chuyển datagram với IP nguồn A, đích B tới tầng liên kết
- R tạo frame tầng liên kết chứa gói dữ liệu IP A-đến-B. Địa chỉ đích của frame là địa chỉ MAC của B



# Định tuyến đến nút trong mạng con khác: gán địa chỉ



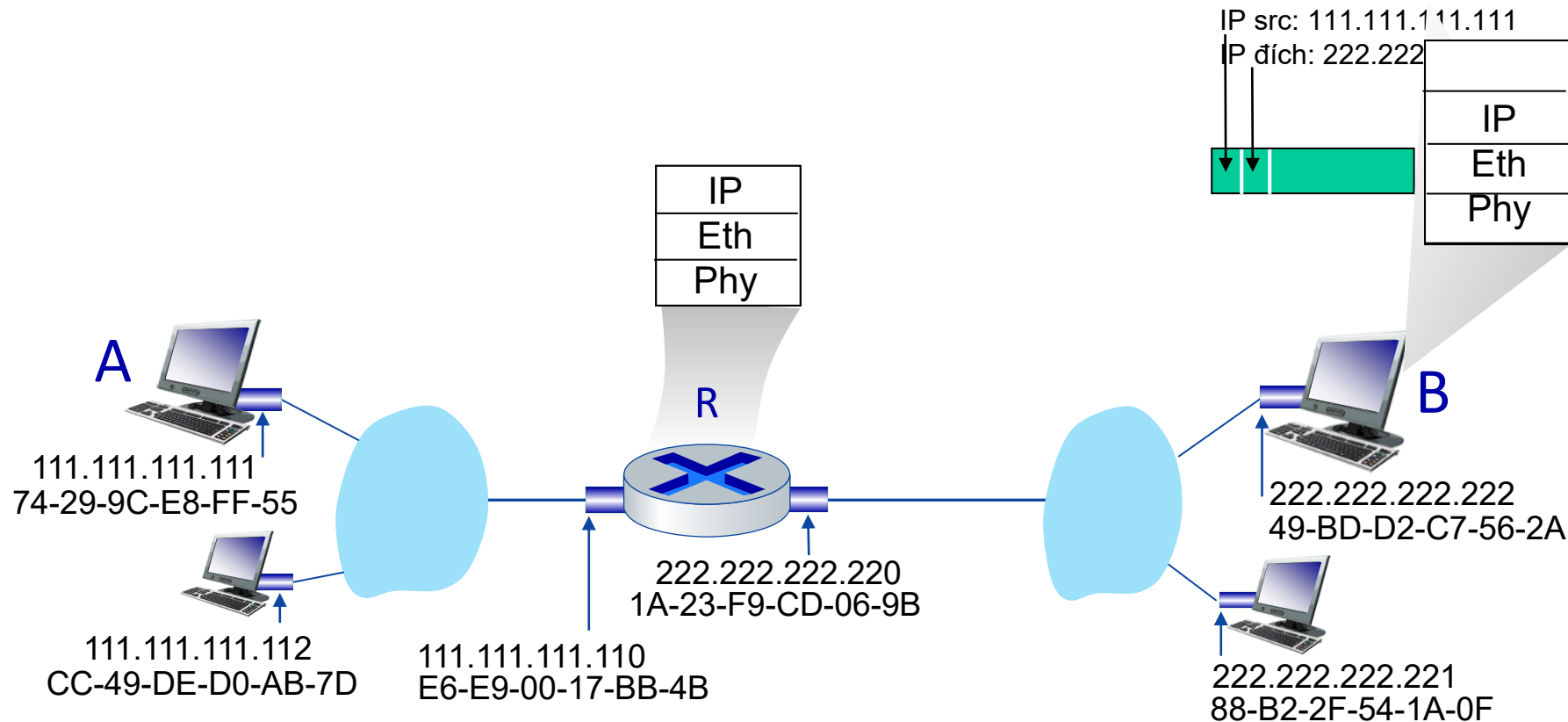
- R xác định cổng gửi đi, chuyển datagram với IP nguồn A, đích B tới lớp liên kết
- R tạo frame tầng liên kết chứa gói dữ liệu IP A-đến-B. Địa chỉ đích của frame là địa chỉ MAC của B
- Truyền frame lớp liên kết



# Định tuyến đến nút trong mạng con khác: gán địa chỉ



- B nhận frame, trích xuất địa chỉ IP datagram B
- B chuyển datagram lên tầng giao thức IP



# Tầng liên kết, LAN

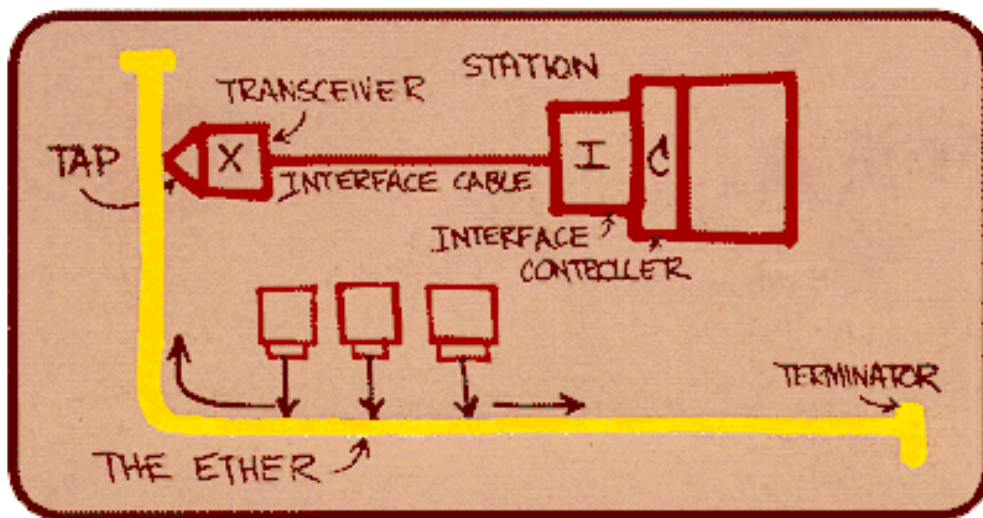


- Giới thiệu
- Phát hiện lỗi, sửa lỗi
- Nhiều giao thức truy cập
- **Mạng LAN**
  - Định địa chỉ, ARP
  - **Ethernet**
  - Switch
  - VLAN
- Ngữ cảnh của một yêu cầu dịch vụ web



Thống trị bởi công nghệ LAN có dây:

- Công nghệ LAN đầu tiên được sử dụng rộng rãi
- Đơn giản hơn, rẻ tiền
- Bắt kịp cuộc đua tốc độ: 10 Mbps – 400 Gbps
- Chip đơn, nhiều tốc độ (ví dụ: Broadcom BCM5761)



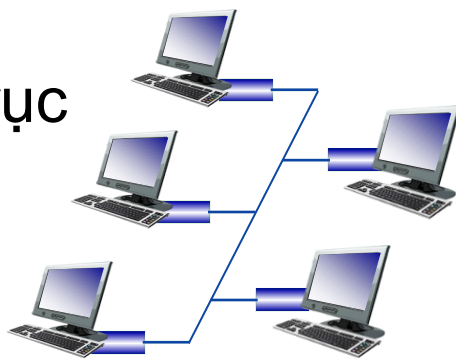
*Bản phác thảo  
Ethernet của Metcalfe*

# Ethernet: Các cấu trúc liên kết vật lý

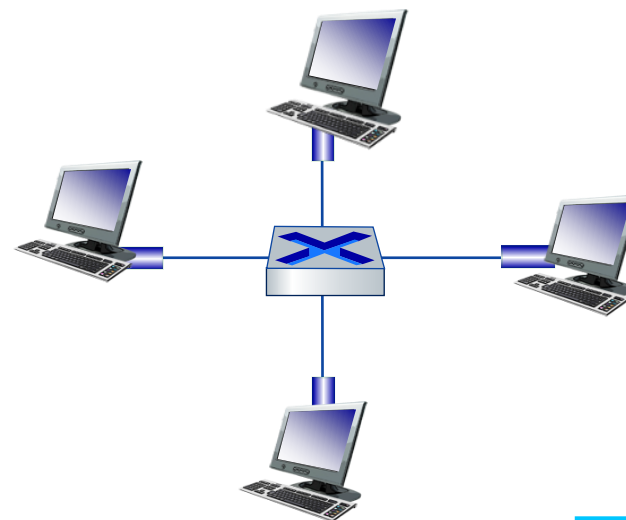


- **Bus:** phổ biến đến giữa những năm 90
  - Tất cả các nút cùng một miền xung đột (có thể xung đột với nhau khi nhiều hơn một nút truyền đồng thời)
- **Switch:** chiếm ưu thế ngày nay
  - **Thiết bị chuyển mạch (switch)** tăng 2 hoạt động ở trung tâm mạng
  - Mỗi “kết nối” chạy một giao thức Ethernet riêng biệt (vì vậy các nút không xung đột với nhau)

**Bus:** cáp đồng trục



**Chuyển mạch**





# Cấu trúc Ethernet frame



Giao diện gửi đóng gói gói dữ liệu IP (hoặc gói giao thức lớp mạng khác) trong **Ethernet frame**.



## *Trường mở đầu (Preamble):*

- Được sử dụng để đồng bộ hóa tốc độ đồng hồ của người nhận, người gửi
- 7 byte 10101010 theo sau là một byte 10101011

# Cấu trúc Ethernet frame (tiếp theo)



- **Địa chỉ:** địa chỉ MAC nguồn và MAC đích có 6 byte
  - Nếu một adapter nhận được frame có địa chỉ đích phù hợp hoặc địa chỉ đích là địa chỉ quảng bá (ví dụ: gói ARP), nó sẽ chuyển dữ liệu trong frame tới giao thức lớp mạng
  - Ngược lại, adapter sẽ loại bỏ frame
- **Kiểu:** biểu thị giao thức lớp cao hơn
  - Chủ yếu là kiểu IP nhưng cũng có thể có những kiểu khác, ví dụ: Novell IPX, AppleTalk
  - Được sử dụng để phân kênh trên tại máy nhận
- **CRC (cyclic redundancy check):** kiểm tra mã CRC tại máy nhận.
  - Nếu phát hiện lỗi: frame bị loại bỏ

# Ethernet: không tin cậy, không kết nối

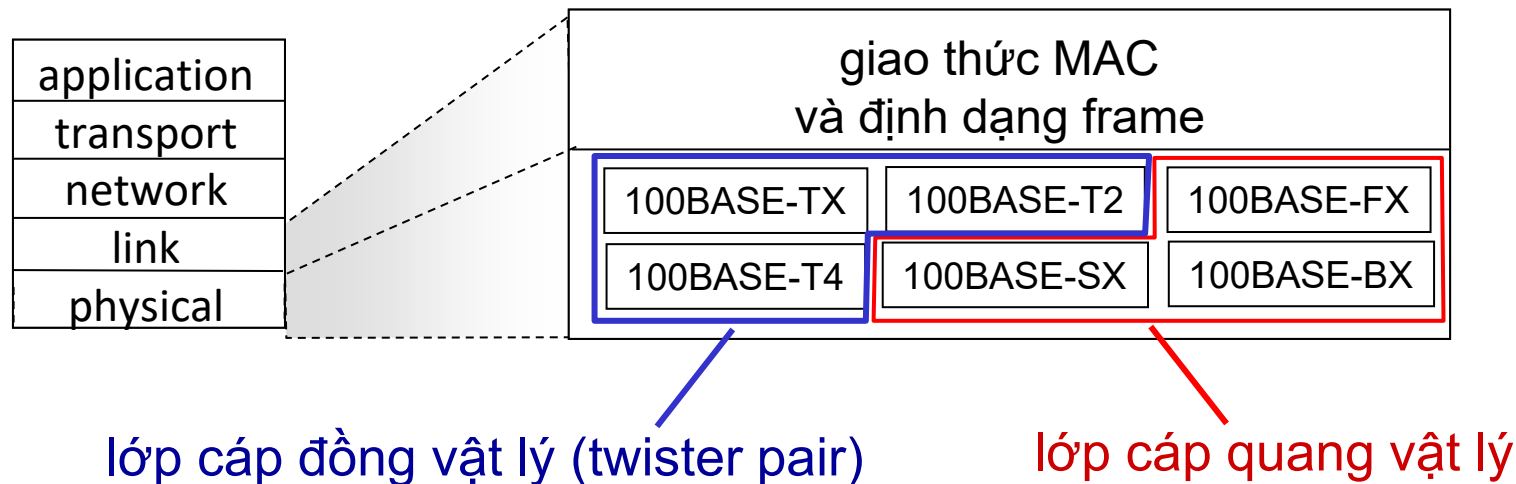


- **Không kết nối:** không bắt tay giữa NIC gửi và NIC nhận
- **Không tin cậy:** NIC nhận không gửi ACK hoặc NAK đến NIC gửi
  - Dữ liệu trong các frame bị mất chỉ được khôi phục nếu người gửi ban đầu sử dụng rdt lớp cao hơn (ví dụ: TCP), nếu không thì sẽ có mất mát dữ liệu
  - Giao thức đa truy cập của Ethernet: unslotted **CSMA/CD** với thuật toán **backoff nhị phân**

# Tiêu chuẩn Ethernet 802.3: tầng liên kết & tầng vật lý



- *Có nhiều* tiêu chuẩn Ethernet khác nhau
  - Giao thức MAC phổ biến và định dạng frame
  - Tốc độ khác nhau: 2 Mbps, 10 Mbps, 100 Mbps, 1Gbps, 10 Gbps, 40Gbps
  - Nhiều môi trường vật lý khác nhau: cáp sợi quang, cáp đồng...



# Tầng liên kết, LAN



- Giới thiệu
- Phát hiện lỗi, sửa lỗi
- Nhiều giao thức truy cập
- **Mạng LAN**
  - Định địa chỉ, ARP
  - Ethernet
  - **Switch**
  - VLAN
- Ngữ cảnh của một yêu cầu dịch vụ web



# Chuyển mạch Ethernet

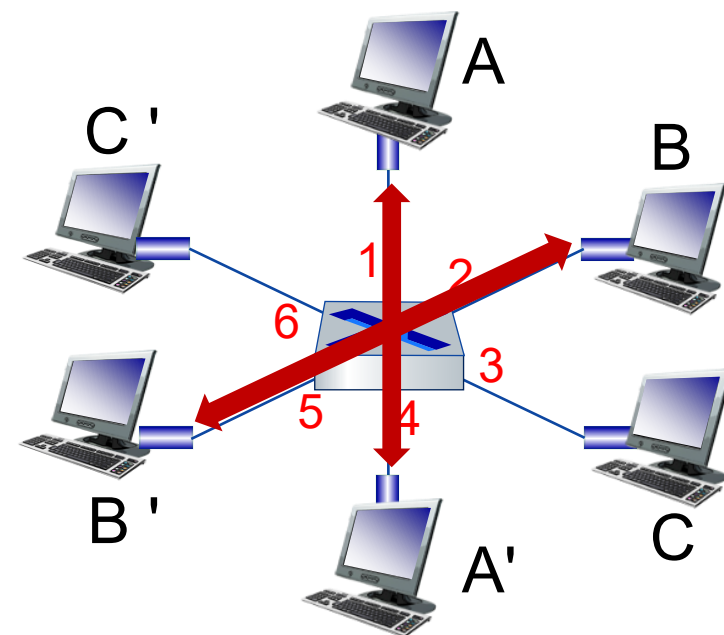


- Switch là thiết bị **tầng liên kết** : đóng vai trò *tích cực* trong việc:
  - Lưu trữ, chuyển tiếp frame
  - Kiểm tra địa chỉ MAC của frame đến, chuyển tiếp frame *có chọn lọc* tới một hoặc nhiều liên kết ra khi frame được chuyển tiếp trên phân đoạn, sử dụng CSMA/CD để truy cập phân đoạn
- **Trong suốt**: máy chủ *không biết* về sự hiện diện của thiết bị switch
- **Plug-and-play, tự học**
  - Switch không cần phải cấu hình, cắm là chạy.

# Switch: nhiều kết nối đồng thời



- Các máy chủ kết nối trực tiếp đến switch
- Switch lưu trữ đệm các gói tin
- Giao thức Ethernet được sử dụng trên *mỗi* liên kết đến switch, vì vậy:
  - Không xung đột; truyền hai chiều hoàn toàn
  - Mỗi liên kết là một miền xung đột riêng của nó
- **Chuyển mạch:** A-đến-A' và B-đến-B' có thể truyền đồng thời mà không có xung đột

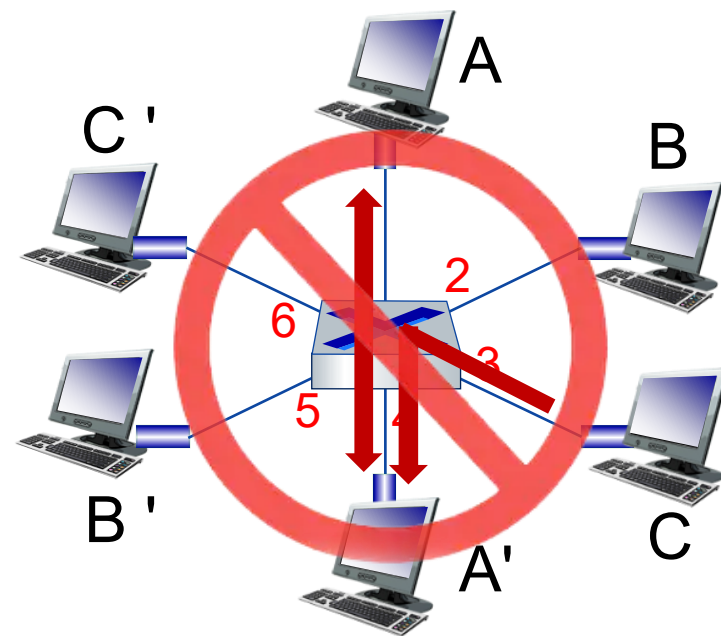


Switch với 6 cổng  
(1, 2, 3, 4, 5, 6)

# Switch: nhiều kết nối đồng thời



- Các máy chủ kết nối trực tiếp đến switch
- Switch lưu trữ đệm các gói tin
- Giao thức Ethernet được sử dụng trên *mỗi* liên kết đến switch, vì vậy:
  - Không xung đột; truyền hai chiều hoàn toàn
  - Mỗi liên kết là một miền xung đột riêng của nó
- **Chuyển mạch:** A-đến-A' và B-đến-B' có thể truyền đồng thời mà không có xung đột
  - Nhưng A-đến-A' và C-đến-A' *không thể* xảy ra đồng thời



Switch với 6 cổng  
(1, 2, 3, 4, 5, 6)



# Bảng chuyển tiếp trong switch



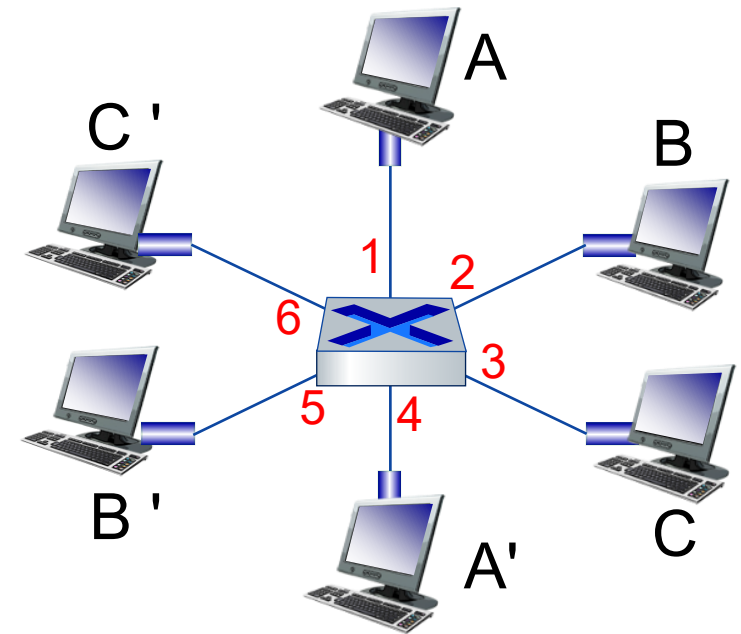
Hỏi: làm cách nào để switch biết A' có thể truy cập qua cổng 4, B' có thể truy cập qua cổng 5?

Đáp: mỗi switch có một **bảng chuyển đổi**, trong đó có nhiều mục, mỗi mục có:

- Địa chỉ MAC của máy chủ, cổng của switch nối với máy chủ, nhãn thời gian)
- Nó giống như một bảng định tuyến!

Hỏi: các mục được tạo, duy trì trong bảng chuyển đổi như thế nào?

- Nguyên tắc hoạt động có giống như một giao thức định tuyến?

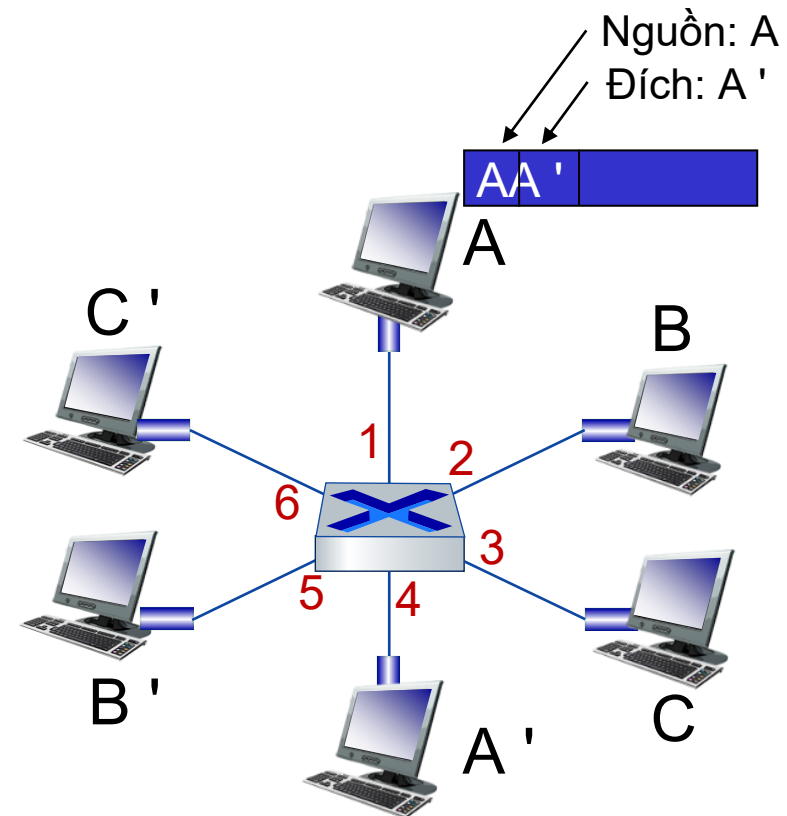


Switch với 6 cổng  
(1, 2, 3, 4, 5, 6)

# Switch: tự học



- Switch tự **học** để biết máy nào kết nối tới cổng nào của nó
- Khi nhận được frame, switch sẽ biết được vị trí của máy gửi ở phân đoạn mạng LAN nào
- Ghi lại cặp máy gửi – số cổng vào bảng chuyển tiếp



| Địa chỉ MAC | Cổng | TTL |
|-------------|------|-----|
| A           | 1    | 60  |
|             |      |     |
|             |      |     |
|             |      |     |
|             |      |     |

*Bảng Chuyển tiếp  
(ban đầu trống rỗng)*

# Switch: lọc/chuyển tiếp frame



Khi frame nhận được tại switch:

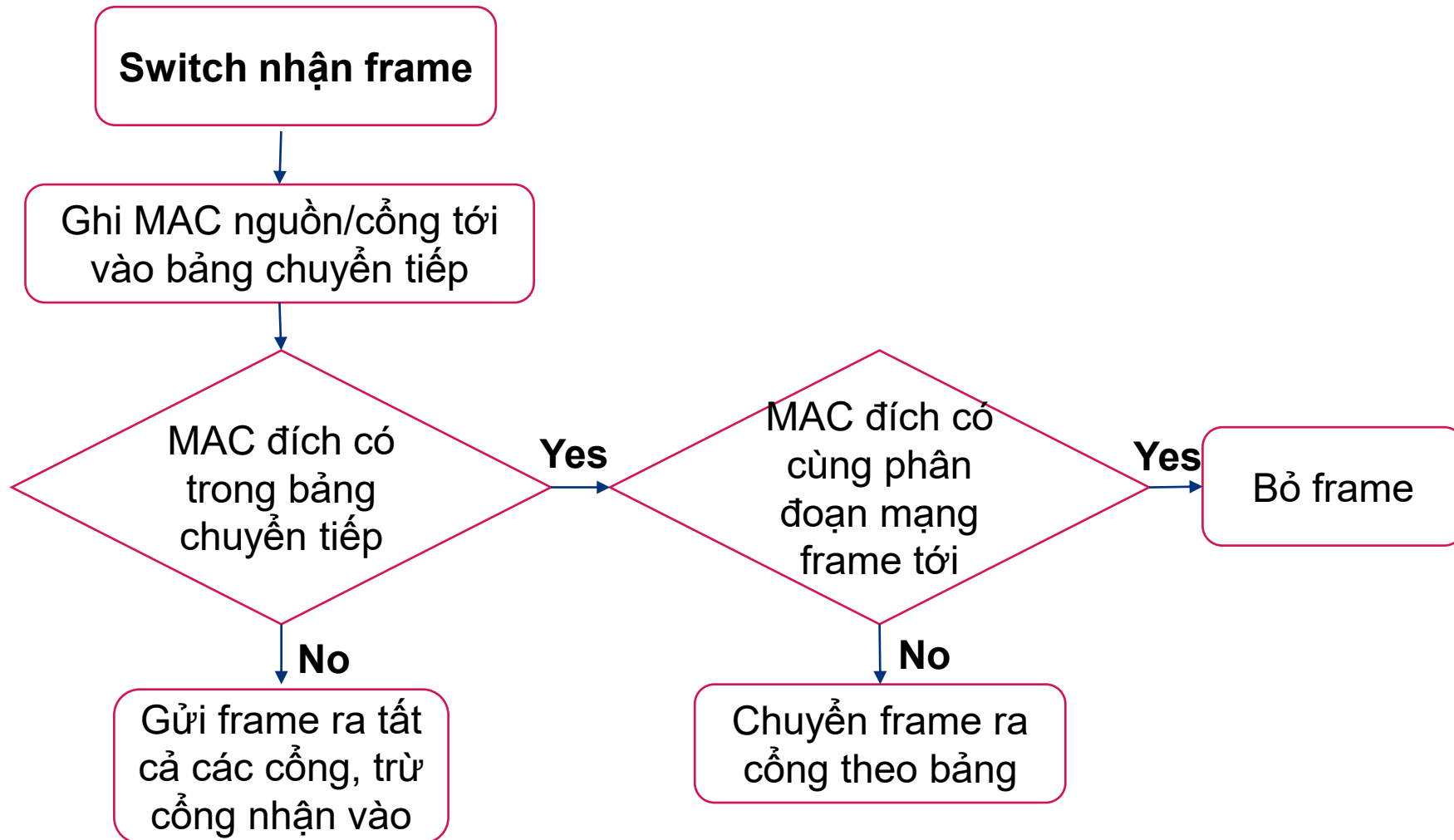
1. Ghi lại số cổng mà frame tới, địa chỉ MAC của máy chủ gửi
2. Tra cứu bảng chuyển tiếp sử dụng địa chỉ MAC đích
3. **Nếu** có một bảng ghi được tìm thấy cho đích tới của frame  
    **kiểm tra tiếp {**  
        **nếu** đích tới của frame trùng với phân đoạn mạng phát ra frame  
        **bỏ** frame  
        **ngược lại thì** chuyển tiếp frame ra cổng được chỉ định bởi bảng  
        ghi trong bảng chuyển tiếp.  
    **}**

**Ngược lại thì flood** /\* gửi frame ra tất cả các cổng trừ cổng nhận frame vào\*/

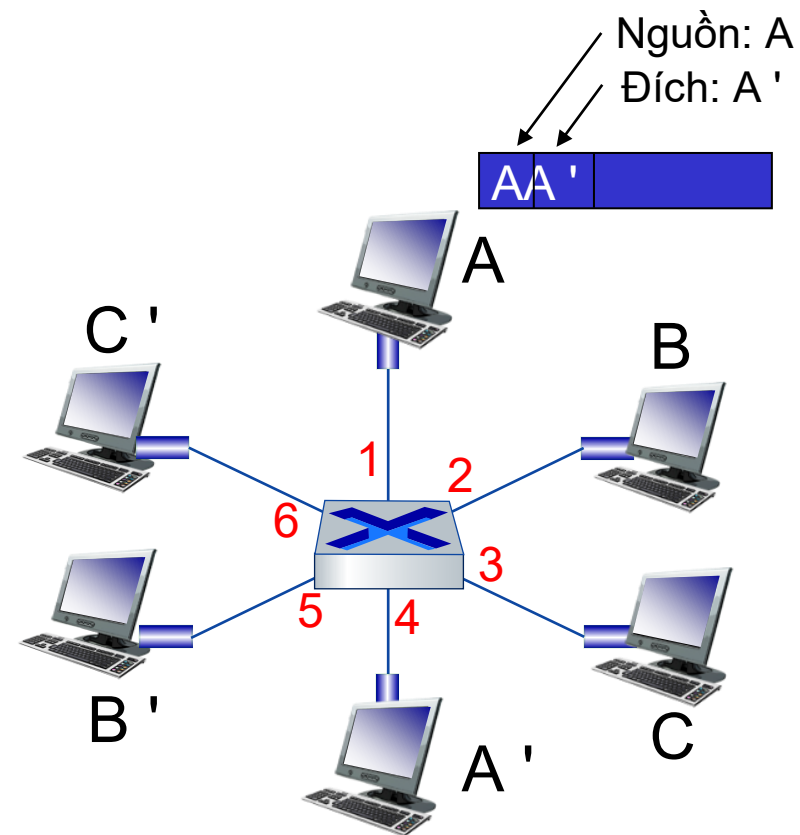
# Switch: lọc/chuyển tiếp frame



khi frame nhận được tại switch:



# Tự học, chuyển tiếp: ví dụ



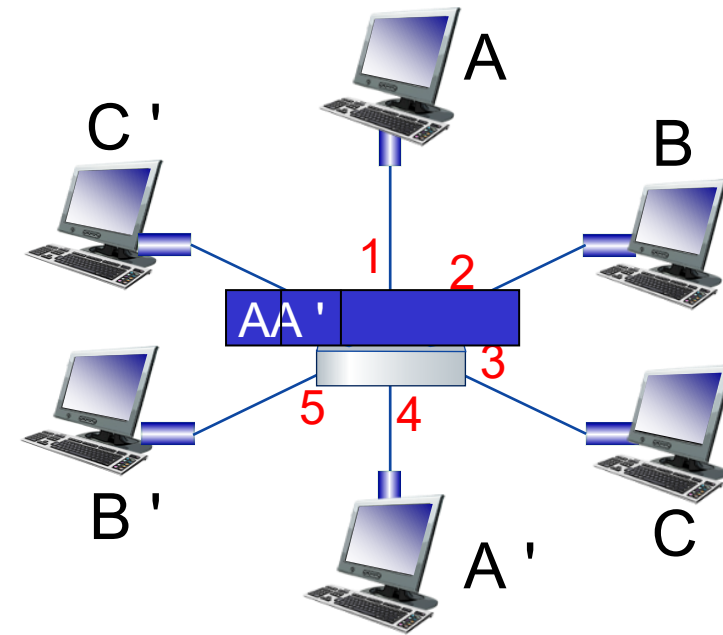
| Địa chỉ MAC | Cổng | TTL |
|-------------|------|-----|
|             |      |     |

*bảng chuyển đổi  
(ban đầu trống rỗng)*

# Tự học, chuyển tiếp: ví dụ



- Đích đến frame là A', vị trí không xác định:



Tự thêm khi nhận frame  
từ A qua cổng 1

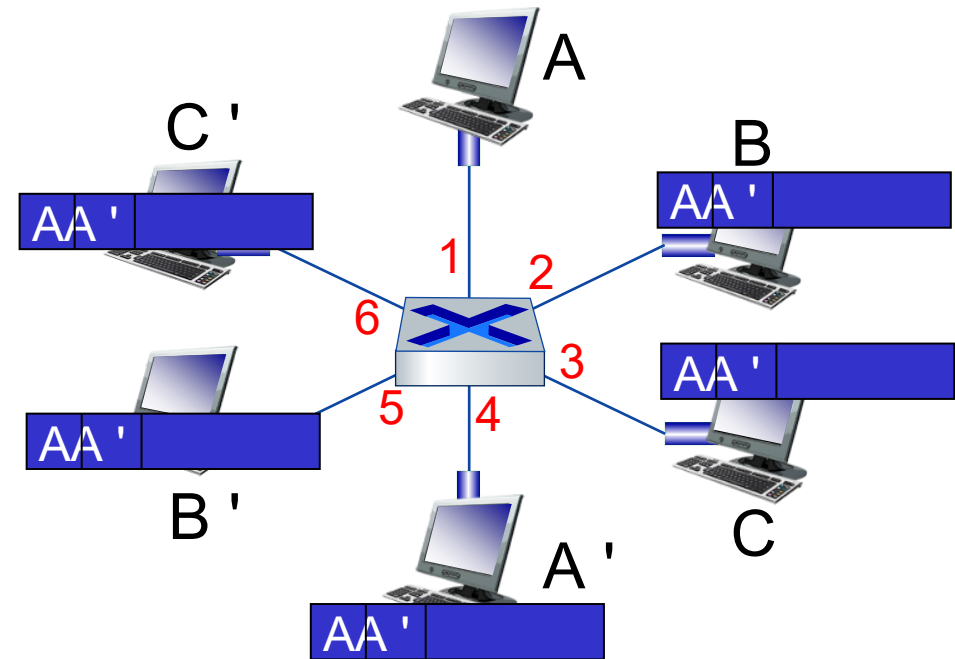
| Địa chỉ MAC | Cổng | TTL |
|-------------|------|-----|
| A           | 1    | 60  |
|             |      |     |
|             |      |     |
|             |      |     |

*bảng chuyển đổi  
(ban đầu trống rỗng)*

# Tự học, chuyển tiếp: ví dụ



- Đích đến frame là A', vị trí không xác định: **gửi flood**  
(trừ cổng nhận được frame)



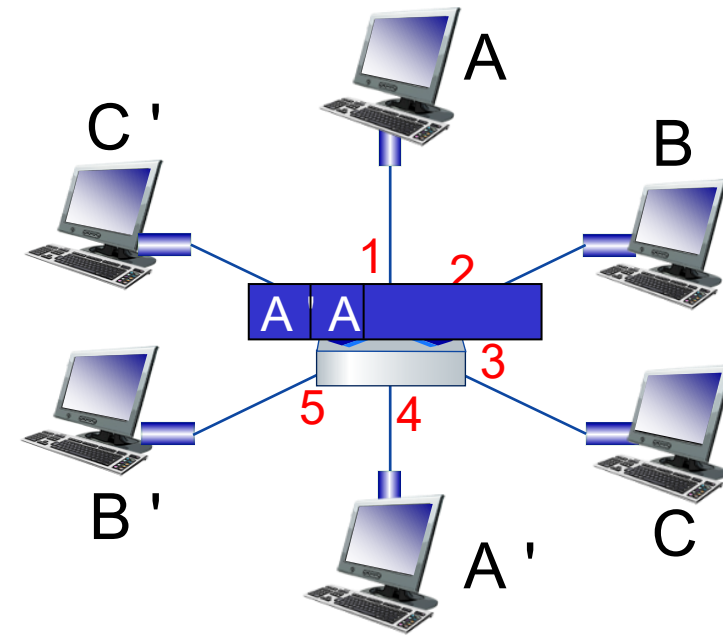
| Địa chỉ MAC | Cổng | TTL |
|-------------|------|-----|
| A           | 1    | 60  |

*bảng chuyển đổi  
(ban đầu trống rỗng)*

# Tự học, chuyển tiếp: ví dụ



- Đích đến frame là A', vị trí không xác định: **gửi flood**  
(trừ cổng nhận được frame)
- Điểm đến A, vị trí đã được biết:



| Địa chỉ MAC | Cổng | TTL |
|-------------|------|-----|
| A           | 1    | 60  |

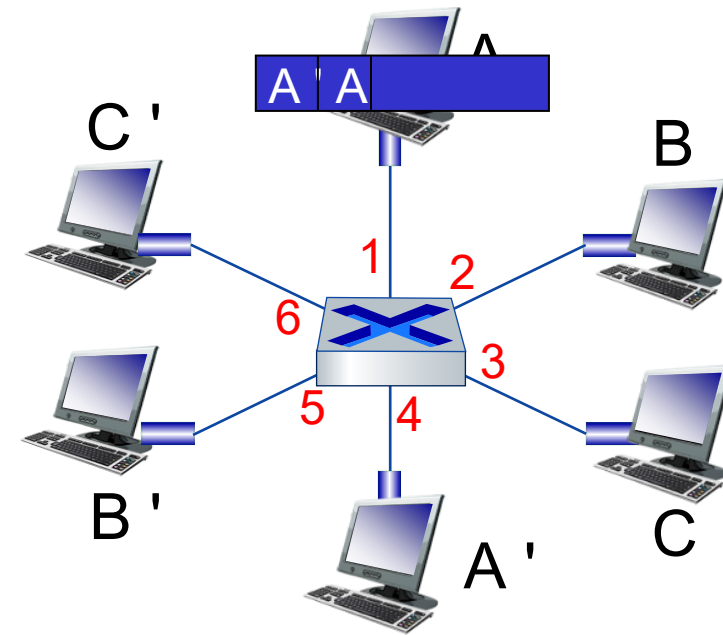
*bảng chuyển đổi  
(ban đầu trống rỗng)*



# Tự học, chuyển tiếp: ví dụ



- Đích đến frame là A', vị trí không xác định: **gửi flood**  
(trừ cổng nhận được frame)
- Điểm đến A, vị trí đã được biết: **gửi có chọn lọc chỉ trên một liên kết**



Tự thêm khi nhận frame  
từ A' qua cổng 4

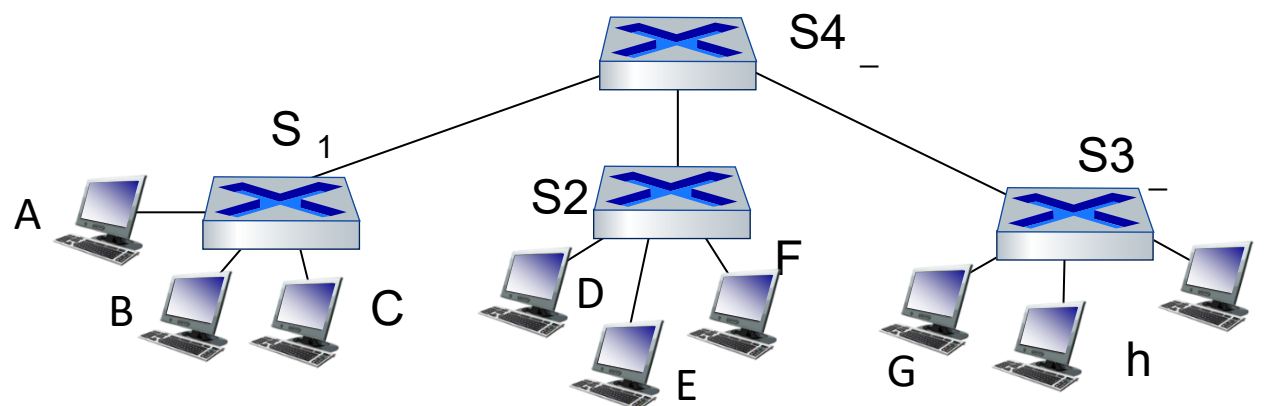
| Địa chỉ MAC | Cổng | TTL |
|-------------|------|-----|
| A           | 1    | 60  |
| A'          | 4    | 60  |

*bảng chuyển đổi  
(ban đầu trống rỗng)*

# Kết nối nhiều Switch



Các Switch tự học có thể được kết nối với nhau:



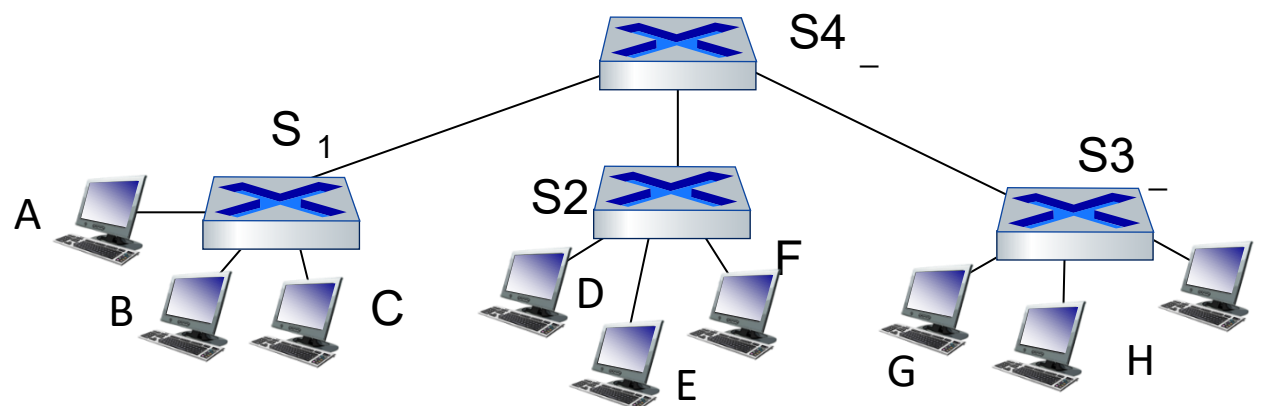
**Hỏi:** gửi từ A đến G - làm thế nào để S<sub>1</sub> biết chuyển tiếp frame đến G thông qua S<sub>4</sub> và S<sub>3</sub> ?

- **Trả lời:** tự học! (hoạt động giống hệt như trong trường hợp một Switch!)

# Ví dụ về nhiều Switch tự học

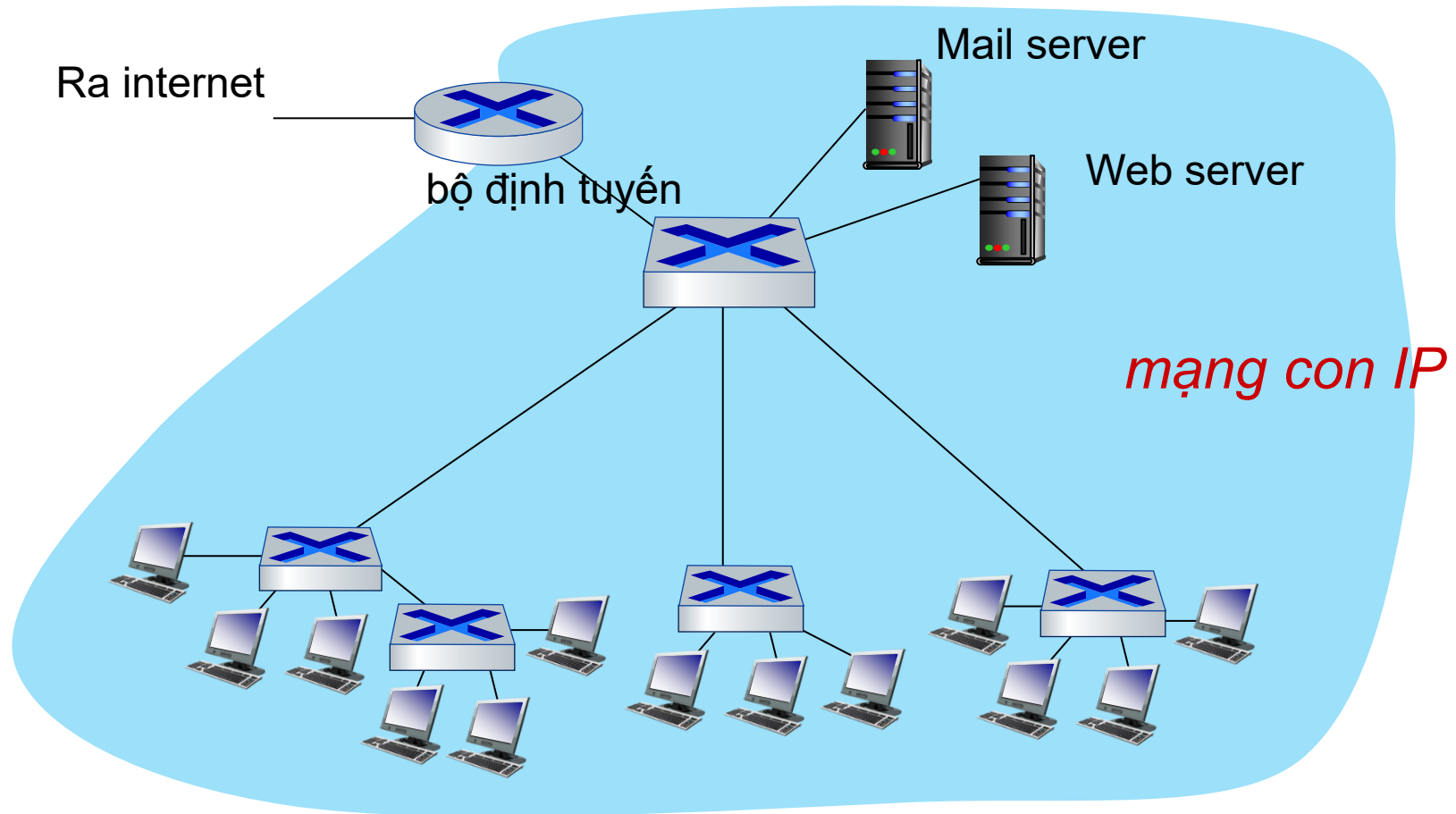


Giả sử C gửi frame cho I, sau đó I trả lời C



Hỏi: hiển thị bảng chuyển đổi và chuyển tiếp gói trong  $S_1$ ,  $S_2$ ,  $S_3$ ,  $S_4$

# Mạng máy tính trong tổ chức nhỏ



# Switch so với bộ định tuyến (router)

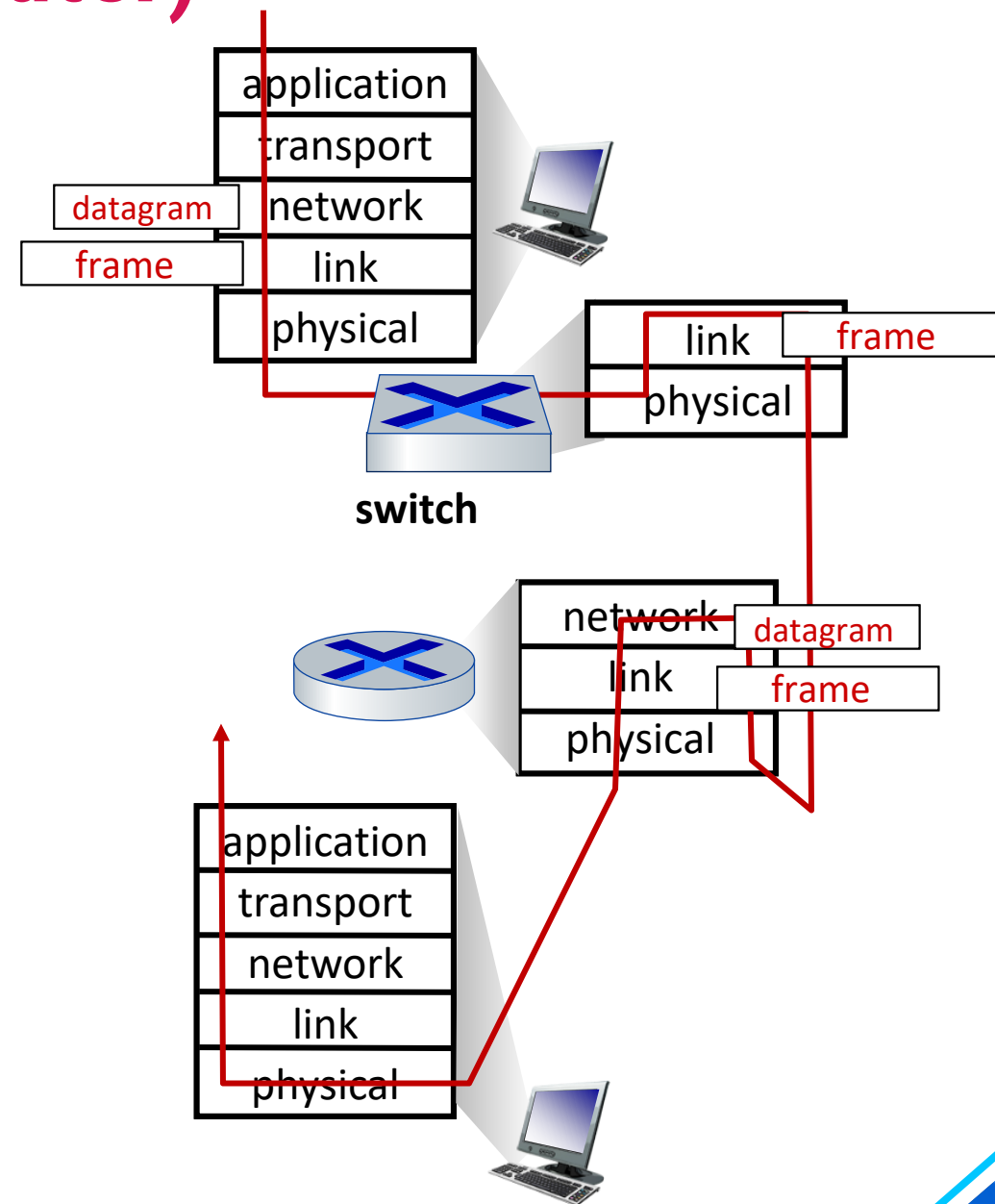


Cả hai đều là lưu trữ và chuyển tiếp:

- **Bộ định tuyến:** thiết bị tầng mạng (kiểm tra header tầng mạng)
- **Switch:** thiết bị lớp tầng kết (kiểm tra header tầng liên kết)

Cả hai đều có bảng chuyển tiếp:

- **Bộ định tuyến:** tính toán bảng bằng thuật toán định tuyến, địa chỉ IP
- **Switch:** học và tạo bảng chuyển tiếp bằng cách flooding, tự học, ghi nhận địa chỉ MAC



# Tầng liên kết, LAN



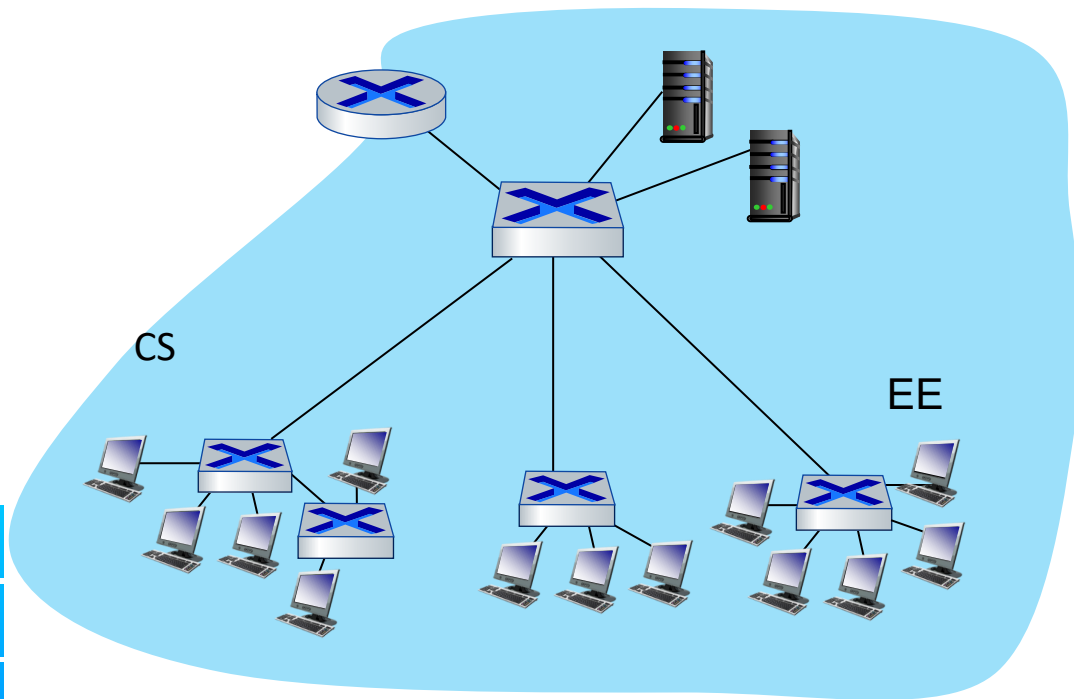
- Giới thiệu
- Phát hiện lỗi, sửa lỗi
- Nhiều giao thức truy cập
- **Mạng LAN**
  - Định địa chỉ, ARP
  - Ethernet
  - Switch
  - **VLAN**
- Ngữ cảnh của một yêu cầu dịch vụ web



# Mạng LAN ảo (VLAN): nhu cầu thực tế



**Hỏi:** điều gì xảy ra khi mở rộng kích thước mạng LAN, người dùng thay đổi vị trí (port) gắn vào mạng LAN ?



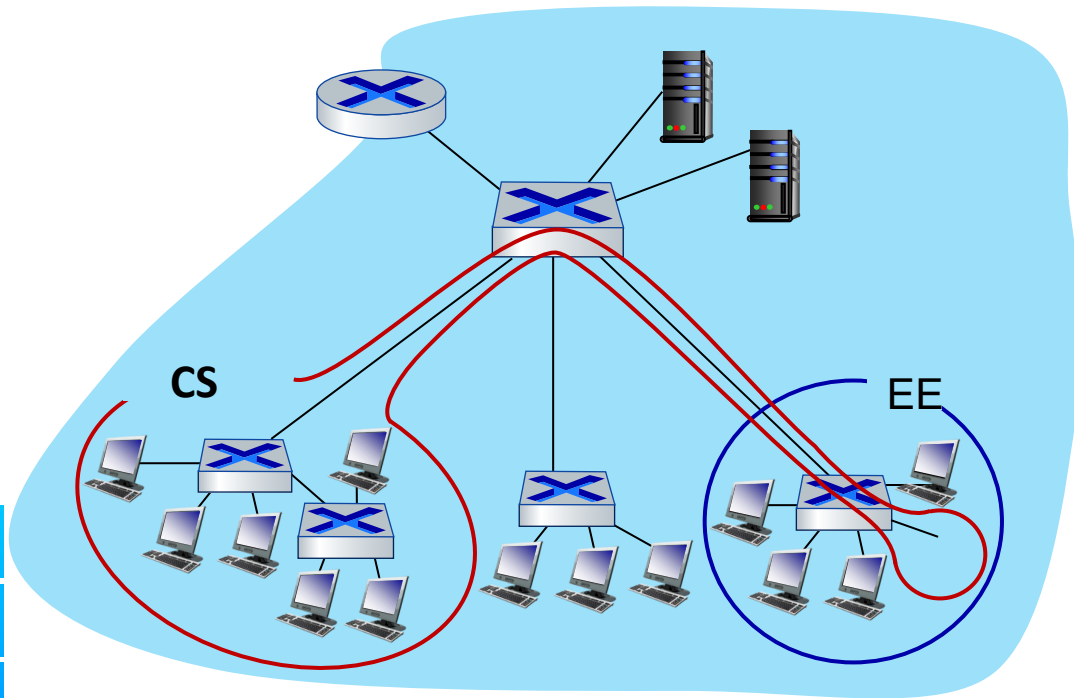
**Miền quảng bá duy nhất:**

- **Kích thước mở rộng:** tất cả lưu lượng quảng bá tầng 2 (ARP, DHCP, MAC chưa biết) phải gửi qua toàn bộ mạng LAN
- Vấn đề hiệu quả, bảo mật, quyền riêng tư

# Mạng LAN ảo (VLAN): nhu cầu thực tế



**Hỏi:** điều gì xảy ra khi mở rộng kích thước mạng LAN, người dùng thay đổi vị trí (port) gắn vào mạng LAN ?



**Miền quảng bá duy nhất:**

- **Kích thước mở rộng:** tất cả lưu lượng quảng bá tầng 2 (ARP, DHCP, MAC chưa biết) phải gửi qua toàn bộ mạng LAN
- Vấn đề hiệu quả, bảo mật, quyền riêng tư

**Những vấn đề quản trị:**

- Người dùng phòng CS chuyển văn phòng sang EE - được kết nối **vật lý** vào switch của phòng EE, nhưng muốn duy trì kết nối **logic** với switch tại CS



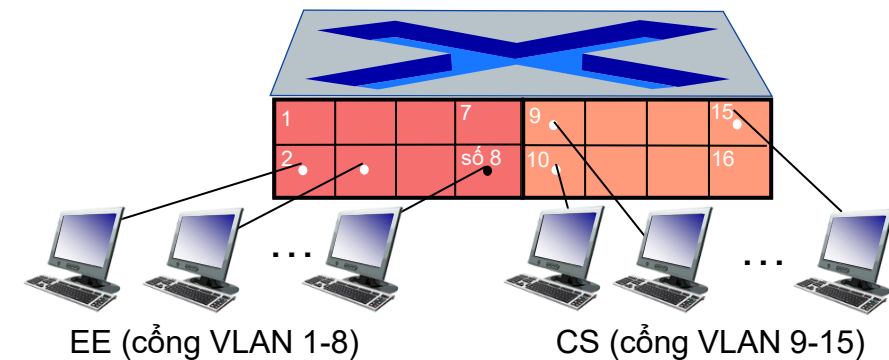
# VLAN tạo theo cổng (port-based VLAN)



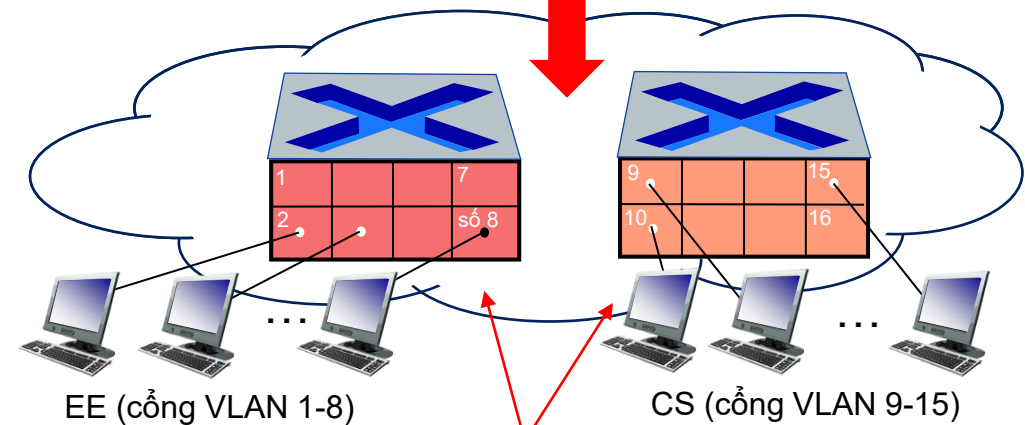
## Mạng cục bộ ảo (VLAN)

(Các) Switch có khả năng hỗ trợ cấu hình để tạo ra nhiều mạng LAN ảo từ một LAN vật lý.

**port-based VLAN:** các cổng của switch có thể được nhóm lại (bằng phần mềm quản lý Switch) như là *một* Switch vật lý



*single* physical switch

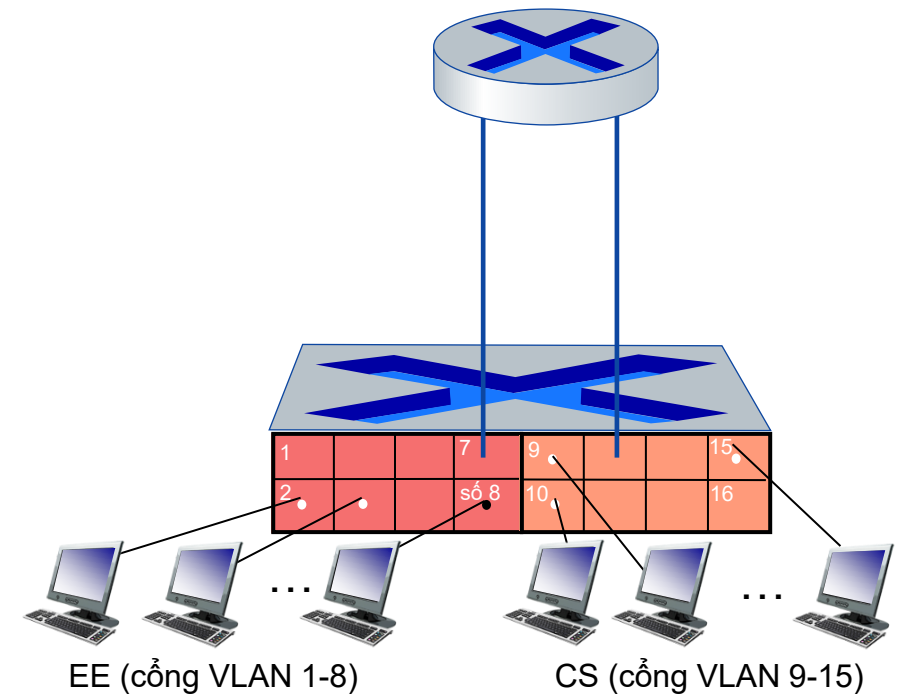


*multiple* virtual switches

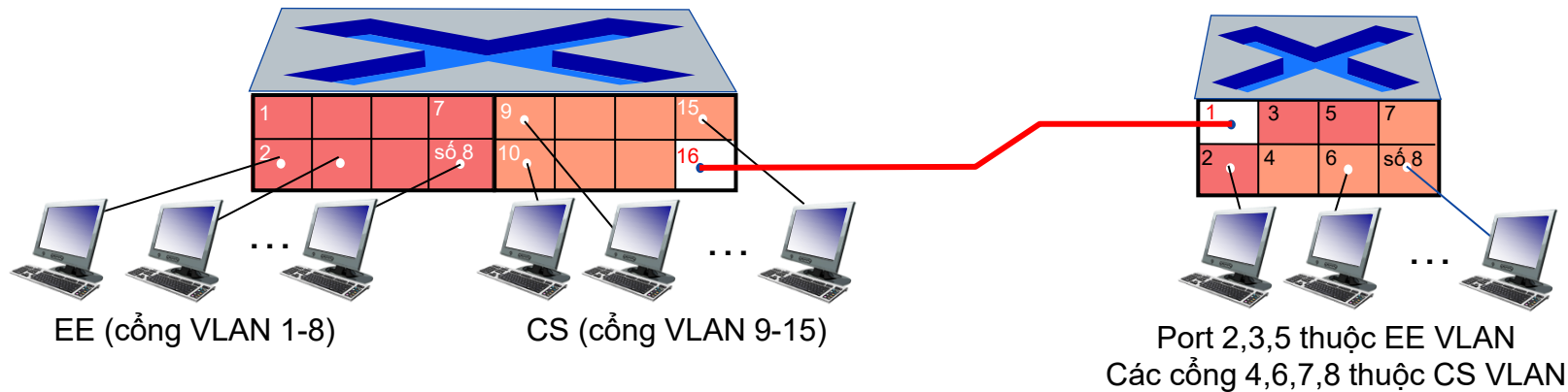
# VLAN tạo theo cổng (port-based VLAN)



- **Cách ly lưu lượng:** các frame đến/từ cổng 1-8 *chỉ có thể* đến cổng 1-8
  - Cũng có thể xác định Vlan dựa trên địa chỉ MAC của đầu cuối, thay vì cổng của switch
- **Thành viên động :** các cổng có thể được gán động (bằng lệnh) giữa các Vlan
- **Chuyển tiếp giữa các Vlan:** được thực hiện thông qua định tuyến (giống như với các Switch riêng biệt)
  - Trong thực tế, các nhà cung cấp bán switch kết hợp chung với bộ định tuyến



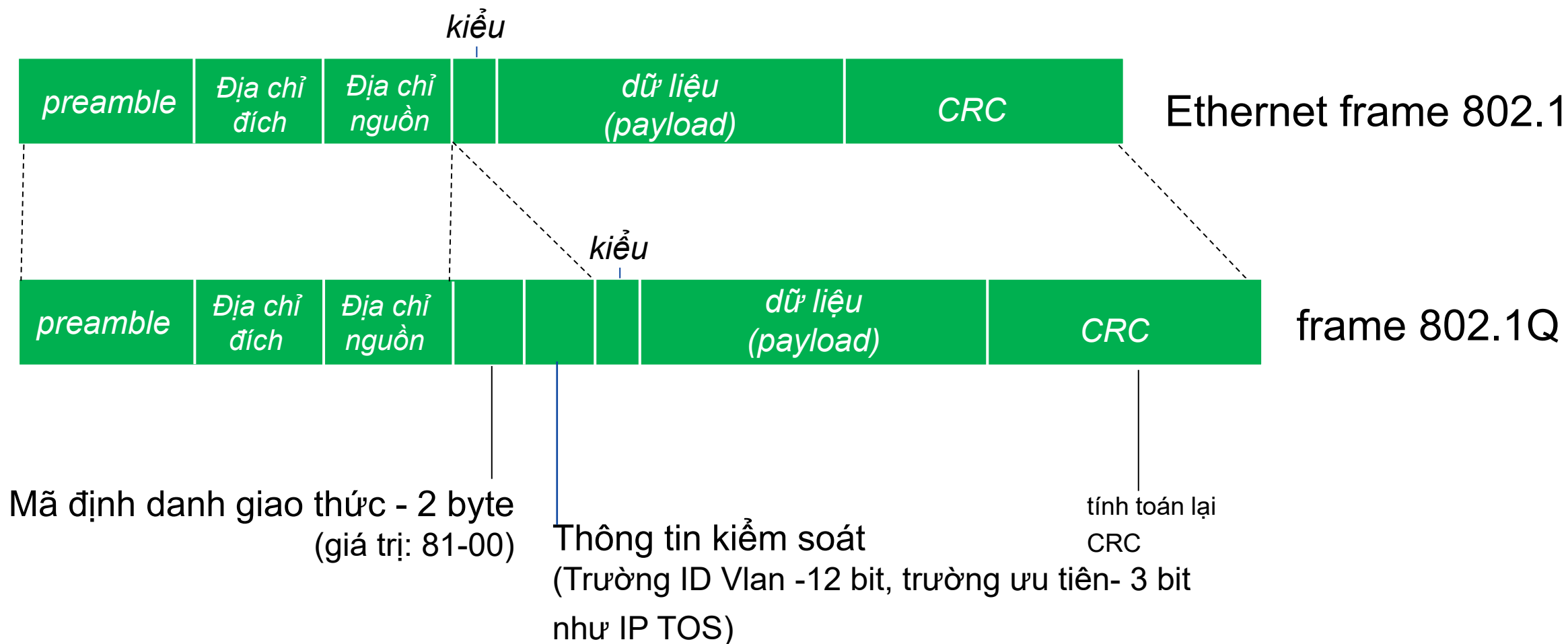
# VLAN mở rộng trên nhiều switch



**Trunk port:** cho phép các frame của các Vlan khác nhau trên nhiều thiết bị switch vật lý khác nhau đi qua

- Các frame được chuyển tiếp trong Vlan giữa các thiết bị switch vật lý không thể là frame chuẩn 802.1 (do phải mang thông tin Vlan ID để phân biệt frame nào thuộc vlan nào)
- Giao thức 802.1q thêm/xóa các trường bổ sung trong header của các frame được chuyển tiếp qua các cổng Trunk.

# Định dạng frame VLAN 802.1Q



# Tầng liên kết, LAN



- Giới thiệu
- Phát hiện lỗi, sửa lỗi
- Các giao thức đa truy cập
- Mạng LAN
  - Địa chỉ MAC, ARP
  - Ethernet
  - Switch
  - VLAN
- Ngữ cảnh của một yêu cầu dịch vụ web

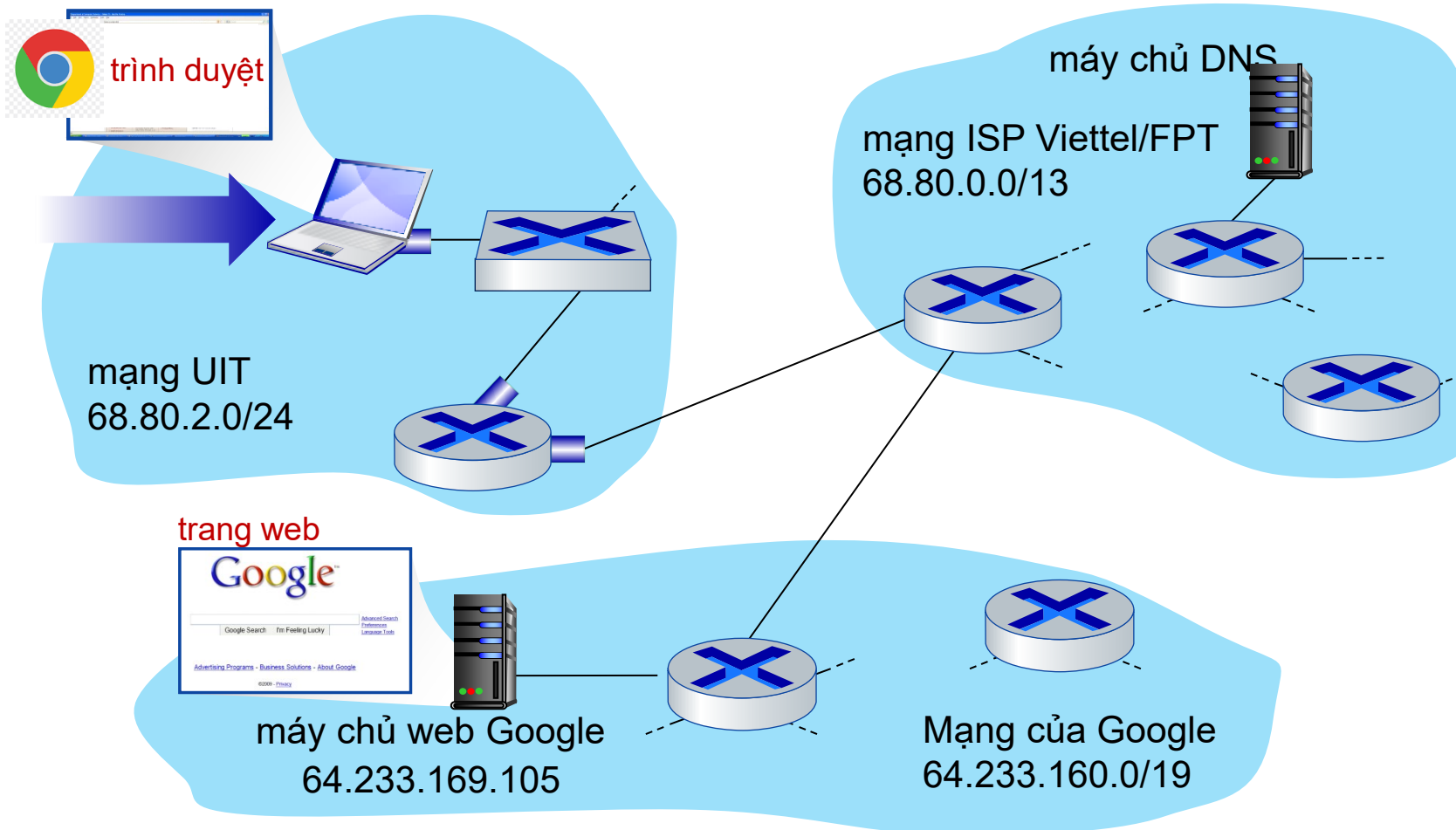


# Tổng hợp: Ngữ cảnh của một yêu cầu dịch vụ web



- Hành trình của chúng ta trong quá trình tìm hiểu mô hình mạng máy tính từ cao-xuống-thấp đã hoàn tất!
  - Thứ tự là: ứng dụng -> vận chuyển -> mạng -> liên kết
- Chúng ta sẽ có một ngữ cảnh kết hợp tất cả lại với nhau:
  - **Mục tiêu:** nhận định, xem xét, hiểu các giao thức (ở tất cả các tầng) liên quan đến tình huống có vẻ đơn giản như sau: yêu cầu trang Web.
  - **Kịch bản:** sinh viên gán máy tính xách tay vào mạng của trường, yêu cầu/nhận nội dung từ địa chỉ: [www.google.com](http://www.google.com)

# Yêu cầu dịch vụ web: kịch bản



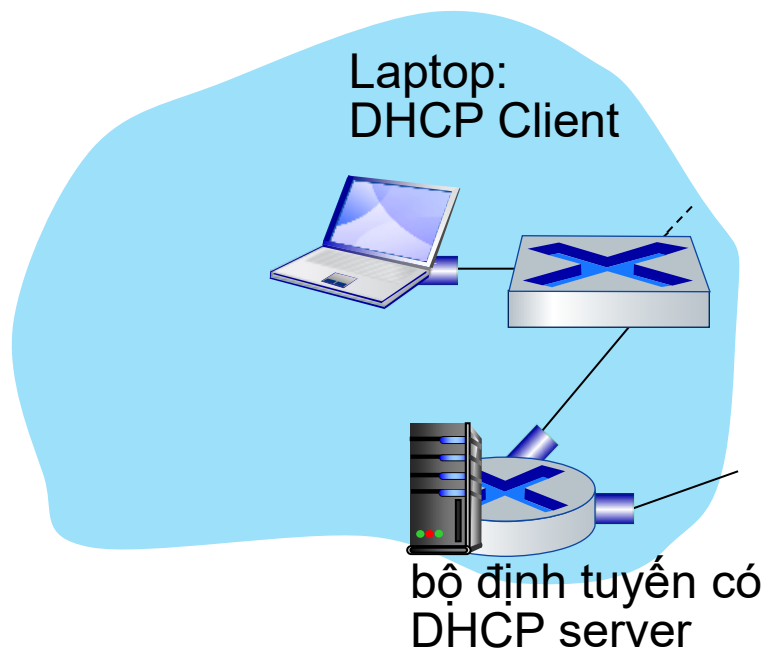
Kịch bản :

- Máy khách di động đến gần vào mạng...
- Yêu cầu trang web:  
[www.google.com](http://www.google.com)

*Đơn giản  
như đang  
giỡn :D*



# Yêu cầu dịch vụ web: kết nối với Internet





# Yêu cầu dịch vụ web: kết nối với Internet



DHCP

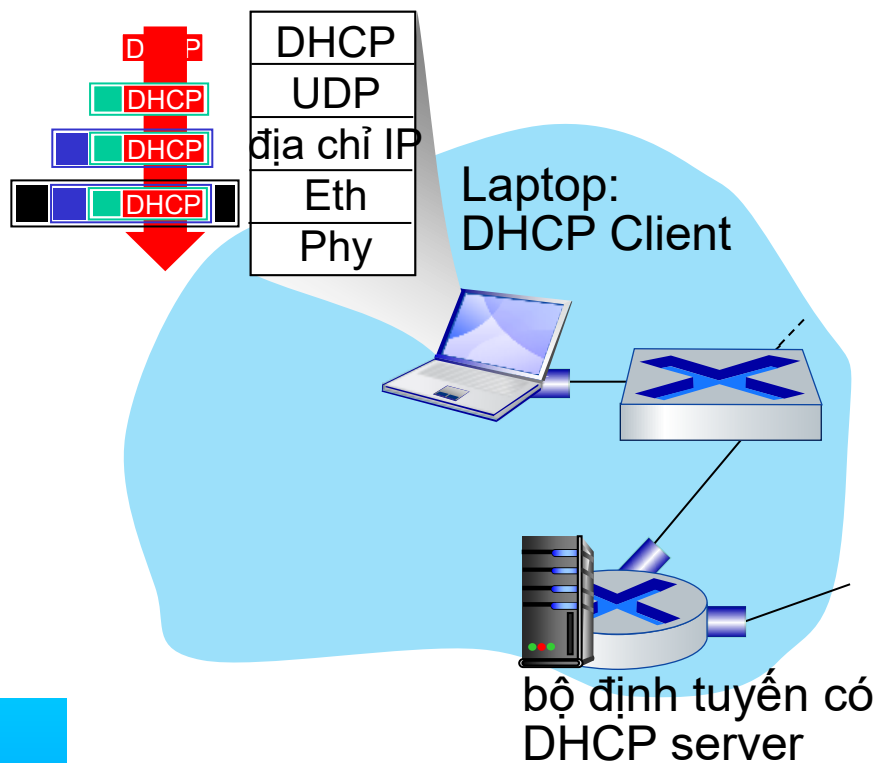
|            |
|------------|
| DHCP       |
| UDP        |
| địa chỉ IP |
| Eth        |
| Phy        |

Laptop:  
DHCP Client

- Laptop của bạn kết nối mạng cần có địa chỉ IP cho nó, IP của default gateway, IP của máy chủ DNS: sử dụng **DHCP**

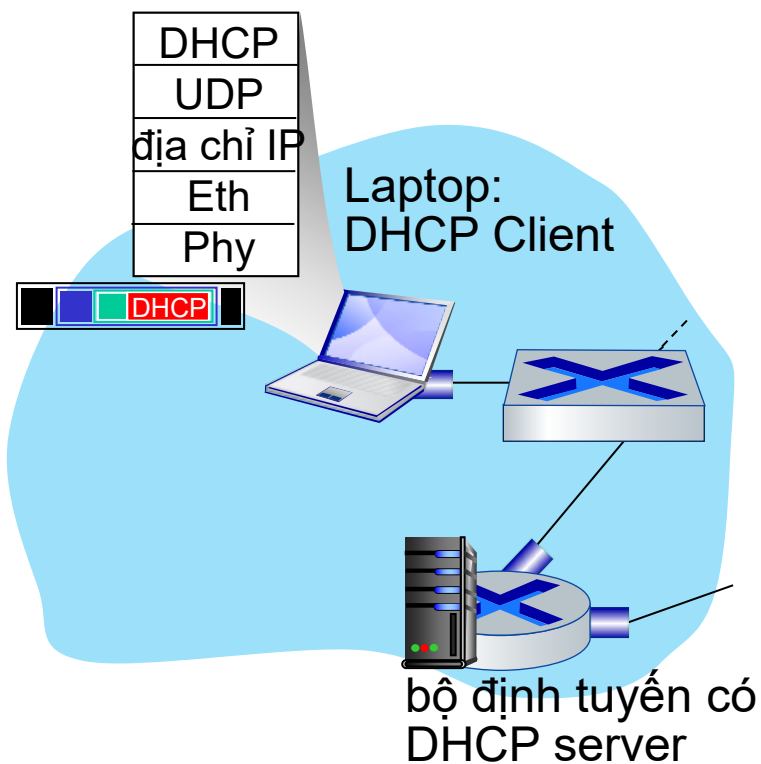
bộ định tuyến có  
DHCP server

# Yêu cầu dịch vụ web: kết nối với Internet



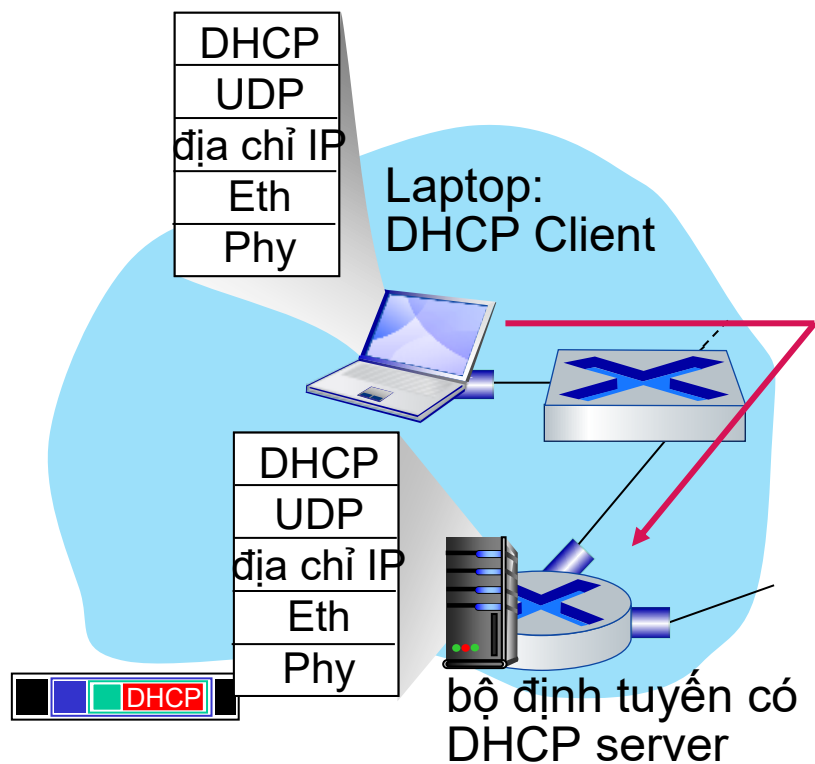
- Laptop của bạn kết nối mạng cần có địa chỉ IP cho nó, IP của default gateway, IP của máy chủ DNS: sử dụng **DHCP**
  - Yêu cầu DHCP được đóng gói trong gói tin **UDP** -> được đóng gói tiếp trong gói tin **IP** -> tiếp theo là được đóng gói trong Ethernet **802.3**

# Yêu cầu dịch vụ web: kết nối với Internet



- Laptop của bạn kết nối mạng cần có địa chỉ IP cho nó, IP của default gateway, IP của máy chủ DNS: sử dụng **DHCP**
  - Yêu cầu DHCP được đóng gói trong gói tin **UDP** -> được đóng gói tiếp trong gói tin **IP** -> tiếp theo là được đóng gói trong Ethernet **802.3**

# Yêu cầu dịch vụ web: kết nối với Internet

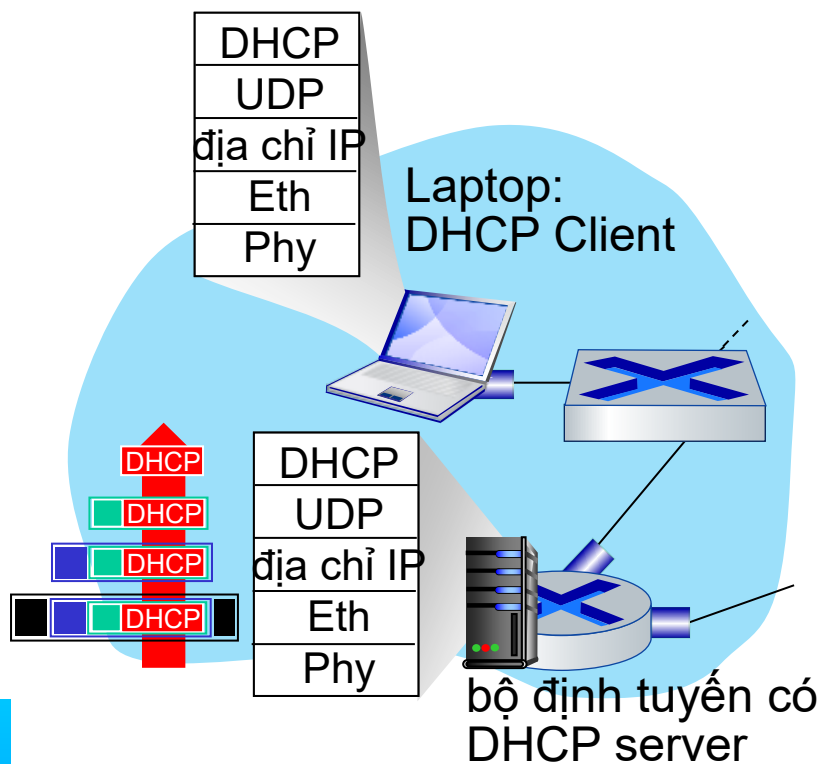


- Laptop của bạn kết nối mạng cần có địa chỉ IP cho nó, IP của default gateway, IP của máy chủ DNS: sử dụng **DHCP**
  - Yêu cầu DHCP được đóng gói trong gói tin **UDP** -> được đóng gói tiếp trong gói tin **IP** -> tiếp theo là được đóng gói trong Ethernet **802.3**
  - Gửi quảng bá Ethernet frame (đích: FFFFFFFFFFFFFFFF) trên mạng LAN, được nhận tại bộ định tuyến chạy máy chủ **DHCP**

# Yêu cầu dịch vụ web: kết nối với Internet

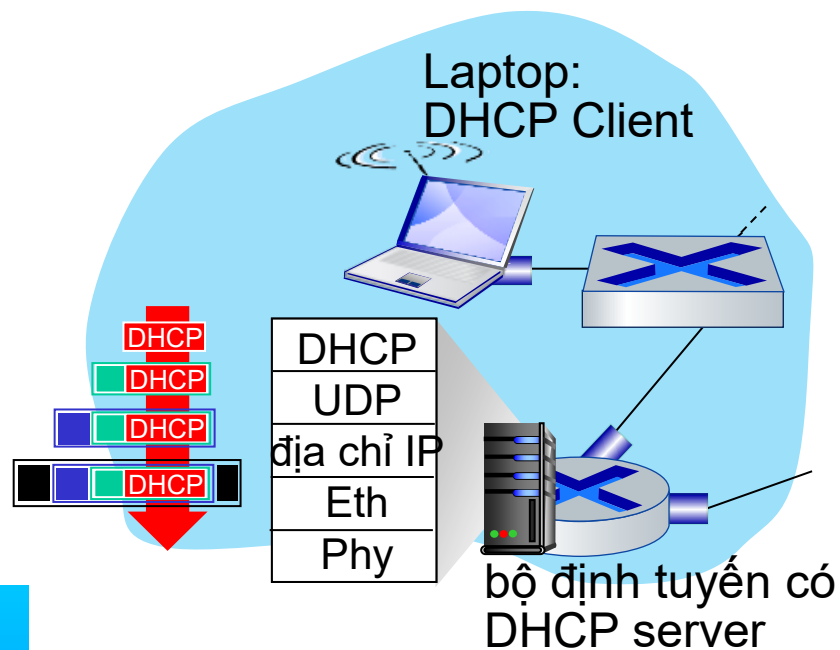


- Laptop của bạn kết nối mạng cần có địa chỉ IP cho nó, IP của default gateway, IP của máy chủ DNS: sử dụng **DHCP**



- Yêu cầu DHCP được đóng gói trong gói tin **UDP** -> được đóng gói tiếp trong gói tin **IP** -> tiếp theo là được đóng gói trong Ethernet **802.3**
- Gửi quảng bá Ethernet frame (đích: FFFFFFFFFFFFFFFF) trên mạng LAN, được nhận tại bộ định tuyến chạy máy chủ **DHCP**
- Tiếp theo Ethernet frame được tách thành gói IP, gói IP được tách thành gói UDP, gói UDP được tách thành DHCP

# Yêu cầu dịch vụ web: kết nối với Internet

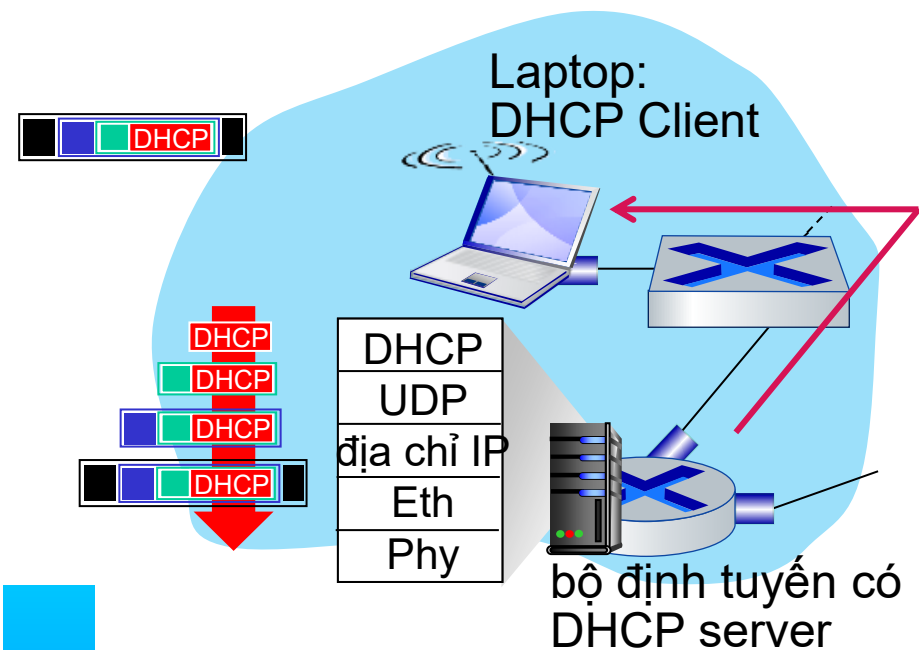


- Tại Máy chủ DHCP: tạo gói **DHCP ACK** chứa địa chỉ IP cấp cho máy khách, địa chỉ IP của default gateway, tên & địa chỉ IP của máy chủ DNS
- Đóng gói tại máy chủ DHCP, chuyển tiếp frame (qua bảng **chuyển đổi của switch**) qua mạng LAN, giải mã tại máy khách

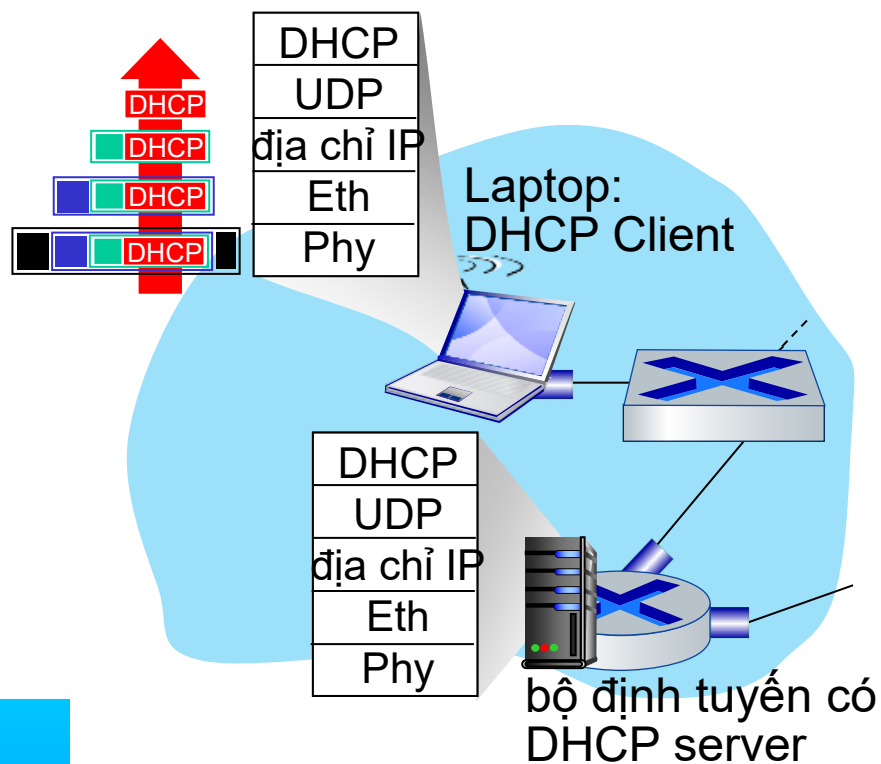
# Yêu cầu dịch vụ web: kết nối với Internet



- Tại Máy chủ DHCP: tạo gói **DHCP ACK** chứa địa chỉ IP cấp cho máy khách, địa chỉ IP của default gateway, tên & địa chỉ IP của máy chủ DNS
- Đóng gói tại máy chủ DHCP, chuyển tiếp frame (qua bảng **chuyển đổi của switch**) qua mạng LAN, giải mã tại máy khách



# Yêu cầu dịch vụ web: kết nối với Internet



- Tại Máy chủ DHCP: tạo gói **DHCP ACK** chứa địa chỉ IP cấp cho máy khách, địa chỉ IP của default gateway, tên & địa chỉ IP của máy chủ DNS
- Đóng gói tại máy chủ DHCP, chuyển tiếp frame (qua bảng **chuyển đổi của switch**) qua mạng LAN, giải mã tại máy khách
- Máy khách DHCP nhận gói tin phản hồi DHCP ACK

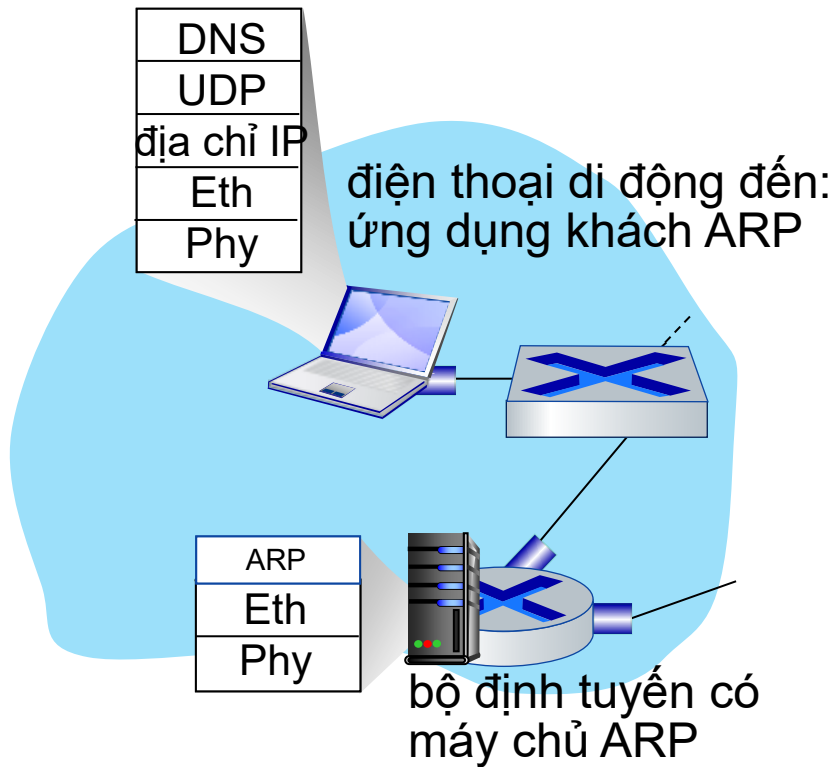
*Máy khách hiện có địa chỉ IP, biết tên và địa chỉ của máy chủ DNS, địa chỉ IP của default gateway của nó.*



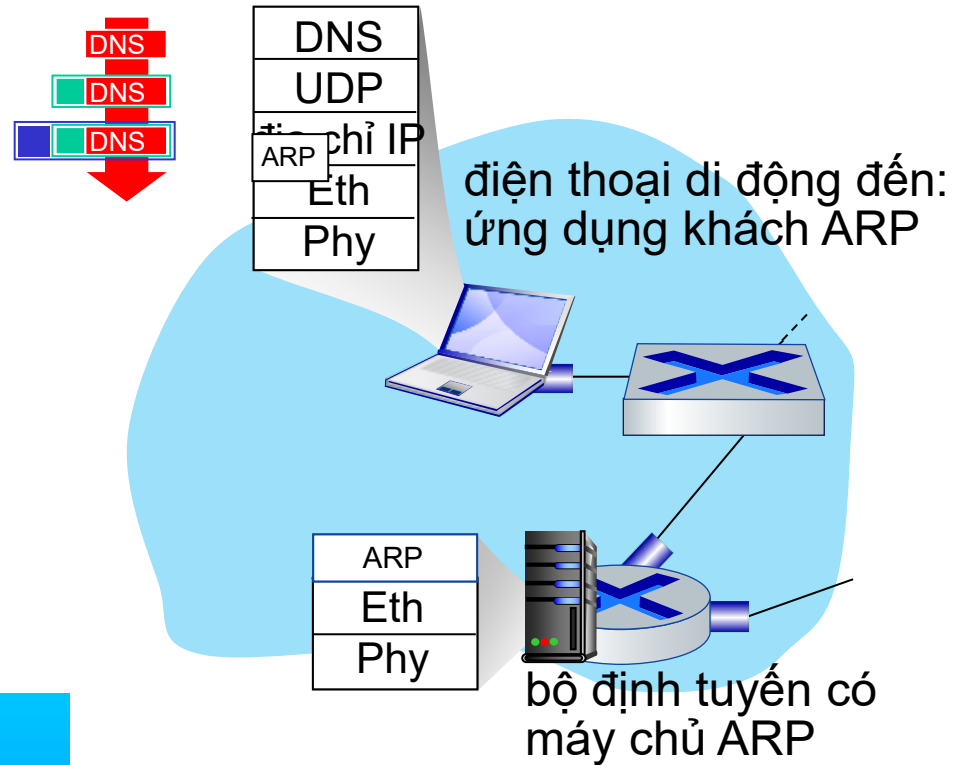
# Yêu cầu dịch vụ web ... ARP (trước DNS và trước HTTP)



- Trước khi gửi yêu cầu **HTTP**, cần địa chỉ IP của **www.google.com**: thông qua **DNS**

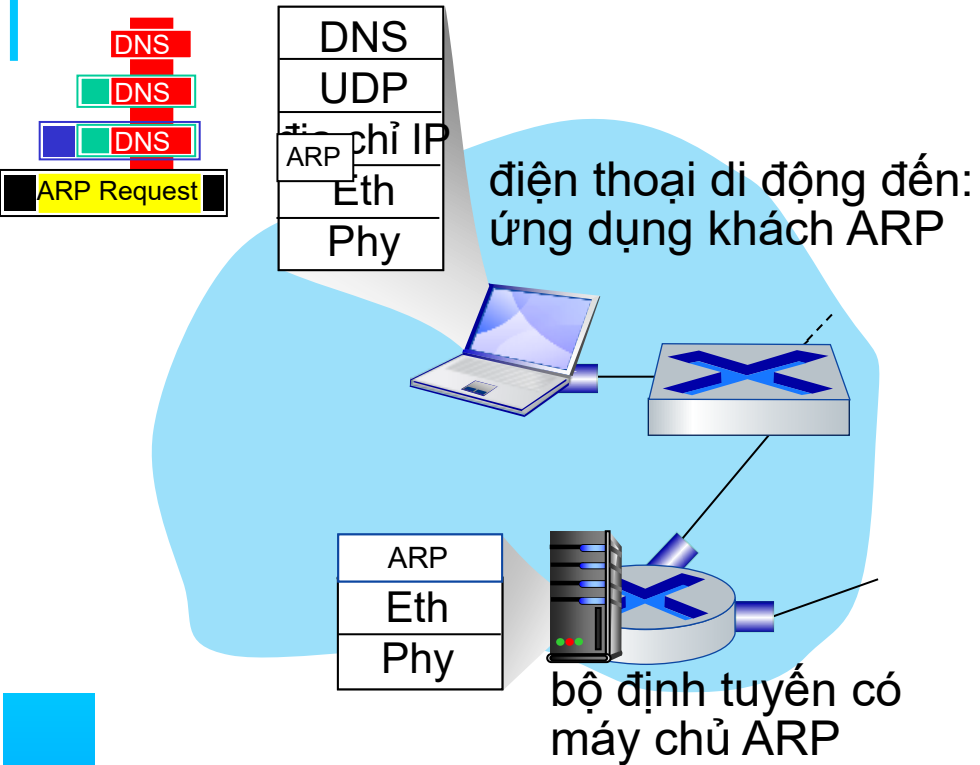


# Yêu cầu dịch vụ web ... ARP (trước DNS và trước HTTP)



- Trước khi gửi yêu cầu **HTTP**, cần địa chỉ IP của **www.google.com**: thông qua **DNS**
- Truy vấn DNS được tạo và đóng gói trong UDP -> tiếp theo đóng gói trong IP -> tiếp theo là đóng gói trong Eth. Để gửi frame tới bộ định tuyến (default gateway), cần có địa chỉ MAC của default gateway: thông qua **ARP**

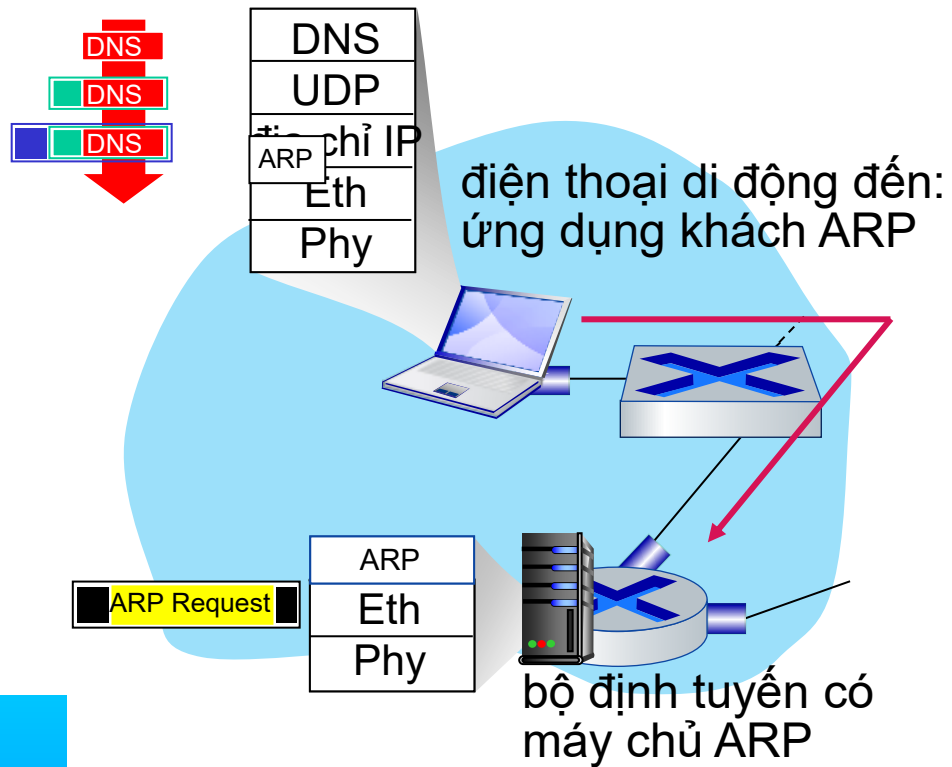
# Yêu cầu dịch vụ web ... ARP (trước DNS và trước HTTP)



- Trước khi gửi yêu cầu **HTTP**, cần địa chỉ IP của **www.google.com**: thông qua **DNS**
- Truy vấn DNS được tạo và đóng gói trong UDP -> tiếp theo đóng gói trong IP -> tiếp theo là đóng gói trong Eth. Để gửi frame tới bộ định tuyến (default gateway), cần có địa chỉ MAC của default gateway: thông qua **ARP**

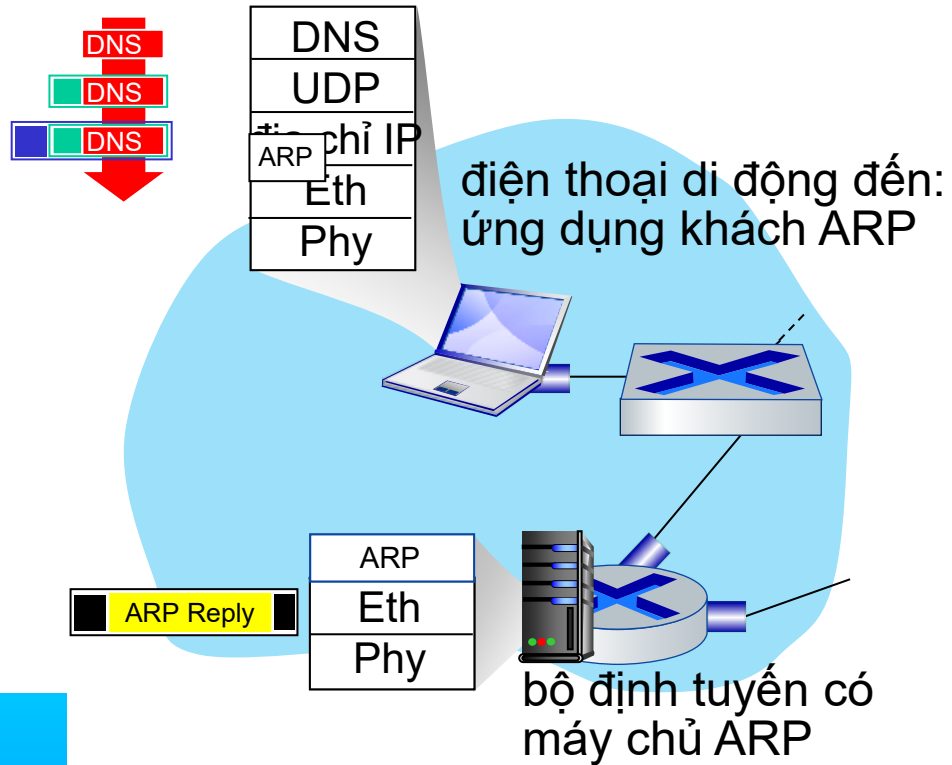


# Yêu cầu dịch vụ web ... ARP (trước DNS và trước HTTP)



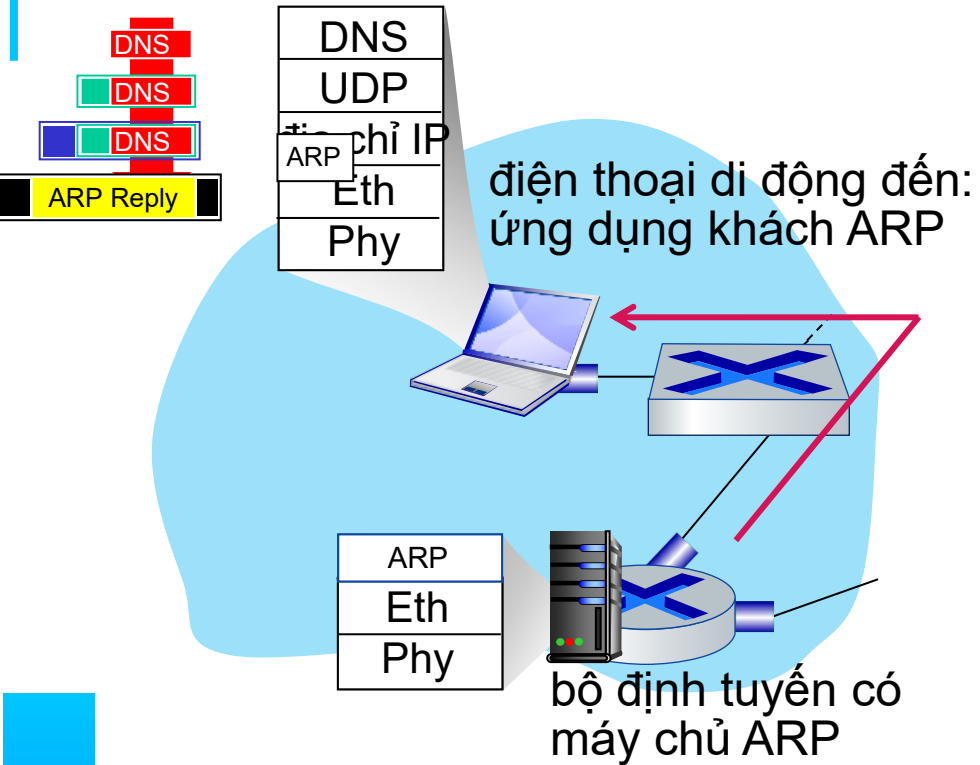
- Trước khi gửi yêu cầu **HTTP**, cần địa chỉ IP của **www.google.com**: thông qua **DNS**
- Truy vấn DNS được tạo và đóng gói trong UDP -> tiếp theo đóng gói trong IP -> tiếp theo là đóng gói trong Eth. Để gửi frame tới bộ định tuyến (default gateway), cần có địa chỉ MAC của default gateway: thông qua **ARP**
- **Bản tin ARP Request** được quảng bá, được bộ định tuyến nhận và trả lời bằng **ARP Reply** sẽ cung cấp địa chỉ MAC của cổng default gateway

# Yêu cầu dịch vụ web ... ARP (trước DNS và trước HTTP)



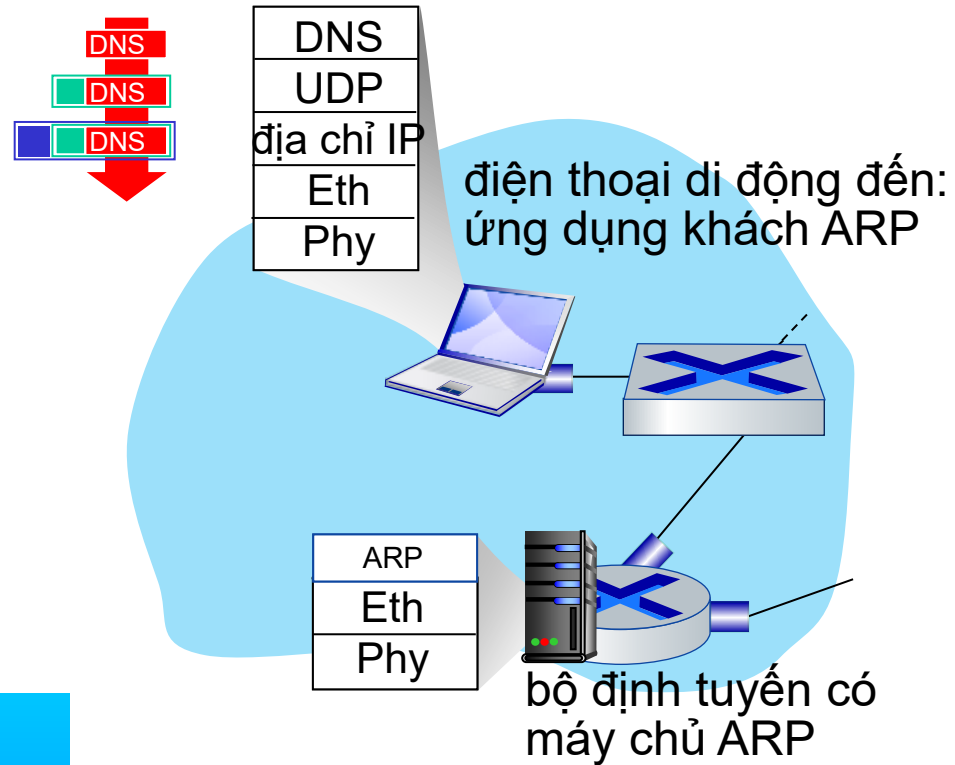
- Trước khi gửi yêu cầu **HTTP**, cần địa chỉ IP của **www.google.com**: thông qua **DNS**
- Truy vấn DNS được tạo và đóng gói trong UDP -> tiếp theo đóng gói trong IP -> tiếp theo là đóng gói trong Eth. Để gửi frame tới bộ định tuyến (default gateway), cần có địa chỉ MAC của default gateway: thông qua **ARP**
- **Bản tin ARP Request** được quảng bá, được bộ định tuyến nhận và trả lời bằng **ARP Reply** sẽ cung cấp địa chỉ MAC của cổng default gateway

# Yêu cầu dịch vụ web ... ARP (trước DNS và trước HTTP)



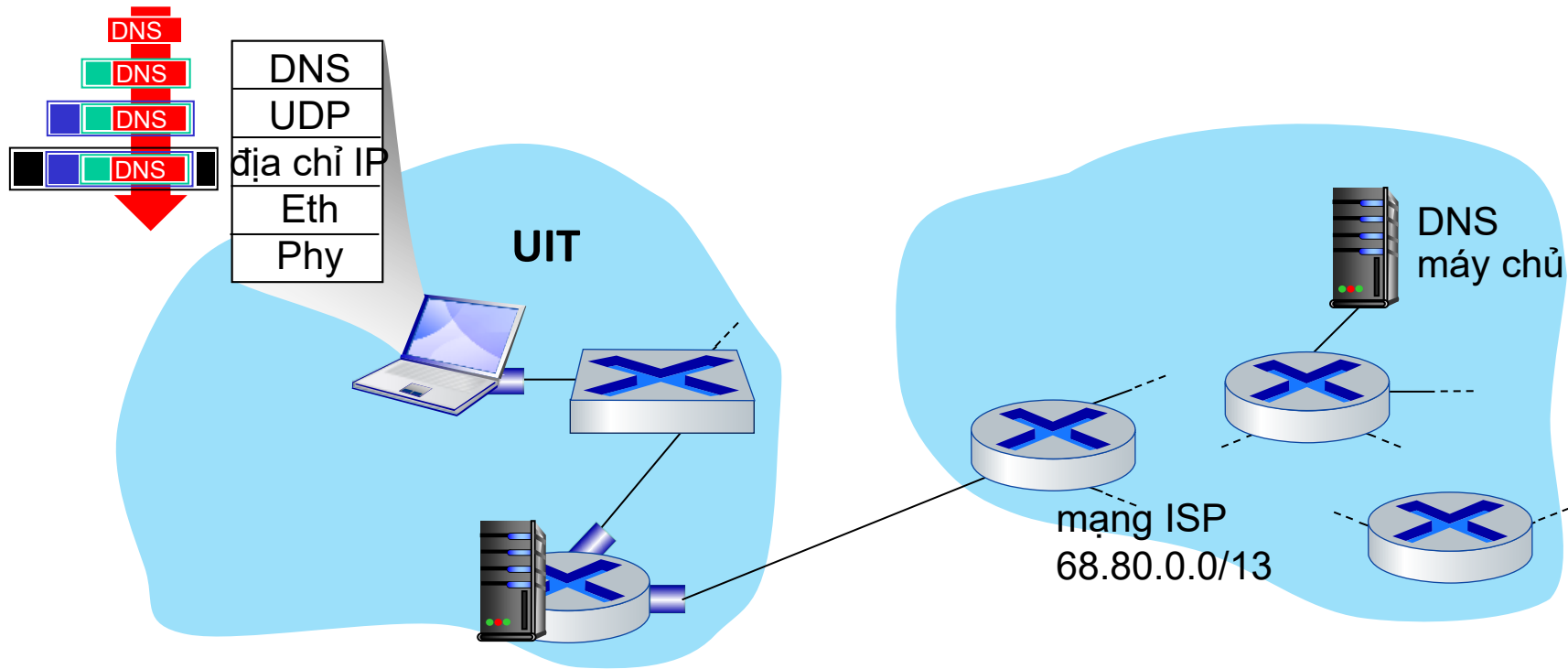
- Trước khi gửi yêu cầu **HTTP**, cần địa chỉ IP của **www.google.com**: thông qua **DNS**
- Truy vấn DNS được tạo và đóng gói trong UDP -> tiếp theo đóng gói trong IP -> tiếp theo là đóng gói trong Eth. Để gửi frame tới bộ định tuyến (default gateway), cần có địa chỉ MAC của default gateway: thông qua **ARP**
- **Bản tin ARP Request** được quảng bá, được bộ định tuyến nhận và trả lời bằng **ARP Reply** sẽ cung cấp địa chỉ MAC của cổng default gateway

# Yêu cầu dịch vụ web ... ARP (trước DNS và trước HTTP)



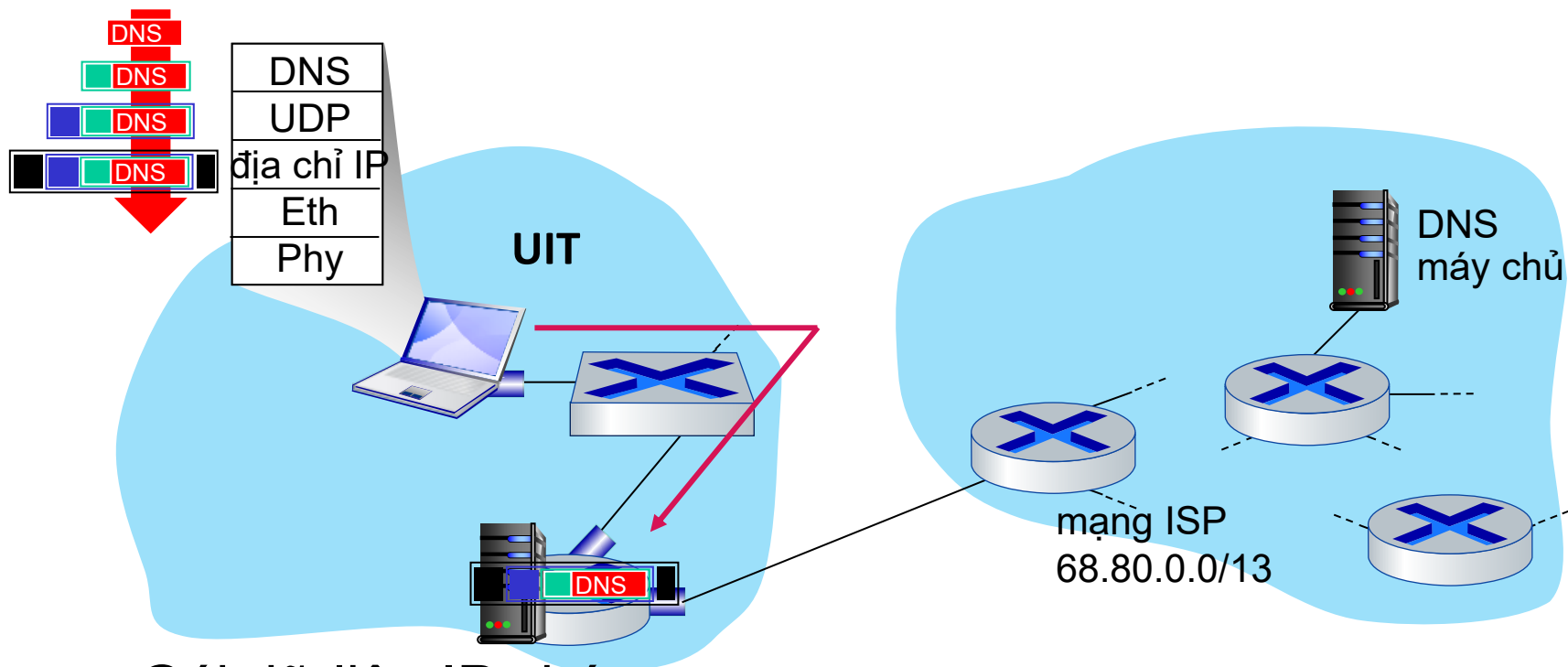
- Trước khi gửi yêu cầu **HTTP**, cần địa chỉ IP của **www.google.com**: thông qua **DNS**
- Truy vấn DNS được tạo và đóng gói trong UDP -> tiếp theo đóng gói trong IP -> tiếp theo là đóng gói trong Eth. Để gửi frame tới bộ định tuyến (default gateway), cần có địa chỉ MAC của default gateway: thông qua **ARP**
- **Bản tin ARP Request** được quảng bá, được bộ định tuyến nhận và trả lời bằng **ARP Reply** sẽ cung cấp địa chỉ MAC của cổng default gateway
- Máy khách giờ đã biết địa chỉ MAC của cổng default gateway, vì vậy bây giờ có thể gửi frame chứa truy vấn DNS

# Yêu cầu dịch vụ web... sử dụng DNS



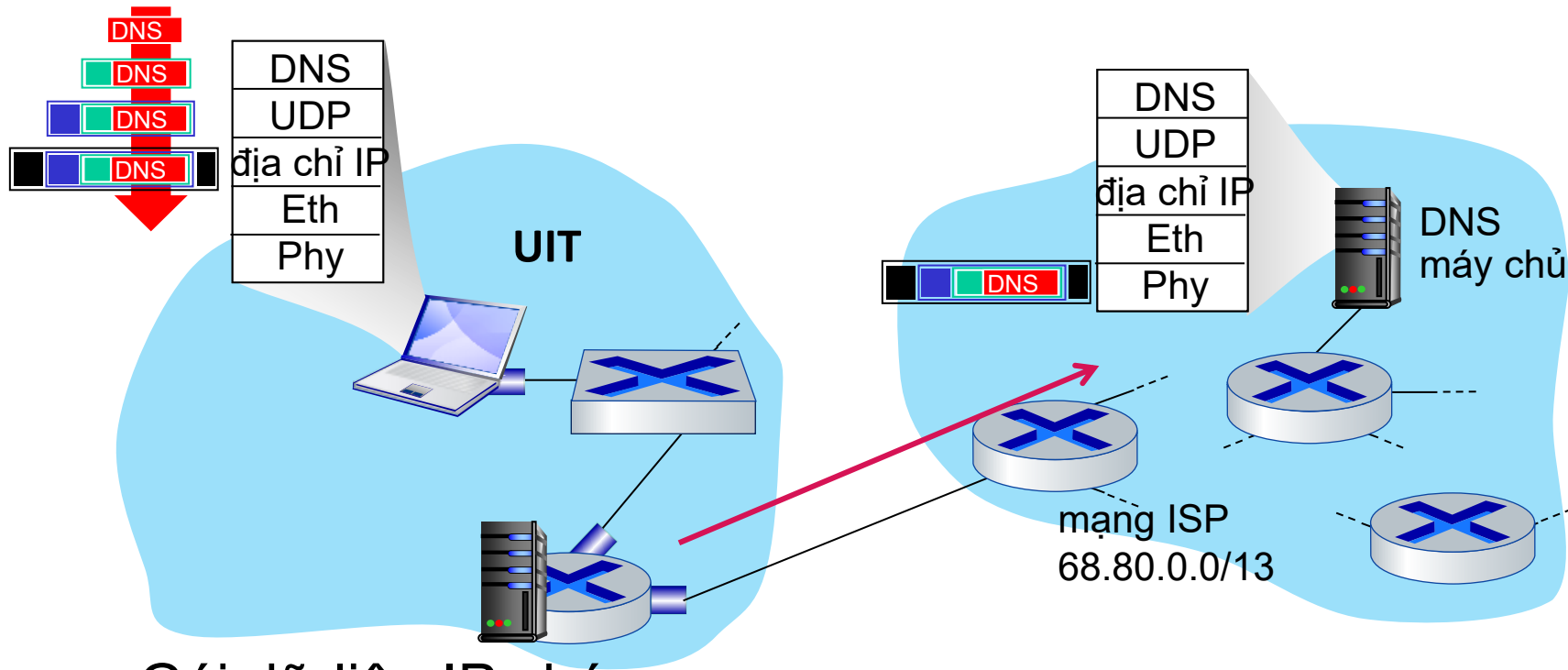


# Yêu cầu dịch vụ web... sử dụng DNS



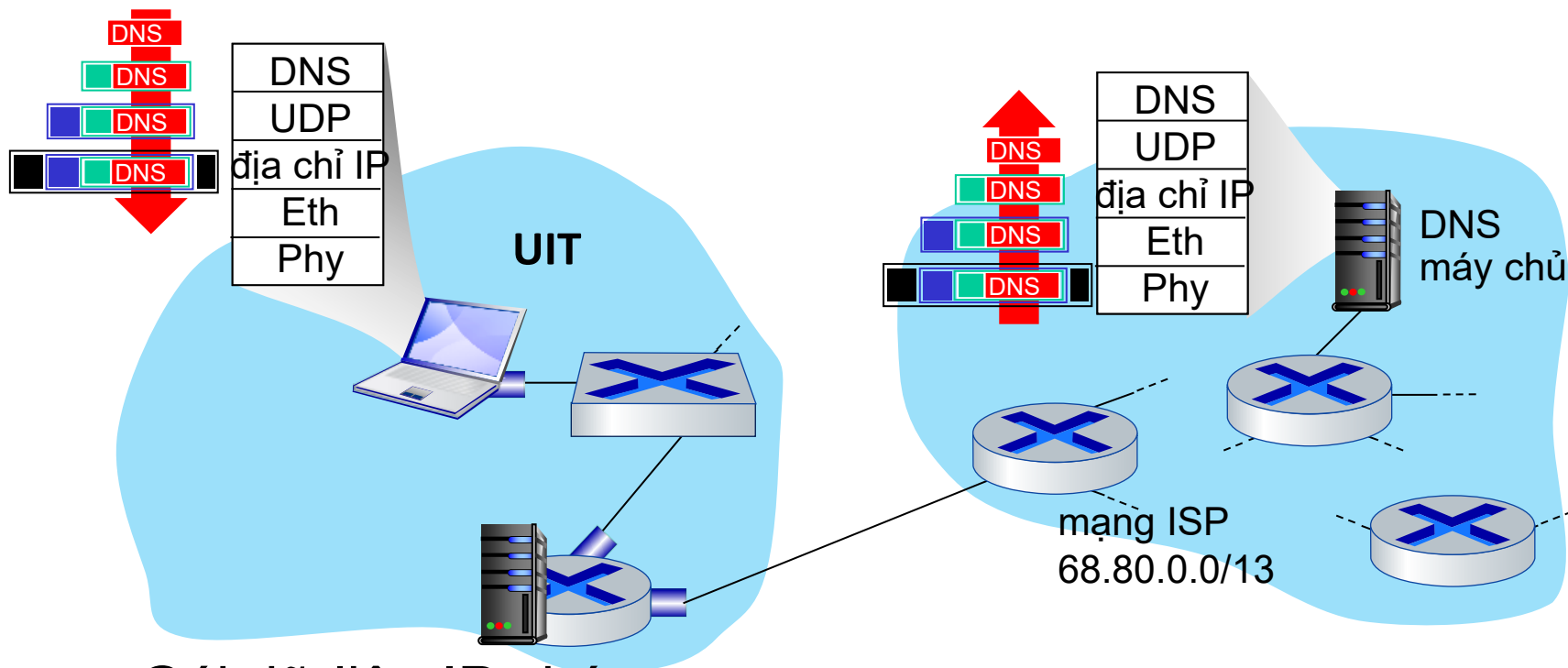
- Gói dữ liệu IP chứa truy vấn DNS được chuyển tiếp qua switch của mạng LAN từ máy khách đến cổng của default gateway

# Yêu cầu dịch vụ web... sử dụng DNS



- Gói dữ liệu IP chứa truy vấn DNS được chuyển tiếp qua switch của mạng LAN từ máy khách đến cổng của default gateway
- Gói dữ liệu IP được chuyển tiếp từ mạng UIT vào mạng ISP (viettele hoặc FPT), được định tuyến (các bảng được tạo bởi các giao thức định tuyến **RIP**, **OSPF**, **IS-IS** và/hoặc **BGP**) đến máy chủ DNS

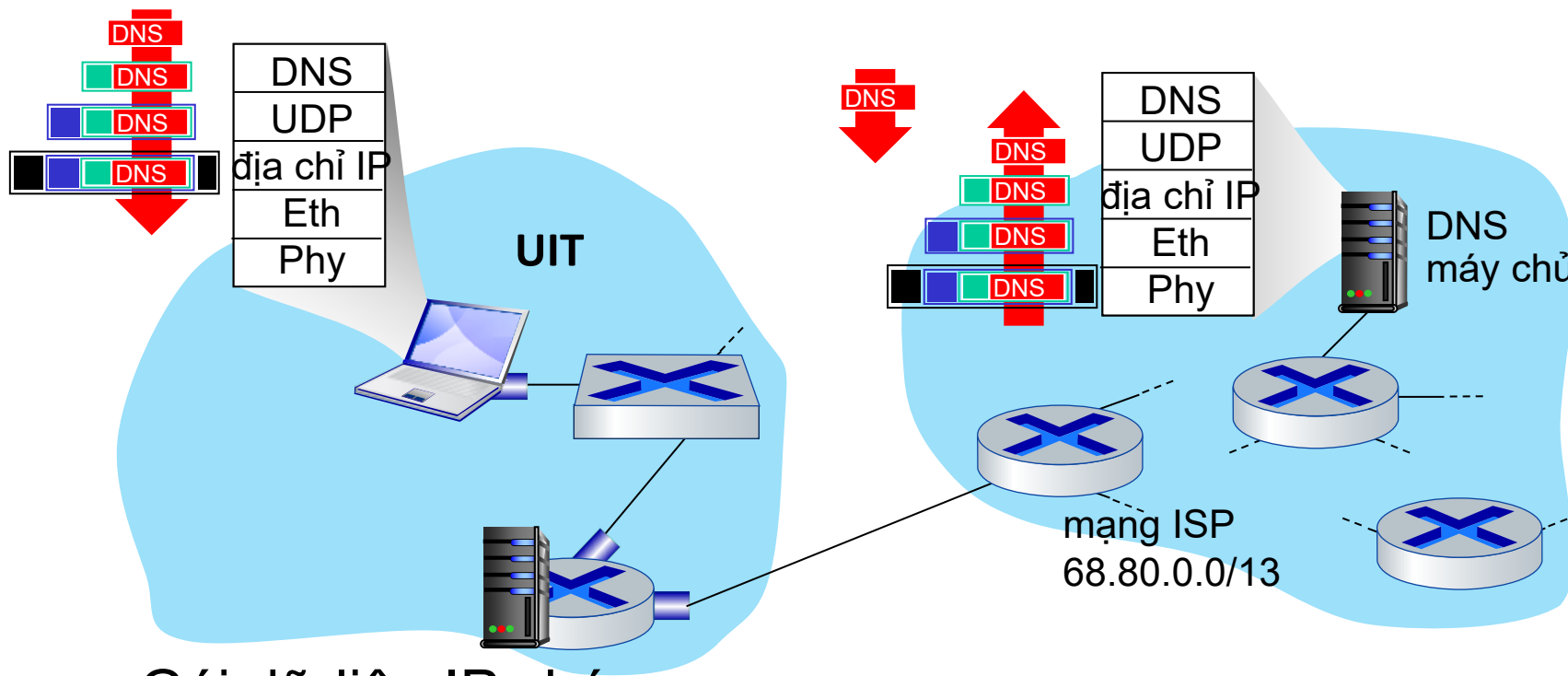
# Yêu cầu dịch vụ web... sử dụng DNS



- Trích xuất bản tin DNS

- Gói dữ liệu IP chứa truy vấn DNS được chuyển tiếp qua switch của mạng LAN từ máy khách đến cổng của default gateway
- Gói dữ liệu IP được chuyển tiếp từ mạng UIT vào mạng ISP (viettele hoặc FPT), được định tuyến (các bảng được tạo bởi các giao thức định tuyến **RIP, OSPF, IS-IS** và/hoặc **BGP**) đến máy chủ DNS

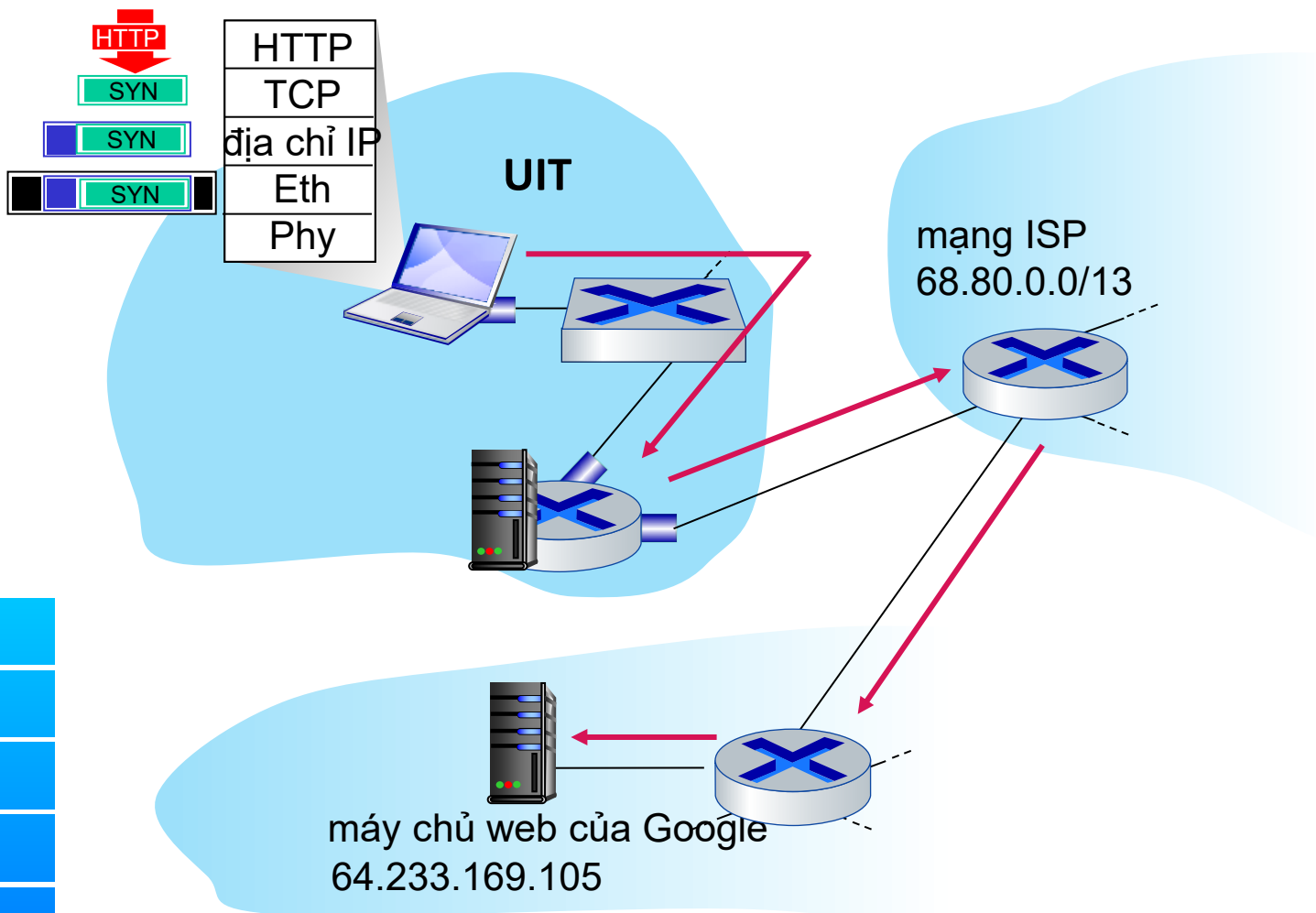
# Yêu cầu dịch vụ web... sử dụng DNS



- Trích xuất bản tin DNS
- DNS trả lời khách hàng bằng địa chỉ IP của [www.google.com](http://www.google.com)

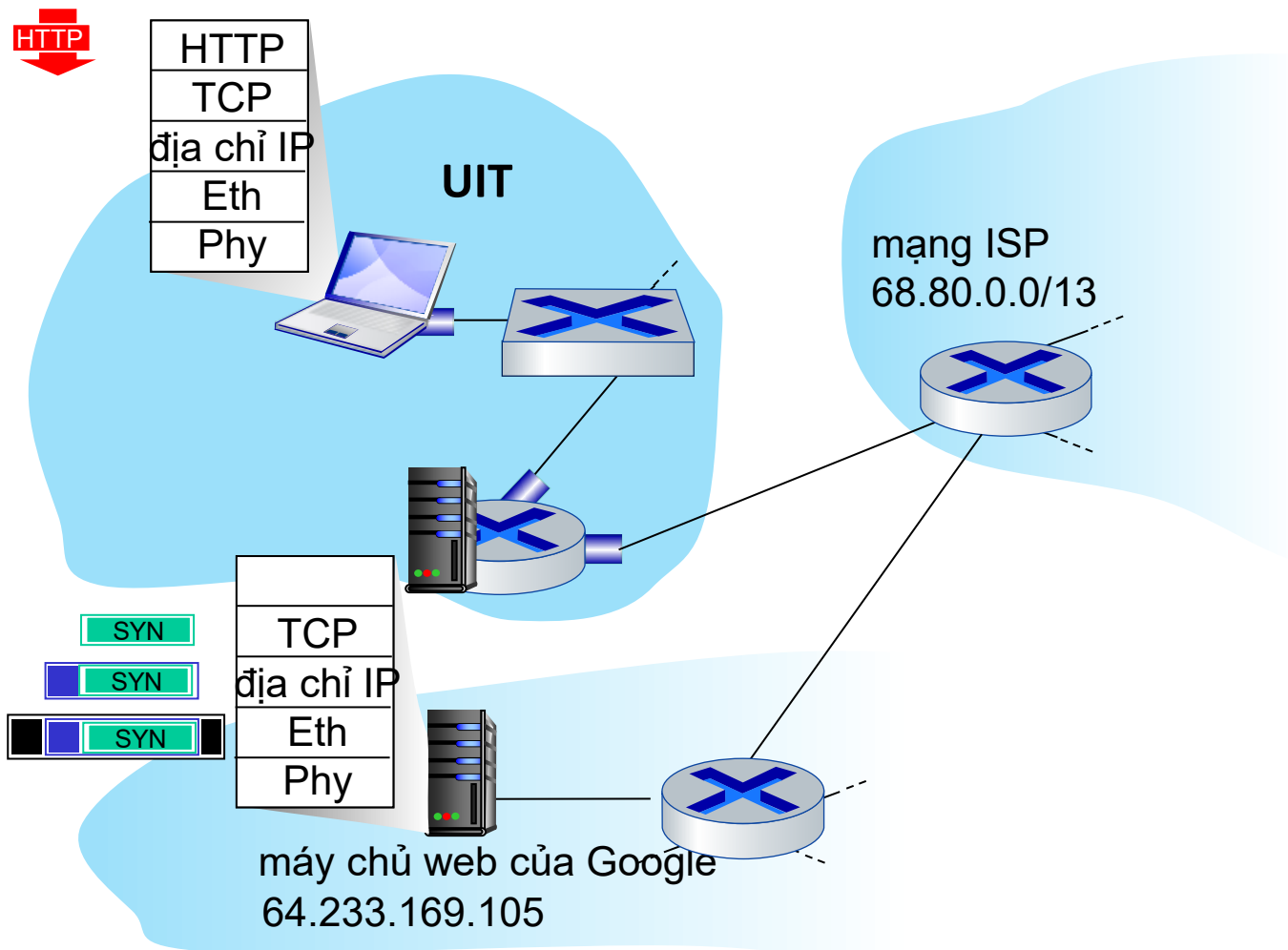
- Gói dữ liệu IP chứa truy vấn DNS được chuyển tiếp qua switch của mạng LAN từ máy khách đến cổng của default gateway
- Gói dữ liệu IP được chuyển tiếp từ mạng UIT vào mạng ISP (viettele hoặc FPT), được định tuyến (các bảng được tạo bởi các giao thức định tuyến **RIP**, **OSPF**, **IS-IS** và/hoặc **BGP**) đến máy chủ DNS

# Yêu cầu dịch vụ web ...Kết nối TCP cho HTTP



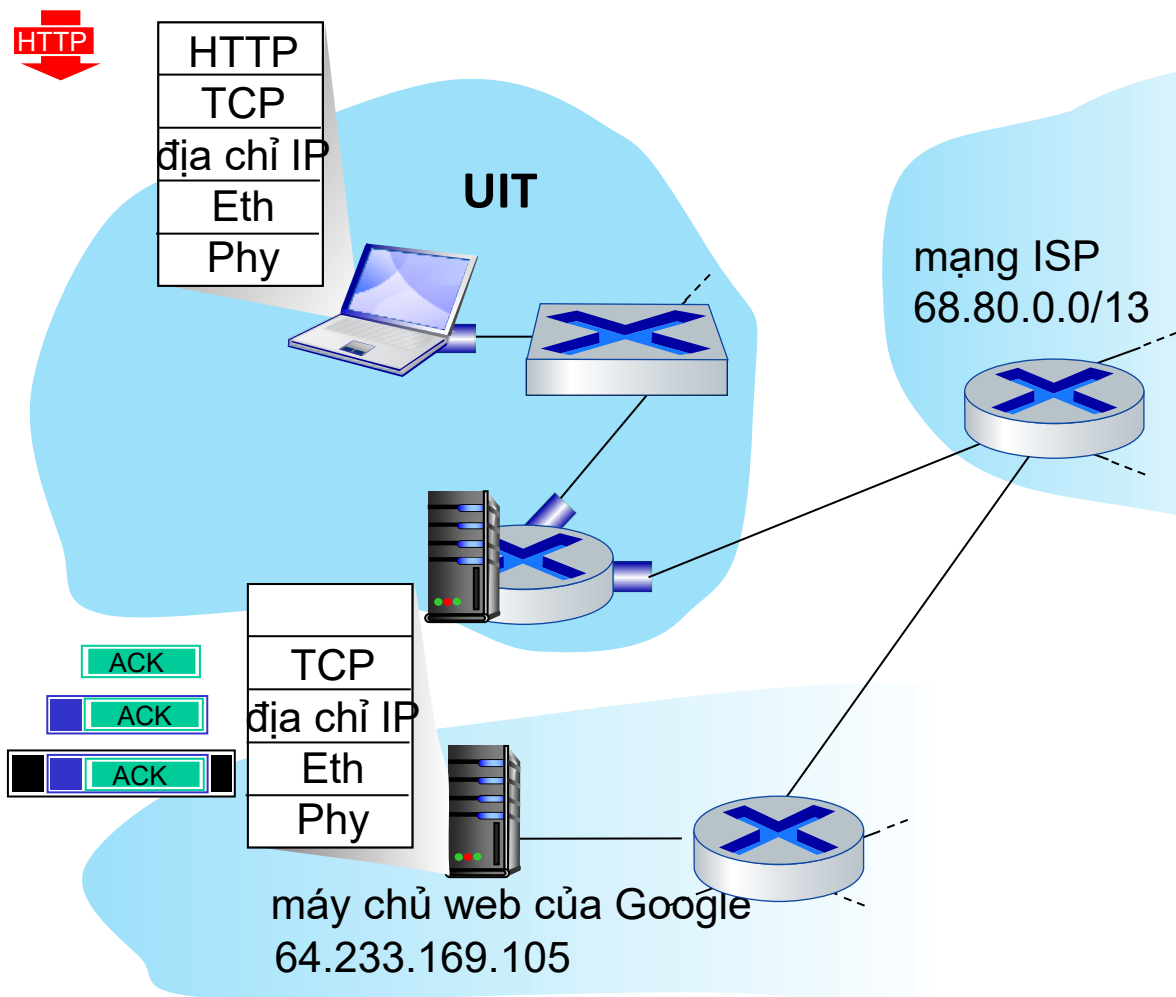
- Để gửi yêu cầu HTTP, trước tiên, máy khách sẽ mở **TCP socket** tới máy chủ web

# Yêu cầu dịch vụ web ...Kết nối TCP cho HTTP



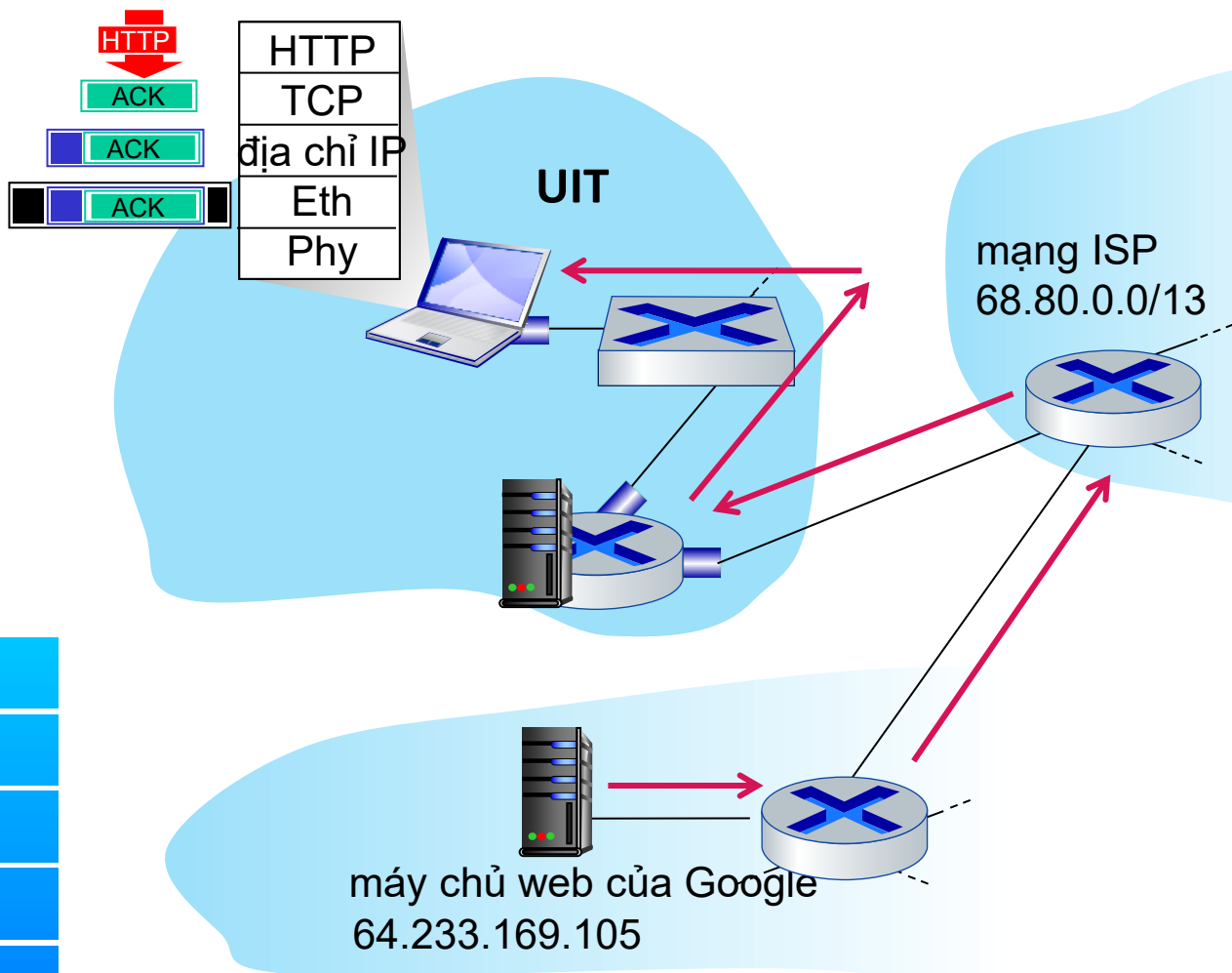
- Để gửi yêu cầu HTTP, trước tiên, máy khách sẽ mở **TCP socket** tới máy chủ web
- Gói tin TCP **SYN** (bước 1 trong bắt tay 3 bước TCP) được định tuyến đến máy chủ web

# Yêu cầu dịch vụ web ...Kết nối TCP cho HTTP



- Để gửi yêu cầu HTTP, trước tiên, máy khách sẽ mở **TCP socket** tới máy chủ web
- Gói tin TCP **SYN** (bước 1 trong bắt tay 3 bước TCP) được định tuyến đến máy chủ web
- Máy chủ web phản hồi với **TCP SYNACK** (bước 2 trong quá trình bắt tay 3 bước của TCP)

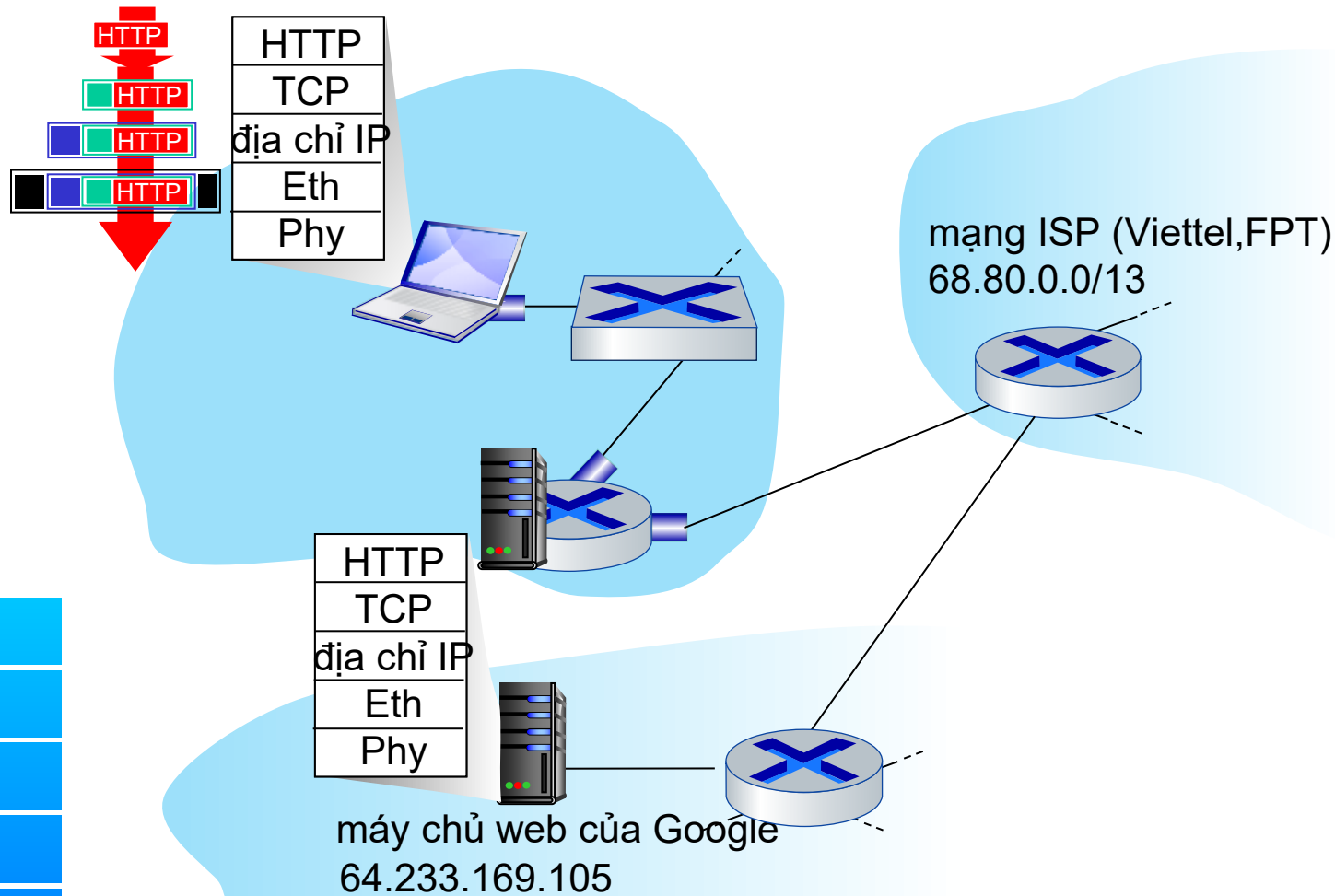
# Yêu cầu dịch vụ web ...Kết nối TCP cho HTTP



- Để gửi yêu cầu HTTP, trước tiên, máy khách sẽ mở **TCP socket** tới máy chủ web
- Gói tin TCP **SYN** (bước 1 trong bắt tay 3 bước TCP) được định tuyến đến máy chủ web
- Máy chủ web phản hồi với **TCP SYNACK** (bước 2 trong quá trình bắt tay 3 bước của TCP)
- Kết nối TCP được thiết lập !

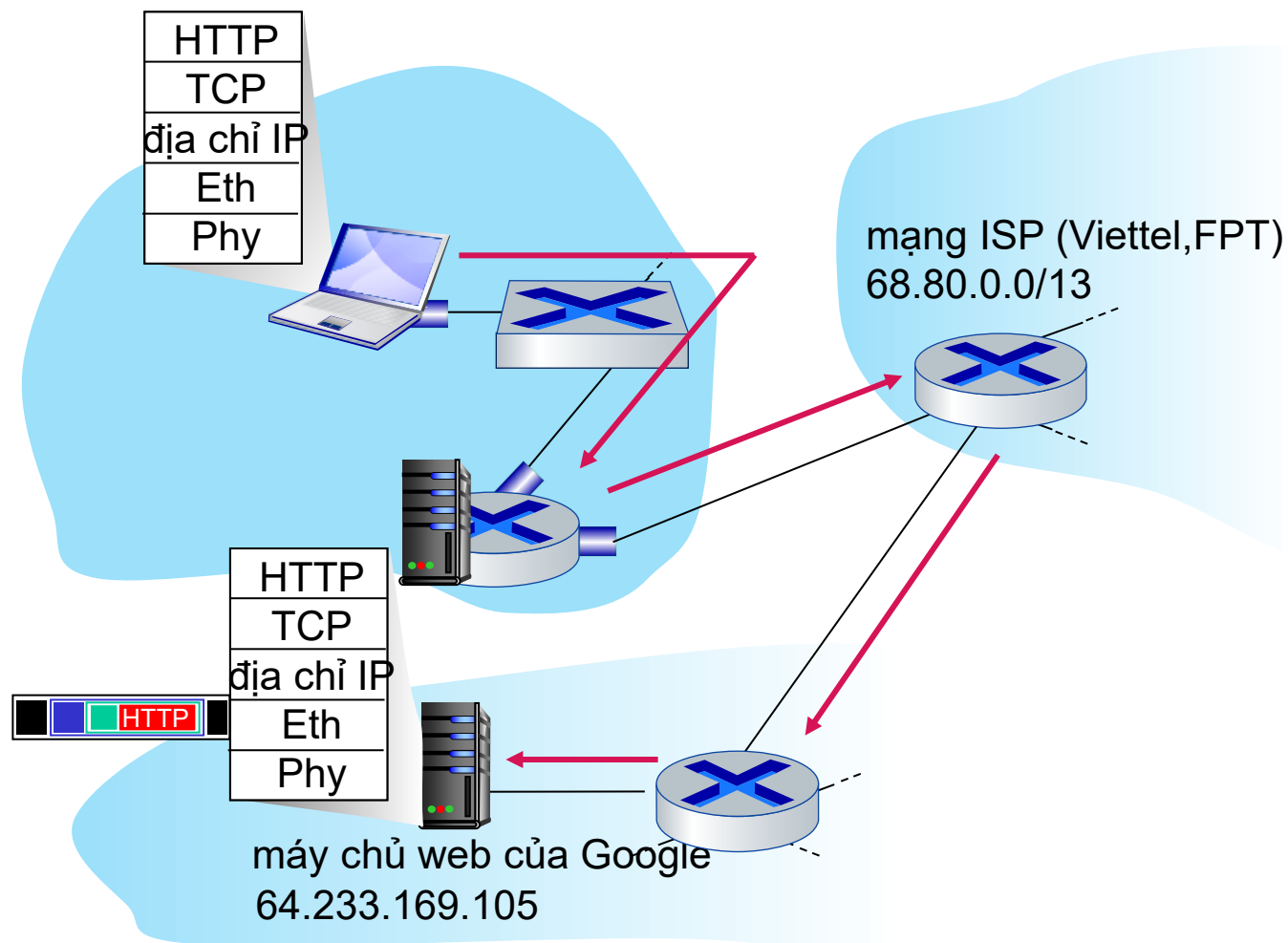


# Yêu cầu dịch vụ web ... Yêu cầu/trả lời HTTP



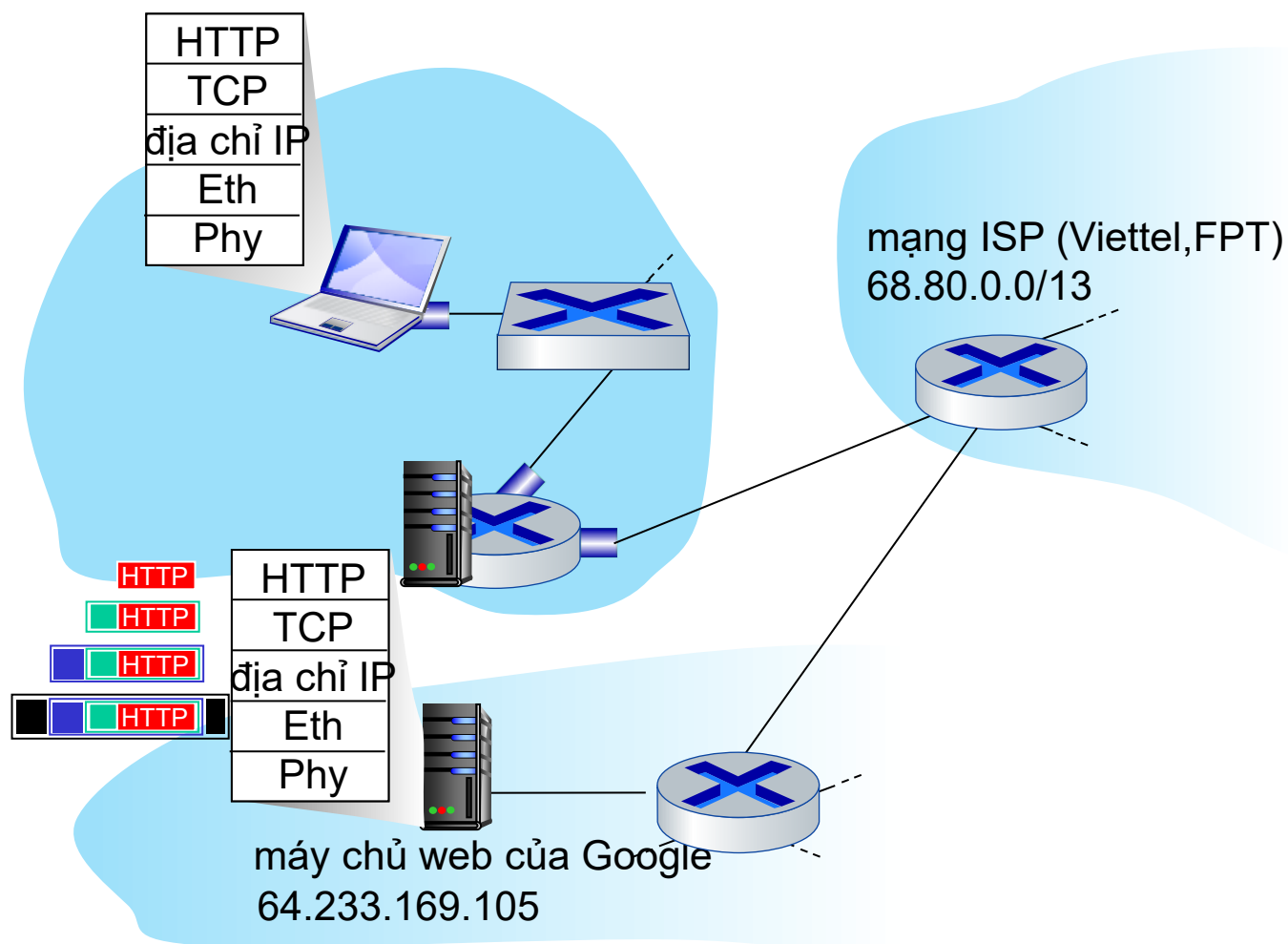
- **HTTP Request** được gửi vào TCP socket

# Yêu cầu dịch vụ web ... Yêu cầu/trả lời HTTP



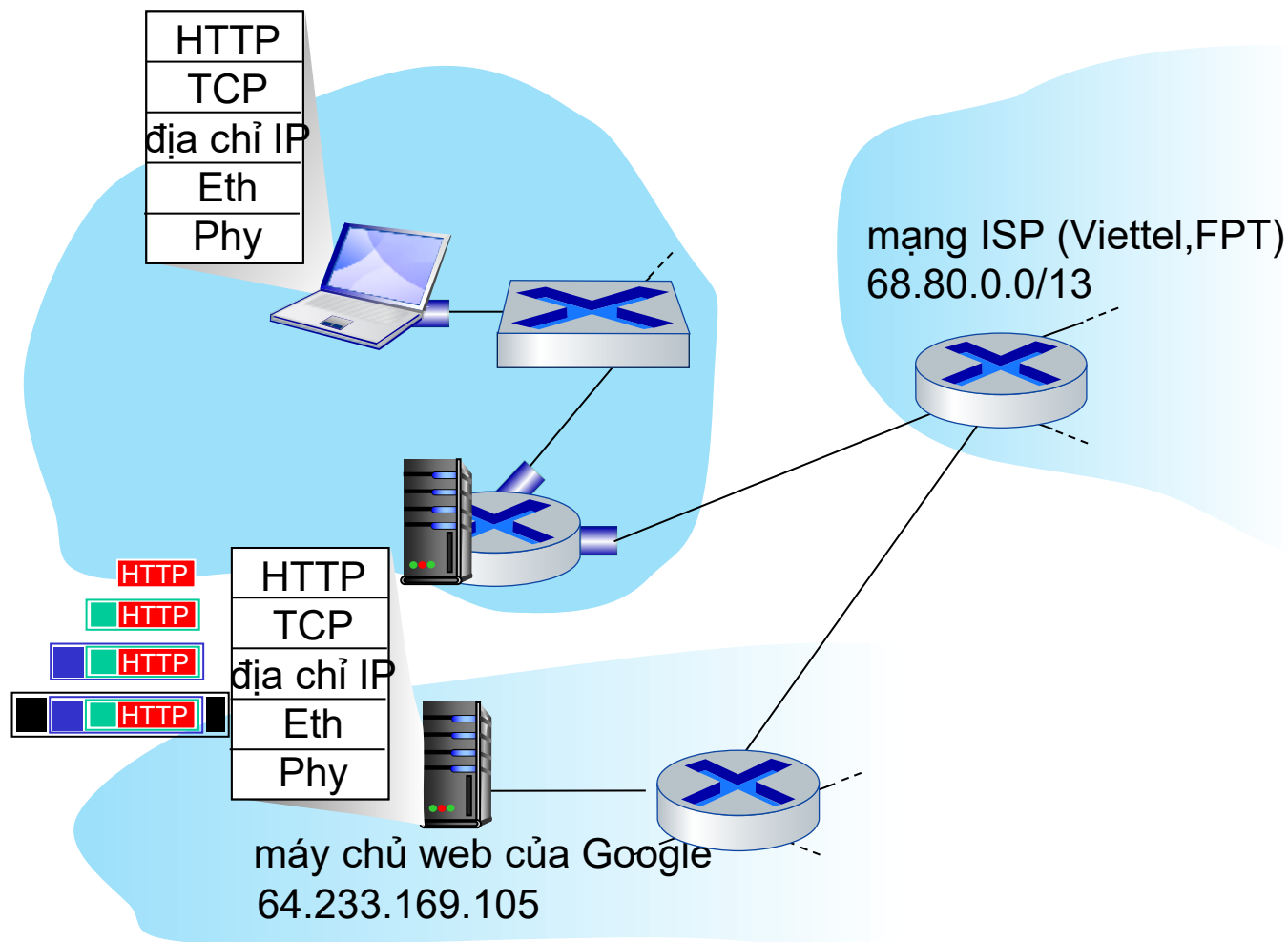
- **HTTP Request** được gửi vào TCP socket
- Gói dữ liệu IP chứa yêu cầu HTTP được định tuyến tới [www.google.com](http://www.google.com)

# Yêu cầu dịch vụ web ... Yêu cầu/trả lời HTTP



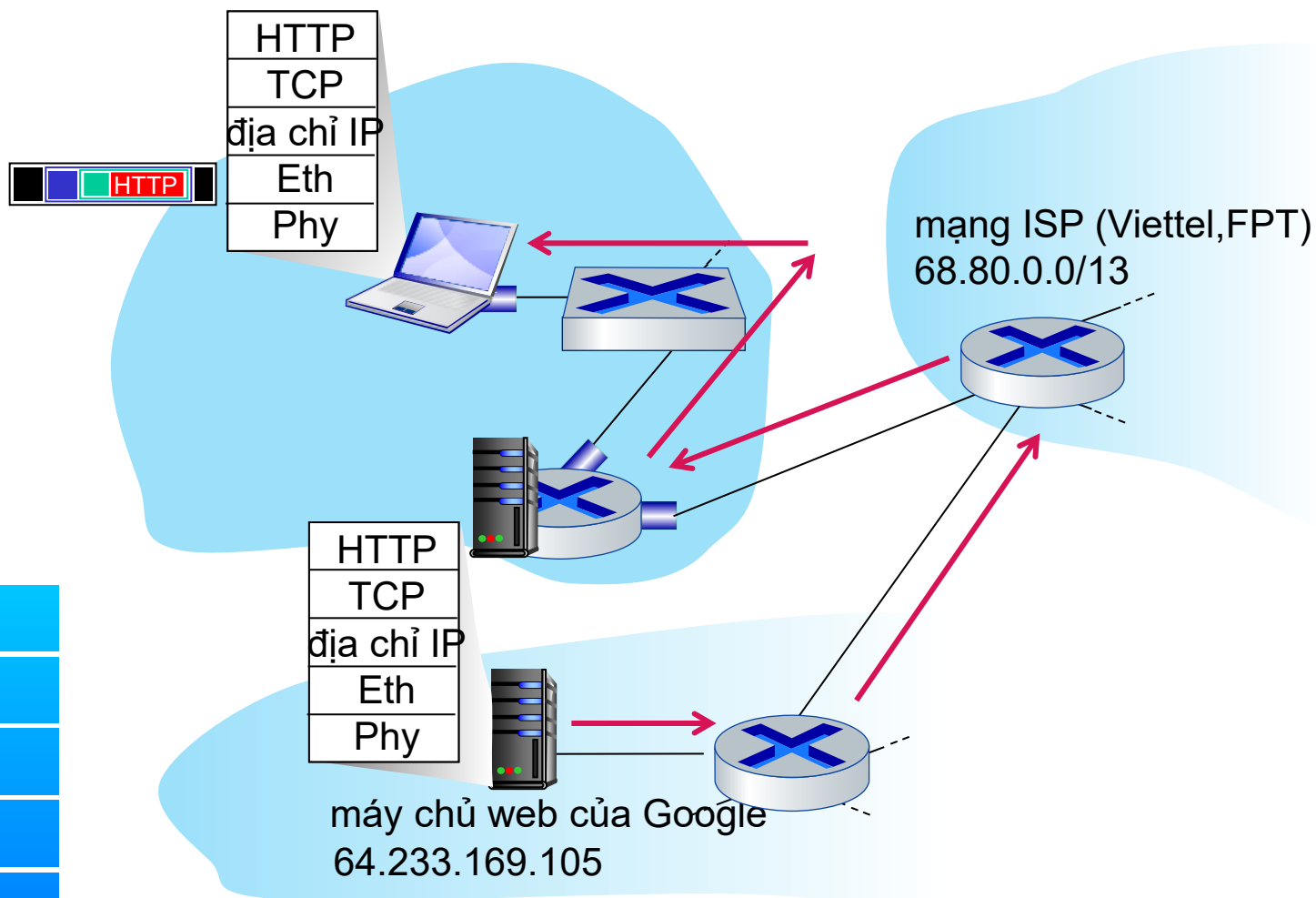
- **HTTP Request** được gửi vào TCP socket
- Gói dữ liệu IP chứa yêu cầu HTTP được định tuyến tới [www.google.com](http://www.google.com)

# Yêu cầu dịch vụ web ... Yêu cầu/trả lời HTTP



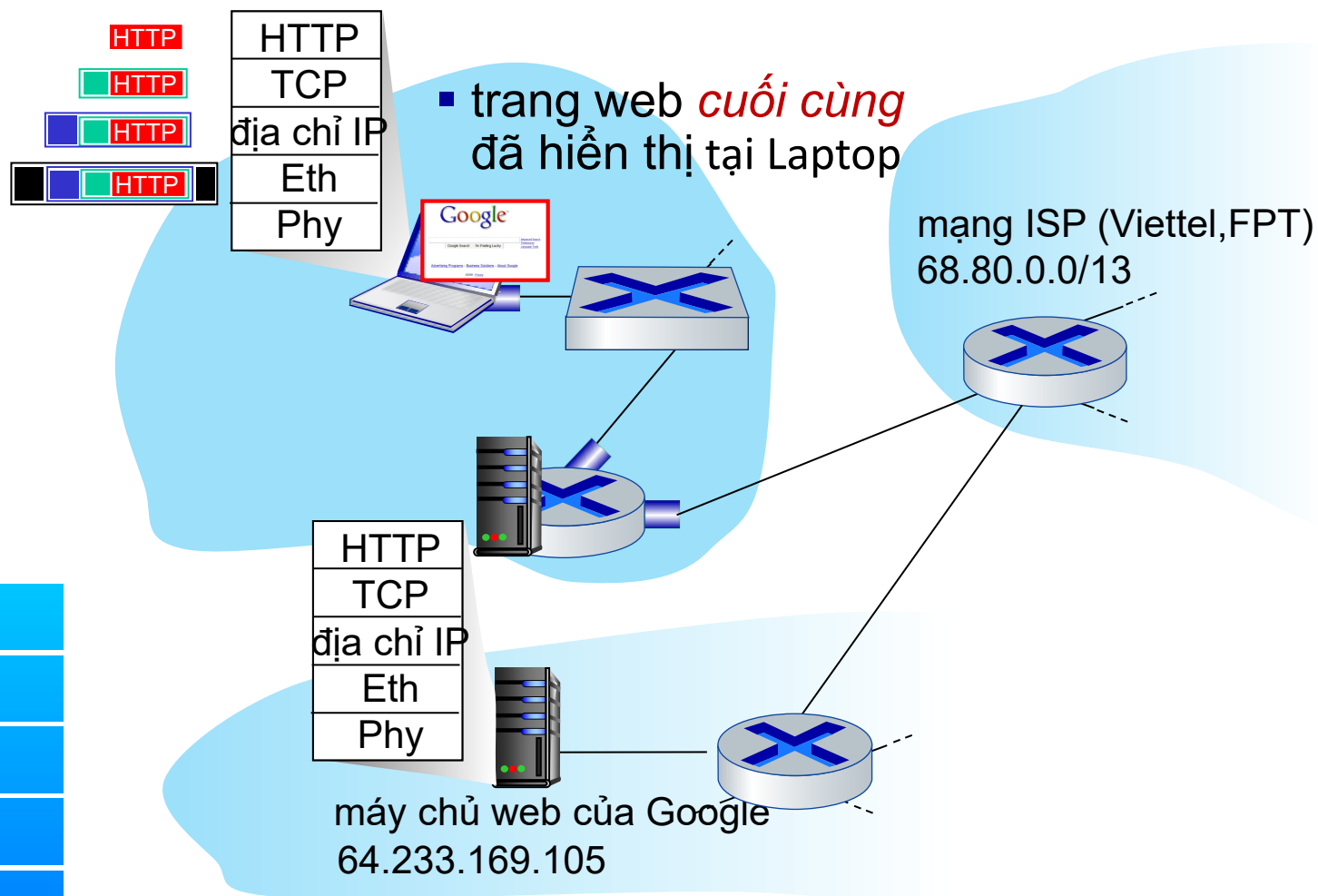
- **HTTP Request** được gửi vào TCP socket
- Gói dữ liệu IP chứa yêu cầu HTTP được định tuyến tới [www.google.com](http://www.google.com)
- Máy chủ web phản hồi bằng **HTTP Response** (chứa trang web)

# Yêu cầu dịch vụ web ... Yêu cầu/trả lời HTTP



- **HTTP Request** được gửi vào TCP socket
- Gói dữ liệu IP chứa yêu cầu HTTP được định tuyến tới [www.google.com](http://www.google.com)
- Máy chủ web phản hồi bằng **HTTP Response** (chứa trang web)
- Gói dữ liệu IP chứa phản hồi HTTP được định tuyến trở lại máy khách

# Yêu cầu dịch vụ web ... Yêu cầu/trả lời HTTP



- **HTTP Request** được gửi vào TCP socket
- Gói dữ liệu IP chứa yêu cầu HTTP được định tuyến tới [www.google.com](http://www.google.com)
- Máy chủ web phản hồi bằng **HTTP Response** (chứa trang web)
- Gói dữ liệu IP chứa phản hồi HTTP được định tuyến trở lại máy khách

# Chương 5: Tóm tắt



- Nguyên tắc đằng sau các dịch vụ tầng liên kết dữ liệu:
  - Phát hiện lỗi, sửa lỗi
  - Chia sẻ một kênh quảng bá: đa truy cập
  - Địa chỉ lớp liên kết
- Khởi tạo, triển khai các công nghệ tầng liên kết khác nhau
  - Ethernet
  - Chuyển mạng LAN, VLAN
- Tổng hợp: việc thường ngày yêu cầu một trang Web